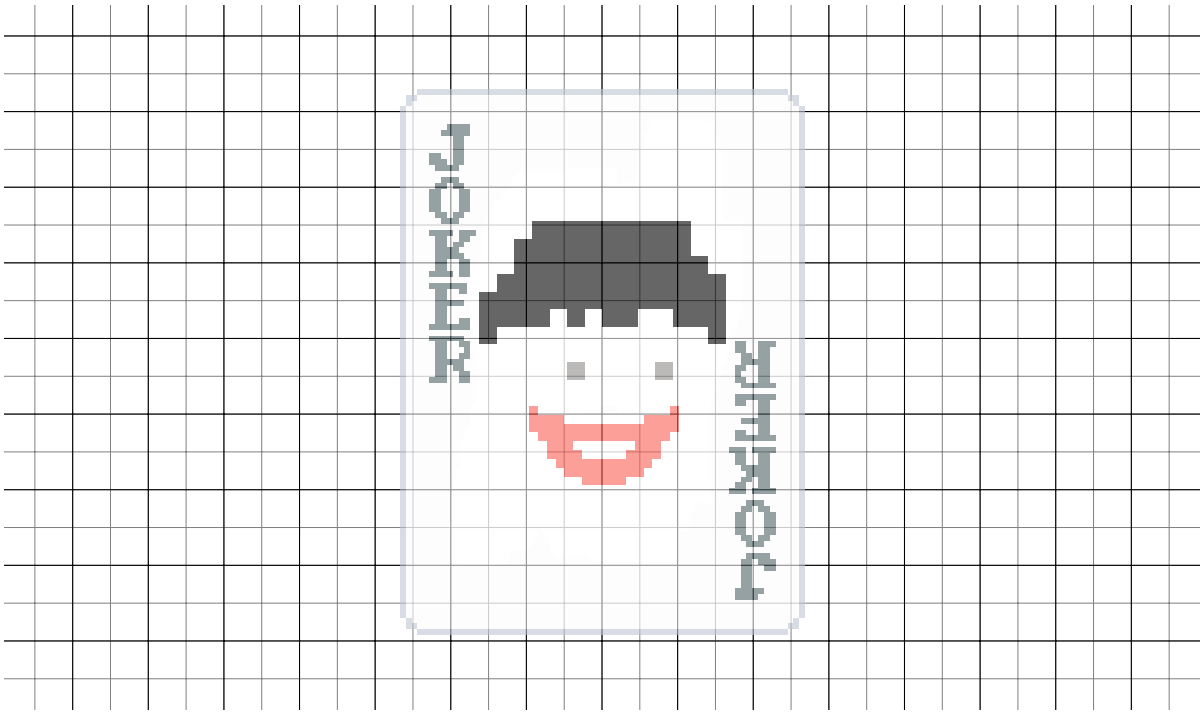
上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

ICPC World Finals 2024
Standard Code Library

Nemesis



Coach	教练
Yong Yu	俞勇
Siyu Sun	孙司宇
Boren Tan	谭博仁
Shangfei Yang	杨尚霏



Contestant	队员
Zonghan Yang	杨宗翰
Mingchi Zhang	张明驰
Jianjun Zhang	张建军

Last Commit: Nov 20, 2024 (851cd5: add rainbow-km)

Contents

1 Geometry 3
1.1 二维几何基础操作 3
1.1.1 点、向量与直线 3
1.1.2 点积、叉积与方向 3
1.1.3 浮点误差 3
1.1.4 点在线段上与线段判交 3
1.1.5 直线求交 3
1.1.6 点到直线与线段距离 3
1.2 凸包与闵可夫斯基和 3
1.3 直线半平面交 3
1.4 整数半平面交 4
1.5 凸包询问：凸包内、切点、交点、最近点 4
1.6 点、线段在简单多边形内 4
1.7 $O(n^2)$ Ear Clipping 三角剖分 5
1.8 单插入动态凸包 5
1.9 圆 5
1.10 圆反演，阿波罗尼茨圆 5
1.11 圆并 6
1.12 多边形与圆交 6
1.13 球面基础, 经纬度球面距离 6
1.14 圆上整点 6
1.15 三相之力 6
1.16 相关公式 6
1.16.1 Heron's Formula 6
1.16.2 四面体内接球球心 6
1.16.3 三角形内心 6
1.16.4 三角形外心 6
1.16.5 三角形垂心 6
1.16.6 三角形偏心 6
1.16.7 三角形内接外接圆半径 6
1.16.8 Pick's Theorem 格点多边形面积 6
1.16.9 Euler's Formula 多面体与平面图的点、边、面 6
1.16.10 三角公式 7
1.16.11 超球坐标系 7
1.16.12 三维旋转公式 7
1.16.13 立体角公式 7
1.16.14 常用体积公式 7
1.16.15 扇形与圆弧重心 7
1.16.16 高维球体积 7
1.17 三维几何基础操作 7
1.18 三维凸包 7
1.19 最小覆盖球 7

2 Tree & Graph 8
2.1 Hopcroft-Karp $O(\sqrt{V}E)$ 8
2.2 Hungarian $O(VE/w)$ 8
2.3 Shuffle 一般图最大匹配 $O(VE)$ 8
2.4 极大团计数 8
2.5 KM 最大权匹配 $O(V^3)$ 8
2.6 欧拉回路 8
2.7 2-SAT, 强连通分量 / Bitset Kosaraju 8
2.7.1 Tarjan SCC（个人代码） 9
2.8 Tarjan 点双，边双 9
2.9 Dominator Tree 支配树 9
2.10 Dinic 最大流 9
2.10.1 个人代码 10
2.11 原始对偶费用流 10
2.12 虚树 10
2.13 网络流总结 10
2.14 Gomory-Hu 无向图最小割树 $O(V^3E)$ 11
2.15 Stoer-Wagner 无向图最小割 $O(VE + V^2 \log V)$ 11
2.16 弦图 11
2.17 Minimum Mean Cycle 最小平均值环 $O(n^2)$ 12
2.18 一般图最大匹配 - Blossom 12
2.19 图论结论 12
2.19.1 最小乘积问题原理 12
2.19.2 最小环 12
2.19.3 度序列的可图性 12
2.19.4 切比雪夫距离与曼哈顿距离转化 12
2.19.5 树链的交 12
2.19.6 带修改MST 12
2.19.7 差分约束 12
2.19.8 Segment Tree Beats 12
2.19.9 二分图 12
2.19.10 稳定婚姻问题 12
2.19.11 竞赛图 Landau's Theorem 12
2.19.12 Ramsey Theorem R(3,3)=6, R(4,4)=18 12
2.19.13 树的计数 Prufer序列 12
2.19.14 有根树计数 1,1,2,4,9,20,48,115,286,719,1842,4766 12
2.19.15 无根树计数 12
2.19.16 生成树计数 Kirchhoff's Matrix-Tree Theorem 12
2.19.17 有向图欧拉回路计数 BEST Theorem 12
2.19.18 Tutte Matrix 13
2.19.19 Edmonds Matrix 13
2.19.20 有向图无环定向, 色多项式 13
2.19.21 拟阵交问题 13
2.19.22 双极定向 13
2.19.23 图中的环 13

3 Data Structure 13
3.1 非递归线段树 13
3.1.1 区间加，区间求最大值 13
3.2 点分治 13
3.3 LCT 动态树 14
3.4 可持久化 Treap 14

4 String 14
4.1 最小表示法 14
4.2 Manacher 14
4.3 KMP, exKMP 14
4.4 AC 自动机 14
4.5 Lydon Word Decomposition 14
4.6 后缀自动机 15
4.7 SAMSA & 后缀树 15
4.8 后缀数组 15
4.9 Suffix Balanced Tree 后缀平衡树 15
4.10 广义在线 SAM 15
4.11 回文树 15
4.12 Runs 16
4.13 字符串 Hash 16
4.14 String Conclusions 16

5 Math 数学 17
5.1 Long Long $O(1)$ 乘, Barrett 17
5.2 exgcd, 逆元 17
5.3 CRT 中国剩余定理 17
5.4 Miller Rabin, Pollard Rho 17
5.4.1 Miller–Rabin（个人代码） 17
5.4.2 Pollard–Rho 质因数分解（个人代码） 17
5.5 扩展卢卡斯 18
5.6 阶乘取模 18
5.7 类欧几里得直线下格点统计 18
5.8 万能欧几里德 18
5.9 平方剩余 18
5.10 线性同余不等式 18
5.11 原根 19
5.12 FFT 19
5.12.1 个人代码 19
5.13 NTT 19
5.13.1 个人代码 19
5.14 MTT 任意模数卷积 20
5.15 多项式运算 20
5.15.1 多项式求逆 开根 ln exp 20
5.15.2 多项式除法 取模 20
5.15.3 多点求值 20
5.15.4 插值 21
5.16 线性递推 21
5.17 Berlekamp-Massey 最小多项式 21
5.18 FWT 22
5.19 K 进制 FWT 22
5.20 Simplex 单纯形 22
5.21 高斯消元最小范数解 22
5.22 Pell 方程 23
5.23 解一元三次方程 23
5.24 自适应 Simpson 23

6 Appendix 23
6.1 Formulas 公式表 23
6.1.1 Mobius Inversion 23
6.1.2 杜教筛 23
6.1.3 降幂公式 23
6.1.4 其他常用公式 23
6.1.5 单位根反演 23
6.1.6 Arithmetic Function 23
6.1.7 Binomial Coefficients 24
6.1.8 Fibonacci Numbers, Lucas Numbers 24
6.1.9 Sum of Powers 24
6.1.10 Catalan Numbers 1, 1, 2, 5, 14, 42, 132, 429, 1430... 24
6.1.11 Motzkin Numbers 1, 1, 2, 4, 9, 21, 51, 127, 323, 835... 24
6.1.12 Derangement 错排数 0, 1, 2, 9, 44, 265, 1854, 14833... 24
6.1.13 Bell Numbers 1, 1, 2, 5, 15, 52, 203, 877, 4140 ... 24
6.1.14 Stirling Numbers 24
6.1.15 Eulerian Numbers 24
6.1.16 Harmonic Numbers, 1, 3/2, 11/6, 25/12, 137/60... 24
6.1.17 卡迈克尔函数 24
6.1.18 求拆分数 25
6.1.19 Bernoulli Numbers 1, 1/2, 1/6, 0, -1/30, 0, 1/42 ... 25
6.1.20 kMAX-MIN反演 25
6.1.21 伍德伯里矩阵不等式 25
6.1.22 Sum of Squares 25
6.1.23 枚举勾股数 Pythagorean Triple 25
6.1.24 四面体体积 Tetrahedron Volume 25
6.1.25 杨氏矩阵与钩子公式 25
6.1.26 常见博弈游戏 25
6.1.27 概率相关 25
6.1.28 邻接矩阵行列式的意义 25
6.1.29 Others (某些近似数值公式在这里) 25
6.2 Calculus, Integration Table 导数积分表 25
6.3 Python Hint 26

7 Miscellany 26
7.1 Zeller 日期公式 26
7.2 有理数二分: Stern-Brocot 树, Farey 序列 26
7.3 黄金三分 27
7.4 DP 优化 27
7.4.1 四边形不等式 27
7.4.2 树形背包优化 27
7.4.3 $O(n \cdot \max a_i)$ Subset Sum 27
7.5 Hash Table 28
7.6 基数排序 28
7.7 Hacks: O3, 读入优化, Bitset, builtin 28
7.8 试机赛与纪律文件 28
7.9 Constant Table 常数表 28

8 Personal Code Snippets 30
8.1 矩阵乘法与快速幂 30

9 main(4) 补充材料 30
7.7 Hacks: 指令集、读入优化、Bitset、builtin、随机种子 30
7.8 试机赛与纪律文件 30
7.9 Constant Table 常数表 30

1. Geometry

1.1 二维几何基础操作

1.1.1 点、向量与直线

点与向量都可用二元组 $p=(x,y)$ 表示；有向直线或线段记为 $l=(s,t)$，方向向量为 $\vec{l}=t-s$。向量加减、数乘和除法均按坐标进行。全文约定叉积为 $\det(a,b)=a_xb_y-a_yb_x$，因而 $\det(b-a,c-a)>0$ 表示 $a\to b\to c$ 左转。

向量长度与单位化为

$$\|a\|=\sqrt{a_x^2+a_y^2},\qquad \mathrm{unit}(a)=\frac{a}{\|a\|}.$$

逆时针旋转 θ 的标准公式是

$$x'=x\cos\theta-y\sin\theta,\qquad y'=x\sin\theta+y\cos\theta,$$

$$\mathrm{rot}_{90^\circ}(x,y)=(-y,x).$$

单位化前必须排除零向量；角度统一使用弧度。

1.1.2 点积、叉积与方向

$$a\cdot b=a_xb_x+a_yb_y=\|a\|\|b\|\cos\theta,\qquad \det(a,b)=\|a\|\|b\|\sin\theta.$$

点积判断投影和夹角锐钝，叉积判断有向面积、平行和左右转。几何判断尽量只比较点积或叉积的符号，避免显式计算角度。

1.1.3 浮点误差

浮点坐标使用

$$\mathrm{sgn}(x)=\begin{cases}1, & x>\varepsilon,\\ -1, & x<-\varepsilon,\\ 0, & |x|\le\varepsilon.\end{cases}$$

ε 必须结合坐标范围与运算次数确定；含除法的直线求交等操作误差通常更明显。能用整数叉积完成的判定不要提前转成浮点数。比较平方距离时阈值也应平方：$\|a-b\|^2\le\varepsilon^2$，不能把长度阈值直接用于平方距离。

1.1.4 点在线段上与线段判交

点 p 在线段 st 上当且仅当

$$\det(t-s,p-s)=0,\qquad (s-p)\cdot(t-p)\le 0.$$

两闭线段相交包括两类：某个端点落在另一条线段上，或两条线段互相严格跨立。严格跨立可用两端点相对另一条直线的叉积符号相反判断。非严格相交必须先处理端点共线；同时注意 $s=t$ 的退化线段。

1.1.5 直线求交

设直线 a 与 b 不平行，令

$$u=\det(a.t-a.s,b.s-a.s),\qquad v=-\det(a.t-a.s,b.t-a.s),$$

则交点为

$$I=\frac{v\,b.s+u\,b.t}{u+v}.$$

求交前先判断分母 $u+v$ 是否为零；该公式包含除法，返回浮点坐标时应适当放宽后续判等的误差。

1.1.6 点到直线与线段距离

点 p 到非退化直线 st 的距离是

$$d(p,\overleftrightarrow{st})=\frac{|\det(t-s,p-s)|}{\|t-s\|}.$$

若垂足在线段内部，点到线段距离等于上式；否则取到两个端点距离的较小值。计算前必须单独处理 $s=t$，否则分母为零。

两个点分别沿线段匀速运动时，将相对位置写成 $r(\lambda)=(a_s-b_s)+\lambda[(a_t-a_s)-(b_t-b_s)]$，$0\le\lambda\le 1$，问题即转化为原点到相对运动线段的最短距离。

```
struct point {
  point rot(LD t) const { // 逆时针
    return {x*cos(t) - y*sin(t), x*sin(t) + y*cos(t)};}
  point rot90() const { return {-y, x}; }};
bool two_side(cp a, cp b, cl c) {
  return turn(c.s, c.t, a) * turn(c.s, c.t, b) < 0; }
point line_inter(cl a, cl b) { // 直线交点
  LD s1 = det(a.t - a.s, b.s - a.s);
  LD s2 = det(a.t - a.s, b.t - a.s);
  return (b.s * s2 - b.t * s1) / (s2 - s1); }
vector <point> cut (const vector<point> &c, line l) {
  vector <point> ret; // 线切凸包
  int n = (int) c.size(); if (!n) return ret;
  for (int i = 0; i < n; i++) {
    int j = (i + 1) % n;
    if (turn (l.s, l.t, c[i]) >= 0) ret.push_back(c[i]);
    if (two_side (c[i], c[j], l))
      ret.push_back(line_inter(l, {c[i], c[j]})); }
  return ret; }
bool pos (cp a,cl b){ // point_on_segment 点在线段上
  return turn(b.s, b.t, a) == 0 // 在直线上
    && sgn(dot(b.s - a, b.t - a)) <= 0;}
bool inter_judge(cl a,cl b) { // 线段判非严格交
  if (pos (b.s, a) || pos (b.t, a)) return true;
  if (pos (a.s, b) || pos (a.t, b)) return true;
  return two_side (a.s, a.t, b)
      && two_side (b.s, b.t, a); }
point proj_to_line (cp a, cl b) { // 点在直线投影
  point st = b.t - b.s;
  return b.s + st * (dot(a - b.s, st) / dot(st, st));}
LD p2l (cp a, cl b) { // point_to_line
  return abs(det(b.t-b.s, a-b.s)) / dis(b.s, b.t); }
LD p2s (cp a, cl b) { // point_to_segment 注意退化
  if ( sgn(dot(b.s - a, b.t - b.s))
     * sgn(dot(b.t - a, b.t - b.s)) >= 0)
    return min (dis (a, b.s), dis (a, b.t));
  return p2l (a, b); }
bool point_on_ray (cp a, cl b) { // 点在射线上
  return turn (b.s, b.t, a) == 0
  && sgn(dot(a - b.s, b.t - b.s)) >= 0; }
bool ray_inter_judge(line a, line b) { // 射线判交
  LD s1, s2; // can be LL
  s1 = det(a.t - a.s, b.s - a.s);
  s2 = det(a.t - a.s, b.t - a.s);
  if (sgn(s1) == 0 && sgn(s2) == 0) {
    return sgn(dot(a.t - a.s, b.s - a.s)) >= 0
        || sgn(dot(b.t - b.s, a.s - b.s)) >= 0; }
  if (!sgn(s1 - s2) || sgn(s1) == sgn(s2 - s1)) return 0;
  swap(a, b);
  s1 = det(a.t - a.s, b.s - a.s);
  s2 = det(a.t - a.s, b.t - a.s);
  return sgn(s1) != sgn(s2 - s1); }
```

1.2 凸包与闵可夫斯基和

```
#define cp const point &
int turn (cp a, cp b, cp c) { return sgn(det(b-a, c-a)); }
vector <point> convex_hull (vector <point> a) {
  sort (a.begin(), a.end()); // 小于号 (y, x) 字典序
  a.erase(unique(a.begin(), a.end()), a.end());//必要时去重
  int n = (int) a.size (), cnt = 0;
  vector <point> ret;
  for (int i = 0; i < n; i++) {
    while (cnt > 1
    && turn (ret[cnt - 2], ret[cnt - 1], a[i]) <= 0)
      --cnt, ret.pop_back (); // 保留边界:    <  0
    ++cnt, ret.push_back (a[i]); }
  for (int i = n - 2, fixed = cnt; i >= 0; i--) {
    while (cnt > fixed
    && turn (ret[cnt - 2], ret[cnt - 1], a[i]) <= 0)
      --cnt, ret.pop_back (); // 所有点共线边界会保留两次
    ++cnt, ret.push_back (a[i]); }
  if (n > 1) ret.pop_back (); // n <= 2 吗?
  return ret; } // 小于号为 (y, x) 时边 [0, 2pi) 逆时针
```

```
vector <point> add (vector <point> a, vector <point> b) {
// size > 0, rotate(begin, min, end), 无重, 小于号 (y, x)
  if (a.size() == 1 || b.size() == 1) {
    vector <point> ret;
    for (auto i : a) for (auto j : b) ret.push_back(i+j);
    return ret; }
  vector <point> x, y;
  for (int i = 0; i < a.size(); i++)
    x.push_back(a[(i + 1) % a.size()] - a[i]);
  for (int i = 0; i < b.size(); i++)
    y.push_back(b[(i + 1) % b.size()] - b[i]);
  vector <point> ret (x.size() + y.size());
  merge(x.begin(), x.end(), y.begin(), y.end(),
    ret.begin(), [](cp u, cp v) {
    return half(u)-half(v) ? half(u) : det(u, v) > 0;});
  point cur = a[0] + b[0];
  for (auto &i : ret) swap(i, cur), cur = cur + i;
  return ret; } // ret 可能共线, 但没有重点
```

1.3 直线半平面交

```
int half(cp a){return a.y > 0||(a.y == 0 && a.x > 0)?1:0;}
bool turn_left(cl a, cl b, cl c) {
  return turn(a.s, a.t, line_inter(b, c)) > 0; }
bool is_para(cl a, cl b){return!sgn(det(a.t-a.s,b.t-b.s));}
bool cmp(cl a, cl b) {
  int sign = half(a.t - a.s) - half(b.t - b.s);
  int dir = sgn(det(a.t - a.s, b.t - b.s));
  if (!dir && !sign) return turn(a.s, a.t, b.t) < 0;
  else return sign ? sign > 0 : dir > 0; }
```

```
vector <point> hpi(vector <line> h) { // 半平面交
  sort(h.begin(), h.end(), cmp);
  vector <line> q(h.size()); int l = 0, r = -1;
  for(auto &i : h) {
   while (l < r && !turn_left(i, q[r - 1], q[r])) --r;
   while (l < r && !turn_left(i, q[l], q[l + 1])) ++l;
   if (l <= r && is_para(i, q[r])) continue;
   q[++r] = i; }
  while (r - l > 1 && !turn_left(q[l], q[r - 1], q[r])) --r;
  while (r - l > 1 && !turn_left(q[r], q[l], q[l + 1])) ++l;
  if(r - l < 2) return {};
  vector <point> ret(r - l + 1);
  for(int i = l; i <= r; i++)
    ret[i - l] = line_inter(q[i], q[i == r ? l : i + 1]);
  return ret; }
// 空集会在队列里留下一个开区域；开区域会被判定为空集。
// 为了保证正确性，一定要加足够大的框，尽可能避免零面积区域。
// 实在需要零面积区域边缘，需要仔细考虑 turn_left 的实现。
```

1.4 整数半平面交

```
struct line : point {
 | LD z; // ax + by + c >= 0
 | line () {}
 | line (LD a, LD b, LD c): point(a, b), z(c) {}
 | line (cp a, cp b): point((b-a).rot90()), z(det(a, b)){}
 | LD operator () (cp a) const{return dot(a, *this) + z;}};
point line_inter (cl u, cl v) {
return point(det({u.z, u.y}, {v.z, v.y}),
             det({u.x, u.z}, {v.x, v.z}) ) / -det(u, v); }
LD dis (cl l, cp x = {0, 0}) { return l(x) / l.len(); }
bool is_para(cl x, cl y) { return !sgn(det(x, y)); }
LD det(cl a, cl b, cl c) {
 | return det(a,b)*c.z + det(b,c)*a.z + det(c,a)*b.z;}
int check(cl a, cl b, cl c) { // sgn left(a, inter(b, c))
 | return sgn(det(b, c, a)) * sgn(det(b, c)); }
bool turn_left(cl a, cl b, cl c){return check(a, b, c) >0;}
bool cmp (cl a, cl b) {
if (is_para(a, b) && dot(a, b) > 0) return dis(a) < dis(b);
return half(a) == half(b) ? sgn(det(a,b))>0 : half(b)>0;}
// 用以上函数替换 HPI 函数，需要 half(point)
line perp(cl l) { return {l.y, -l.x, 0}; } // 垂直
line para(cl l, cp o) { // 过一点平行
    return {l.x, l.y, l.z - l(o)}; }
point proj(cp x, cl l) {return x - l * (l(x)/l.len2());}
point refl(cp x, cl l) {return x - l * (l(x)/l.len2())*2;}
bool is_perp(cl x, cl y) { return !sgn(dot(x, y)); }
LD area(cl a, cl b, cl c) { // 0 when error
 | LD d = det(a, b, c);
 | return d * d / (det(a, b) * det(b, c) * det(c, a)); }
vector <line> cut (const vector <line>& o, line l){
 | vector <line> ret; int n = (int) o.size();
 | for (int i = 0; i < n; i++) {
 |   | cl u = o[i], v = o[(i+1) % n], w = o[(i + 2) % n];
 |   | int va = check(l, u, v), vb = check(l, v, w);
 |   | if (va > 0 || vb > 0 || (va == 0 && vb == 0))
 |   |   | ret.push_back(v);
 |   | if (va >= 0 && vb < 0) ret.push_back(l);
 | } return ret; }
```

1.5 凸包询问：凸包内、切点、交点、最近点

```
int n; vector <point> a; // 可以封装成一个 struct
bool inside (cp u) { // 点在凸包内
 | int l = 1, r = n - 2;
 | while (l < r) {
 |   | int mid = (l + r + 1) / 2;
 |   | if (turn(a[0], a[mid], u) >= 0) l = mid;
 |   | else r = mid - 1; }
 | return turn (a[0], a[l], u) >= 0
 |     && turn (a[l], a[l + 1], u) >= 0
 |     && turn (a[l + 1], a[0], u) >= 0; }
int search (auto f) { // 凸包求极值，需要 C++17
 | int l = 0, r = n - 1;
 | int d = f(a[r], a[l]) ? (swap(l, r), -1) : 1;
 | while (d * (r - l) > 1) {
 |   | int mid = (l + r) / 2;
 |   | if (f(a[mid], a[l]) && f(a[mid], a[mid - d])) l = mid;
 |   | else r = mid; } return l; }
pair<int, int> get_tan(cp u) { // 求切线
 return // 严格在凸包外；需要边界上时，特判 a[n-1] -> a[0]
 {search([&](cp x, cp y){return turn(u, y, x) > 0;}),
  search([&](cp x, cp y){return turn(u, x, y) > 0;})};}
point at (int i) { return a[i % n]; }
int inter (cp u, cp v, int l, int r) {
 | int sl = turn(u, v, at(l));
 | while (l + 1 < r) {
 |   | int m = (l + r) / 2;
 |   | if (sl == turn(u, v, at(m))) l = m;
 |   | else r = m; | } return l % n; }
bool get_inter(cp u, cp v, int &i, int &j) { // 求直线交点
 | int p0 = search([&](cp x, cp y){
 |  return det(v - u, x - u) < det(v - u, y - u);}),
 | p1 = search([&](cp x, cp y) {
 |  return det(v - u, x - u) > det(v - u, y - u);});
 | if (turn(u, v, a[p0]) * turn(u, v, a[p1]) < 0) {
 |   | if (p0 > p1) swap(p0, p1);
 |   | i = inter (u, v, p0, p1);
 |   | j = inter (u, v, p1, p0 + n);
 |   | return true; } else return false; }
LD near (cp u, int l, int r) {
 | if (l > r) r += n;
 | int sl = sgn (dot(u - at(l), at(l + 1) - at(l)));
 | LD ret = p2s (u, {at(l), at(l + 1)});
 | while (l + 1 < r) {
 |   | int m = (l + r) / 2;
 |   | if (sl == sgn (dot(u - at(m), at(m + 1) - at(m))))
 |   |   | l = m; else r = m; }
   return min(ret, p2s(u, {at(l), at(l + 1)})); }
LD get_near (cp u) { // 求凸包外点到凸包最近点
 | if (inside(u)) return 0;
 | auto [x, y] = get_tan(u);
 | return min(near(u, x, y), near(u, y, x)); }
```

1.6 点、线段在简单多边形内

```
bool point_in_polygon (cp u, const vector <point> & p) {
 | int n = (int) p.size (), cnt = 0;
 | for (int i = 0; i < n; ++i) {
 |   | point a = p[i], b = p[(i + 1) % n];
 |   | if (pos (u, {a, b})) return true;
 |   | int x = turn (a, u, b);
 |   | int y = sgn (a.y - u.y);
 |   | int z = sgn (b.y - u.y);
 |   | if (x > 0 && y <= 0 && z > 0) ++cnt;
 |   | if (x < 0 && z <= 0 && y > 0) --cnt; }
 | return cnt != 0; } // < 0 在逆时针多边形内; > 0 顺时针
bool in_polygon (cp u, cp v) {
 | // u, v in polygon; p contain those u, v on border
 | for (int i = 0; i < n; i++) {
 |   | int j = (i + 1) % n, k = (i + 2) % n;
 |   | cp ii = p[i], jj = p[j], kk = p[k];
 |   | if (inter_judge_strict({u, v}, {ii, jj})) return 0;
 |   | if (point_on_segment (jj, {u, v})) {
 |   |   | bool good = true, left = turn (ii, jj, kk) >= 0;
 |   |   | for (auto x : {u, v})
 |   |   |   | if (left)
 |   |   |   |   | good &=    turn(ii, jj, x) >= 0
 |   |   |   |   |          && turn(jj, kk, x) >= 0;
 |   |   |   | else
 |   |   |   |   | good &= !(   turn(jj, x, kk) > 0
 |   |   |   |   |           && turn(jj, ii, x) > 0);
 |   |   | if (!good) return 0;
 |   | } } return 1; }
LD get_far (int uid, int vid) {
 | // u -> v in polygon, check the ray u -> polygon
 | cp u = p[uid], v = p[vid];
 | LD far = 1e9;
 | for (int i = 0; i < n; i++) {
 |   | int j = (i + 1) % n, k = (i + 2) % n;
 |   | cp ii = p[i], jj = p[j], kk = p[k];
 |   | if (two_side (ii, jj, {u, v})) {
 |   |   | LD s1 = det(jj - ii, u - ii);
 |   |   | LD s2 = det(jj - ii, v - ii);
 |   |   | if (sgn(s1 - s2) && sgn(s1) != sgn(s2 - s1))
 |   |   |   | far = min(far,
 |   |   |           dis(u, line_inter({ii,jj},{u,v})));}
 |   | if (j != uid && point_on_ray (jj, {u, v})) {
 |   |   | bool good = turn(ii, jj, kk) <= 0;
 |   |   | for (auto x : {u - (jj - u), jj + (jj - u)})
 |   |   |   | good &= !(   turn(ii, jj, x) > 0
 |   |   |   |           && turn(jj, kk, x) > 0);
 |   |   | if (!good) far = min(far, dis(u, jj));
 |   | } } return far; }
void work() {
    for (int i = 0; i < n; i++)
 |   | for (int j = 0; j < n; j++) if (i != j) {
 |   |   | if (!in_polygon(p[i], p[j])) continue;
 |   |   | LD ret = get_far(i,j)+get_far(j,i)-dis(p[i],p[j]);
```

```
| | | ans = max(ans, ret); } }
```

1.7 $O(n^2)$ Ear Clipping 三角剖分

```
vector <array <int, 3>> tri;
void solve () {
 | list <int> l;
 | for (int k = 0; k < n; k++) l.push_back(k);
 | auto check = [&](auto u, auto v, auto w) {
 | | if (turn(u, v, w) <= 0) return false;
 | | for (auto i : l)
 | | | if (turn(u, v, p[i]) == 1 &&
 | | |     turn(v, w, p[i]) == 1 &&
 | | |     turn(w, u, p[i]) == 1) return false;
 | | return true; };
 | for (auto it = l.begin(); l.size() > 3; ) {
 | | auto u = (it == l.begin() ? prev(l.end()) : prev(it));
 | | auto v = (next(it) == l.end() ? l.begin() : next(it));
 | | if (!check(p[*u], p[*it], p[*v])) it = v;
 | | else {
 | | | tri.push_back({*u, *it, *v});
 | | | l.erase(it);
 | | | it = u; } }
 | tri.push_back({l.front(), *next(l.begin()), l.back()}); }
```

1.8 单插入动态凸包

```
struct hull { // upper hull, left to right
set <point> a; LL tot;
hull () {tot = 0;}
LL calc(auto it) {
 | auto u = it == a.begin() ? a.end() : prev(it);
 | auto v = next(it);
 | LL ret = 0;
 | if (u != a.end()) ret += det(*u, *it);
 | if (v != a.end()) ret += det(*it, *v);
 | if (u != a.end() && v != a.end()) ret -= det(*u, *v);
 | return ret; }
void insert (point p) {
 | if (!a.size()) { a.insert (p); return; }
 | auto it = a.lower_bound (p);
 | bool out;
 | if (it == a.begin()) out = (p < *it); // special case
 | else if (it == a.end()) out = true;
 | else out = turn(*prev(it), *it, p) > 0;
 | if (!out) return;
 | while (it != a.begin()) {
 | | auto o = prev(it);
 | | if (o == a.begin() || turn(*prev(o), *o, p) < 0)
       ↪ break;
 | | else erase(o); }
 | while (it != a.end()) {
 | | auto o = next(it);
 | | if (o == a.end() || turn(p, *it, *o) < 0) break;
 | | else erase(it), it = o; }
 | tot += calc(a.insert(p).first); }
void erase(auto it) { tot -= calc(it); a.erase(it); } };
```

1.9 圆

```
struct circle { point c; LD r;};
bool in_circle(cp a, const circle &b) {
 | return sgn(b.r - dis(b.c, a)) >= 0; }
circle make_circle(cp a, cp b) { // 以 a b 为直径的圆
 | point p = (a + b) / 2;
 | return {p, dis(a, p)}; }
circle make_circle(cp a, cp b, cp c) { // 三点共线 inf / nan
 | point bc = c - b, ca = a - c, ab = b - a;
 | point o = (b + c - bc.rot90()*dot(ca,ab)/det(ca,ab)) / 2;
 | return {o, dis(o, a)}; } // 检查上一行正负号
circle min_circle (vector <point> p) { // 最小覆盖圆
 circle ret({0, 0}, 0);
 shuffle (p.begin (), p.end (), rng);
 for (int i = 0; i < (int) p.size (); i++)
  if (!in_circle(p[i], ret)) {
   ret = circle (p[i], 0);
   for (int j = 0; j < i; j++) if (!in_circle(p[j], ret)) {
    ret = make_circle (p[i], p[j]);
    for (int k = 0; k < j; k++) if (!in_circle(p[k], ret))
     ret = make_circle (p[i], p[j], p[k]);
  } } return ret; }
pair <point, point> line_circle_inter (cl a, circle c) {
 | LD d = p2l (c.c, a);
 | // 需要的话返回 vector <point>
 | /* if (sgn (d - R) >= 0) return {}; */
 | LD x = sqrt (sqr(c.r) - sqr(d)); // sqrt(max(0., ...))
 | return {
 | | proj_to_line (c.c, a) + (a.s - a.t).unit() * x,
 | | proj_to_line (c.c, a) - (a.s - a.t).unit() * x }; }
LD circle_inter_area (circle a, circle b) { // 圆面积交
 | LD d = dis (a.c, b.c);
 | if (sgn (d - (a.r + b.r)) >= 0) return 0;
 | if (sgn (d - abs(a.r - b.r)) <= 0) {
 | | LD r = min (a.r, b.r);
 | | return r * r * PI; }
 | LD x = (d * d + a.r * a.r - b.r * b.r) / (2 * d),
 | |    t1 = acos (min (1., max (-1., x / a.r))),
 | |    t2 = acos (min (1., max (-1., (d - x) / b.r)));
 | return sqr(a.r)*t1 + sqr(b.r)*t2 - d*a.r*sin(t1);}
vector <point> circle_inter (circle a, circle b) { // 圆交点
 | if (a.c == b.c || sgn (dis (a.c, b.c) - a.r - b.r) > 0
 | | || sgn (dis (a.c, b.c) - abs (a.r - b.r)) < 0)
 | | return {};
 | point r = (b.c - a.c).unit();
 | LD d = dis (a.c, b.c);
 | LD x = ((sqr (a.r) - sqr (b.r)) / d + d) / 2;
 | LD h = sqrt (sqr (a.r) - sqr (x));
 | if (sgn (h) == 0) return {a.c + r * x};
 | return {a.c + r * x + r.rot90 () * h,
 |          a.c + r * x - r.rot90 () * h}; }
// 返回按照顺时针方向
vector <point> tangent (cp a, circle b) {
 | return circle_inter (make_circle (a, b.c), b); }
vector <line> extangent (circle a, circle b) {
 | vector <line> ret; // 未考虑两圆内切的一条外切线
 | if (sgn(dis(a.c, b.c)-abs(a.r - b.r))<=0) return ret;
 | if (sgn(a.r - b.r) == 0) {
 | | point dir = b.c - a.c;
 | | dir = (dir * a.r / dir.len()).rot90();
 | | ret.push_back({a.c + dir, b.c + dir});
 | | ret.push_back({a.c - dir, b.c - dir});
 | } else {
 | | point p = (b.c * a.r - a.c * b.r) / (a.r - b.r);
 | | auto u = tangent(p, a), v = tangent(p, b);
 | | if (u.size() == 2 && v.size() == 2) {
 | | | if (sgn(a.r-b.r) < 0)
 | | | | swap(u[0], u[1]), swap(v[0], v[1]);
 | | | ret.push_back({u[0], v[0]});
 | | | ret.push_back({u[1], v[1]}); } }
 | return ret; }
vector <line> intangent(circle a, circle b) {
 | vector <line> ret; // 未考虑两圆外切的一条内切线
 | point p = (b.c * a.r + a.c * b.r) / (a.r + b.r);
 | vector u = tangent(p, a), v = tangent(p, b);
 | if (u.size() == 2 && v.size() == 2) {
 | | ret.push_back({u[0], v[0]});
 | | ret.push_back({u[1], v[1]}); } return ret; }
```

1.10 圆反演，阿波罗尼茨圆

所有关于两点 A, B 满足 $PA/PB = k$ 且不等于 1 的点 P 的轨迹是一个圆.
圆幂: 半径为 R 的圆 O, 任意一点 P 到 O 的幂为 $h = OP^2 - R^2$
圆幂定理: 过 P 的直线交圆在 A 和 B 两点, 则 $PA \cdot PB = |h|$
根轴: 到两圆等幂点的轨迹是一条垂直于连心线的直线
反演: 已知一圆 C, 圆心为 O, 半径为 r, 如果 P 与 P' 在过圆心 O 的直线上, 且 $OP \cdot OP' = r^2$, 则称 P 与 P' 关于 O 互为反演. 一般 C 取单位圆.
反演的性质:
不过反演中心的直线反形是过反演中心的圆, 反之亦然.
不过反演中心的圆, 它的反形是一个不过反演中心的圆.
两条直线在交点 A 的夹角, 等于它们的反形在相应点 A' 的夹角, 但方向相反.
两个相交圆周在交点 A 的夹角等于它们的反形在相应点 A' 的夹角, 但方向相反.
直线和圆周在交点 A 的夹角等于它们的反演图形在相应点 A' 的夹角, 但方向相反.
正交圆反形也正交. 相切圆反形也相切, 当切点为反演中心时, 反形为两条平行线. 两两相切的圆 r1, r2, r3, 求与他们都相切的圆 r4. 分母取负号, 答案再取绝对值, 为外切圆半径. 分母取正号为内切圆半径. $r_4^{\pm} = \frac{r_1 r_2 r_3}{r_1 r_2 + r_1 r_3 + r_2 r_3 \pm 2\sqrt{r_1 r_2 r_3 (r_1 + r_2 + r_3)}}$

```
circle inv_c2c(point O, LD R, circle A) {
 | LD OA = dis(A.c, O);
 | LD RB = 0.5 * R * R * (1 / (OA - A.r) - 1 / (OA + A.r));
 | LD OB = OA * RB / A.r;
 | point B = O + (A.c - O) * (OB / OA);
 | return {B, RB};
}  // 点 O 在圆 A 外, 求圆 A 的反演圆 B, R 是反演半径
circle inv_l2c(point O, LD R, line l) {
 | point P = proj_to_line(O, l);
 | LD d = dis(O, P);
 | LD RB = R * R / (2 * d);
 | point VB = (P - O) / d * RB;
 | return {O + VB, RB};
}  // 不过 O 点的直线反演为过 O 点的圆, R 是反演半径
```

```
line inv_c2l (point O, LD R, circle A) {
 | LD t = R * R / (2 * A.r);
 | point p = O + (A.c - O).unit() * t;
 | return {p, p + (O - p).rot90()};
}  // 过 O 点的圆反演为不过 O 点的直线，R 是反演半径
```

1.11 圆并

```
int C; circle c[MAXN]; LD area[MAXN];
struct event { // 如果需要边界而非面积，那么仔细考虑事件顺序
 point p; LD ang; int delta;
 bool operator <(const event &a){return ang < a.ang;}};
void addevent(cc a, cc b, vector<event> &e, int &cnt) {
 LD d2=dis2(a.c, b.c),dw=((a.r-b.r)*(a.r+b.r)/d2+1)/2,pw=
sqrt(max(0.,-(d2-sqr(a.r-b.r)*(d2-sqr(a.r+b.r))/sqr(2*d2));
 point d = b.c - a.c, p = d.rot(PI / 2),
  q0 = a.c + d * dw + p * pw,
  q1 = a.c + d * dw - p * pw;
 LD ang0 = atan2((q0 - a.c).y, (q0 - a.c).x),
    ang1 = atan2((q1 - a.c).y, (q1 - a.c).x);
 e.push_back({q1,ang1,1}); e.push_back({q0,ang0,-1});
 cnt += ang1 > ang0; }
bool issame(cc a, cc b) {
 return sgn(dis(a.c,b.c)) == 0 && sgn(a.r-b.r) == 0; }
bool overlap(cc a, cc b) {
 return sgn(a.r - b.r - dis(a.c, b.c)) >= 0; }
bool intersect(cc a, cc b) {
 return sgn(dis(a.c, b.c) - a.r - b.r) < 0; }
void solve() {
 fill(area, area + C + 2, 0);
 for(int i = 0; i < C; ++i) { int cnt = 1;
  vector<event> e;
  for(int j=0; j<i; ++j) if(issame(c[i],c[j])) ++cnt;
  for(int j = 0; j < C; ++j)
   if(j != i && !issame(c[i], c[j]) && overlap(c[j], c[i]))
     ↪ ++cnt;
  for(int j = 0; j < C; ++j)
   if(j != i && !overlap(c[j], c[i]) && !overlap(c[i], c[j])
     ↪ && intersect(c[i], c[j]))
    addevent(c[i], c[j], e, cnt);
  if(e.empty()) area[cnt] += PI * c[i].r * c[i].r;
  else {
   sort(e.begin(), e.end());
   e.push_back(e.front());
   for(int j = 0; j + 1 <(int)e.size(); ++j) {
    cnt += e[j].delta;
    area[cnt] += det(e[j].p,e[j + 1].p) / 2;
    LD ang = e[j + 1].ang - e[j].ang;
    if(ang < 0) ang += PI * 2;
    area[cnt] += ang * c[i].r * c[i].r / 2 - sin(ang) *
      ↪ c[i].r * c[i].r / 2; } } } }
```

1.12 多边形与圆交

```
LD angle (cp u, cp v) {
 | return 2 * asin(dis(u.unit(), v.unit()) / 2); }
LD area(cp s, cp t, LD r) { // 2 * area
 | LD theta = angle(s, t);
 | LD dis = p2s ({0, 0}, {s, t});
 | if (sgn(dis - r) >= 0) return theta * r * r;
 | auto [u, v] = line_circle_inter({s, t}, {{0, 0}, r});
 | point lo = sgn(det(s, u)) >= 0 ? u : s;
 | point hi = sgn(det(v, t)) >= 0 ? v : t;
 | return det(lo, hi) + (theta - angle(lo, hi)) * r * r; }
LD solve(vector<point> &p, cc c) {
 | LD ret = 0;
 | for (int i = 0; i < (int) p.size (); ++i) {
 |  | auto u = p[i] - c.c;
 |  | auto v = p[(i + 1) % p.size()] - c.c;
 |  | int s = sgn(det(u, v));
 |  | if      (s > 0) ret += area (u, v, c.r);
 |  | else if (s < 0) ret -= area (v, u, c.r);
 | } return abs (ret) / 2; } //ret在p逆时针时为正
```

1.13 球面基础, 经纬度球面距离

球面距离: 连接球面两点的大圆劣弧 (所有曲线中最短)
球面角: 球面两个大圆弧所在半平面形成的二面角
球面凸多边形: 把一个球面多边形任意一边向两方无限延长成大圆, 其余边都在此大圆的同旁.
球面角盈 E: 球面凸n边形的内角和与 $(n-2)\pi$ 的差

离北极夹角 θ, 距离 h 的球冠: $S = 2\pi Rh = 2\pi R^2(1-\cos\theta)$, $V = \frac{\pi h^2}{3}(3R-h)$
球面凸n边形面积: $S = ER^2$

```
// lontitude 经度范围: ±π, latitude 纬度范围: ±π/2
LD sphereDis(LD lon1, LD lat1, LD lon2, LD lat2, LD R) {
 | return R * acos(cos(lat1) * cos(lat2) * cos(lon1 - lon2)
   ↪ + sin(lat1) * sin(lat2)); }
```

1.14 圆上整点

```
vector <LL> solve(LL r) {
 | vector <LL> ret; // non-negative Y pos
 | ret.push_back(0);
 | LL l = 2 * r, s = sqrt(l);
 | for (LL d=1; d<=s; d++) if (l%d==0) {
 |  | LL lim=LL(sqrt(l/(2*d)));
 |  | for (LL a = 1; a <= lim; a++) {
 |  |  | LL b = sqrt(l/d-a*a);
 |  |  | if (a*a+b*b==l/d && __gcd(a,b)==1 && a!=b)
 |  |  |  | ret.push_back(d*a*b);
 |  | } if (d*d==l) break;
 |  | lim = sqrt(d/2);
 |  | for (LL a=1; a<=lim; a++) {
 |  |  | LL b = sqrt(d - a * a);
 |  |  | if (a*a+b*b==d && __gcd(a,b)==1 && a!=b)
 |  |  |  | ret.push_back(l/d*a*b);
 | } } return ret; }
```

1.15 三相之力

```
point incenter (cp a, cp b, cp c) {
 | double p = dis(a, b) + dis(b, c) + dis(c, a);
 | return ( a*dis(b, c) + b*dis(c, a) + c*dis(a, b) ) / p;}
point circumcenter (cp a, cp b, cp c) {
 | point p = b - a, q = c - a, s (dot(p,p)/2, dot(q,q)/2);
 | double d = det(p, q); return a + point(det(s, {p.y,
   ↪ q.y}), det({p.x, q.x}, s)) / d; }
point orthocenter (cp a, cp b, cp c) {
 | return a + b + c - circumcenter (a, b, c) * 2.0; }
point fermat_point (cp a, cp b, cp c) {
 | if (a == b) return a; if (b == c) return b;
 | if (c == a) return c;
 | double ab = dis(a, b), bc = dis(b, c), ca = dis(c, a);
 | double cosa = dot(b - a, c - a) / ab / ca;
 | double cosb = dot(a - b, c - b) / ab / bc;
 | double cosc = dot(b - c, a - c) / ca / bc;
 | double sq3 = PI / 3.0; point mid;
 | if (sgn (cosa + 0.5) < 0) mid = a;
 | else if (sgn (cosb + 0.5) < 0) mid = b;
 | else if (sgn (cosc + 0.5) < 0) mid = c;
 | else if (sgn (det(b - a, c - a)) < 0)
 |      mid = line_inter ({a, b + (c - b).rot (sq3)}, {b, c
   ↪ + (a - c).rot (sq3)});
 | else mid = line_inter ({a, c + (b - c).rot (sq3)}, {c, b
   ↪ + (a - b).rot (sq3)});
 | return mid; } // minimize(|A-x|+|B-x|+|C-x|)
```

1.16 相关公式

1.16.1 Heron's Formula

$p = \frac{a+b+c}{2}$, $a \geq b \geq c$,

$S = \sqrt{p(p-a)(p-b)(p-c)}$,

$S = \frac{1}{2}\sqrt{a^2c^2 - \left(\frac{a^2+c^2-b^2}{2}\right)^2}$.

1.16.2 四面体内接球球心

假设 s_i 是第 i 个顶点相对面的面积，则有

$$\begin{cases} x = \dfrac{s_1x_1 + s_2x_2 + s_3x_3 + s_4x_4}{s_1 + s_2 + s_3 + s_4} \\ y = \dfrac{s_1y_1 + s_2y_2 + s_3y_3 + s_4y_4}{s_1 + s_2 + s_3 + s_4} \\ z = \dfrac{s_1z_1 + s_2z_2 + s_3z_3 + s_4z_4}{s_1 + s_2 + s_3 + s_4} \end{cases}$$

体积可以使用 1/6 混合积求, 内接球半径为

$$r = \frac{3V}{s_1 + s_2 + s_3 + s_4}$$

1.16.3 三角形内心

$$\vec{I} = \frac{a\vec{A} + b\vec{B} + c\vec{C}}{a+b+c}$$

1.16.4 三角形外心

$$\vec{O} = \frac{\vec{A} + \vec{B} - \frac{\vec{BC}\cdot\vec{CA}}{\vec{AB}\times\vec{BC}}\vec{AB}^T}{2}$$

1.16.5 三角形垂心

$$\vec{H} = 3\vec{G} - 2\vec{O}$$

1.16.6 三角形偏心

$$\frac{-a\vec{A} + b\vec{B} + c\vec{C}}{-a+b+c}$$

内角的平分线和对边的两个外角平分线交点, 外切圆圆心. 剩余两点的同理.

1.16.7 三角形内接外接圆半径

$$r = \frac{2S}{a+b+c}, \quad R = \frac{abc}{4S}$$

1.16.8 Pick's Theorem 格点多边形面积

$S = I + \frac{B}{2} - 1$. I 内部点, B 边界点。

1.16.9 Euler's Formula 多面体与平面图的点、边、面

Convex polyhedron: $V - E + F = 2$.
Planar graph: $|F| = |E| - |V| + n + 1$, n : #(connected components).

1.16.10 三角公式

$$\sin(a)+\sin(b)=2\sin(\tfrac{a+b}{2})\cos(\tfrac{a-b}{2})$$
$$\sin(a)-\sin(b)=2\cos(\tfrac{a+b}{2})\sin(\tfrac{a-b}{2})$$
$$\cos(a)+\cos(b)=2\cos(\tfrac{a+b}{2})\cos(\tfrac{a-b}{2})$$
$$\cos(a)-\cos(b)=-2\sin(\tfrac{a+b}{2})\sin(\tfrac{a-b}{2})$$
$$\sin(a\pm b)=\sin a\cos b\pm\cos a\sin b$$
$$\cos(a\pm b)=\cos a\cos b\mp\sin a\sin b$$
$$\tan(a\pm b)=\frac{\tan(a)\pm\tan(b)}{1\mp\tan(a)\tan(b)}$$
$$\tan(a)\pm\tan(b)=\frac{\sin(a\pm b)}{\cos(a)\cos(b)}$$
$$\sin(na)=n\cos^{n-1}a\sin a-\binom{n}{3}\cos^{n-3}a\sin^3a+\binom{n}{5}\cos^{n-5}a\sin^5a-\dots$$
$$\cos(na)=\cos^n a-\binom{n}{2}\cos^{n-2}a\sin^2a+\binom{n}{4}\cos^{n-4}a\sin^4a-\dots$$

1.16.11 超球坐标系

$$\begin{aligned}
x_1 &= r\cos(\phi_1)\\
x_2 &= r\sin(\phi_1)\cos(\phi_2)\\
&\cdots\\
x_{n-1} &= r\sin(\phi_1)\cdots\sin(\phi_{n-2})\cos(\phi_{n-1})\\
x_n &= r\sin(\phi_1)\cdots\sin(\phi_{n-2})\sin(\phi_{n-1})\\
\phi_{n-1} &\in [0,2\pi]\\
\forall i=1..n-1\phi_i &\in [0,\pi]
\end{aligned}$$

1.16.12 三维旋转公式

绕着 $(0,0,0)-(ux,uy,uz)$ 旋转 θ, (ux,uy,uz) 是单位向量. $\begin{bmatrix}x'\\y'\\z'\end{bmatrix}=R\begin{bmatrix}x\\y\\z\end{bmatrix}$

$$R=\begin{pmatrix}\cos\theta+u_x^2(1-\cos\theta) & u_xu_y(1-\cos\theta)-u_z\sin\theta & u_xu_z(1-\cos\theta)+u_y\sin\theta\\ u_yu_x(1-\cos\theta)+u_z\sin\theta & \cos\theta+u_y^2(1-\cos\theta) & u_yu_z(1-\cos\theta)-u_x\sin\theta\\ u_zu_x(1-\cos\theta)-u_y\sin\theta & u_zu_y(1-\cos\theta)+u_x\sin\theta & \cos\theta+u_z^2(1-\cos\theta)\end{pmatrix}.$$

1.16.13 立体角公式

ϕ：二面角

$$\Omega=(\phi_{ab}+\phi_{bc}+\phi_{ac})\text{ rad}-\pi\text{ sr}$$

$$\tan\left(\frac{1}{2}\Omega/\text{rad}\right)=\frac{\left|\vec{a}\,\vec{b}\,\vec{c}\right|}{abc+\left(\vec{a}\cdot\vec{b}\right)c+\left(\vec{a}\cdot\vec{c}\right)b+\left(\vec{b}\cdot\vec{c}\right)a}$$

$$\theta_s=\frac{\theta_a+\theta_b+\theta_c}{2}$$

1.16.14 常用体积公式

- 棱锥 Pyramid $V=\frac{1}{3}Sh$.
- 球 Sphere $V=\frac{4}{3}\pi R^3$.
- 棱台 Frustum $V=\frac{1}{3}h(S_1+\sqrt{S_1S_2}+S_2)$.
- 椭球 Ellipsoid $V=\frac{4}{3}\pi abc$.

1.16.15 扇形与圆弧重心

扇形重心与圆心距离为 $\frac{4r\sin(\theta/2)}{3\theta}$,

圆弧重心与圆心距离为 $\frac{4r\sin^3(\theta/2)}{3(\theta-\sin(\theta))}$.

1.16.16 高维球体积

$$V_2=\pi R^2,\ S_2=2\pi R$$
$$V_3=\frac{4}{3}\pi R^3,\ S_3=4\pi R^2$$
$$V_4=\frac{1}{2}\pi^2R^4,\ S_4=2\pi^2R^3$$
$$\text{Generally, }V_n=\frac{2\pi}{n}V_{n-2},\ S_{n-1}=\frac{2\pi}{n-2}S_{n-3}$$
$$\text{Where, }S_0=2,\ V_1=2,\ S_1=2\pi,\ V_2=\pi$$

1.17 三维几何基础操作

```
/* 右手系逆时针绕轴旋转，(x,y,z)A = (x_new, y_new, z_new)
new[i] += old[j] * A[j][i] */
void calc(p3 n, LD cosw) {
 | LD sinw = sqrt(1 - cosw * cosw);
 | n = n.unit();
 | for (int i = 0; i < 3; i++) {
 |  | int j = (i + 1) % 3, k = (j + 1) % 3;
 |  | LD x = n[i], y = n[j], z = n[k];
 |  | A[i][i] = (y * y + z * z) * cosw + x * x;
 |  | A[i][j] = x * y * (1 - cosw) + z * sinw;
 |  | A[i][k] = x * z * (1 - cosw) - y * sinw; } }
p3 cross (p3 a, p3 b) { return p3(
 a.y*b.z - a.z*b.y, a.z*b.x - a.x*b.z, a.x*b.y - a.y*b.x);}
LD mix(p3 a, p3 b, p3 c) { return dot(cross(a, b), c); }
struct l3 { p3 s, t; };
struct plane { // nor 为单位法向量，离原点距离 m
 | p3 nor; LD m;
 | plane(p3 r, p3 a) : nor(r.unit()), m(dot(nor, a)) {} };
// 除法注意除零；点到直线投影：与二维一致
p3 project_to_plane(p3 a, plane b) { // 点到平面投影
 | return a + b.nor * (b.m - dot(a, b.nor)); }
pair<p3, p3> l3_closest(l3 x, l3 y) { // 两直线最近点
 | LD a = dot(x.t - x.s, x.t - x.s);
 | LD b = dot(x.t - x.s, y.t - y.s);
 | LD e = dot(y.t - y.s, y.t - y.s);
 | LD d = a*e - b*b; p3 r = x.s - y.s;
 | LD c = dot(x.t - x.s, r), f = dot(y.t - y.s, r);
 | LD s = (b*f - c*e) / d, t = (a*f - c*b) / d;
 | return {x.s + (x.t - x.s)*s, y.s + (y.t - y.s)*t}; }
p3 intersect(plane a, l3 b) { // 直线与平面交点
 | LD t = dot(a.nor, a.nor*a.m - b.s)/dot(a.nor, b.t - b.s);
 | return b.s + (b.t - b.s) * t; }
// 平面与平面求交线
l3 intersect(plane a, plane b) {
 | p3 d = cross(a.nor, b.nor), d2 = cross(b.nor, d);
 | LD t = dot(d2, a.nor);
 | p3 s = d2 * (a.m - dot(b.nor*b.m, a.nor))/t + b.nor*b.m;
 | return {s, s + d}; }
// 三个平面求交点
p3 intersect(plane a, plane b, plane c) {
return intersect(a, intersect(b, c));
p3 c1 (a.nor.x, b.nor.x, c.nor.x);
p3 c2 (a.nor.y, b.nor.y, c.nor.y);
p3 c3 (a.nor.z, b.nor.z, c.nor.z);
p3 c4 (a.m, b.m, c.m);
return 1 / mix(c1, c2, c3) * p3(mix(c4, c2, c3), mix(c1, c4,
  ↪ c3), mix(c1, c2, c4)); }
```

1.18 三维凸包

```
vector <p3> p;
int mark[N][N], stp;
typedef array <int, 3> Face;
vector <Face> face;
LD volume (int a, int b, int c, int d) {
 | return mix (p[b] - p[a], p[c] - p[a], p[d] - p[a]); }
void ins(int a, int b, int c) {face.push_back({a, b, c});}
void add(int v) {
 | vector <Face> tmp; int a, b, c; stp++;
 | for (auto f : face) {
 |  | if (sgn(volume(v, f[0], f[1], f[2])) < 0) {
 |  |  | for (auto i : f) for (auto j : f)
 |  |  |  | mark[i][j] = stp; }
 |  | else {
 |  |  | tmp.push_back(f);}
 | } face = tmp;
 | for (int i = 0; i < (int) tmp.size(); i++) {
 |  | a = tmp[i][0], b = tmp[i][1], c = tmp[i][2];
 |  | if (mark[a][b] == stp) ins(b, a, v);
 |  | if (mark[b][c] == stp) ins(c, b, v);
 |  | if (mark[c][a] == stp) ins(a, c, v); } }
bool Find(int n) {
 | for (int i = 2; i < n; i++) {
 |  | p3 ndir = cross (p[0] - p[i], p[1] - p[i]);
 |  | if (ndir == p3(0,0,0)) continue;
 |  | swap(p[i], p[2]);
 |  | for (int j = i + 1; j < n; j++) {
 |  |  | if (sgn(volume(0, 1, 2, j)) != 0) {
 |  |  |  | swap(p[j], p[3]);
 |  |  |  | ins(0, 1, 2);
 |  |  |  | ins(0, 2, 1);
 |  |  |  | return 1;
 | } } } return 0; }
mt19937 rng;
bool solve() {
 | face.clear();
 | int n = (int) p.size();
 | shuffle(p.begin(), p.end(), rng);
 | if (!Find(n)) return 0;
 | for (int i = 3; i < n; i++) add(i);
 | return 1; }
```

1.19 最小覆盖球

```
vector<p3> b; Circle calc() {
 if(b.empty()) { return Circle(p3(0, 0, 0), 0);
 }else if(1 == b.size()) {return Circle(b[0], 0);
 }else if(2 == b.size()) {
  return Circle((b[0] + b[1]) / 2, (b[0] - b[1]).len() / 2);
 }else if(3 == b.size()) {
  LD r = dis(b[0], b[1]) * dis(b[1], b[2]) * dis(b[2], b[0])
    ↪ / 2 / cross(b[0] - b[2], b[1] - b[2]).len();
```

```
  return Circle(intersect({b[1] - b[0], (b[1] + b[0]) / 2},
    ↪{b[2] - b[1], (b[2] + b[1]) / 2}, {cross(b[1] - b[0],
    ↪b[2] - b[0]), b[0]}), r);
 }else { p3 o(intersect({b[1] - b[0], (b[1] + b[0]) / 2},
   ↪{b[2] - b[0], (b[2] + b[0]) / 2}, {b[3] - b[0], (b[3] +
   ↪b[0]) / 2})); return Circle(o, (o - b[0]).len()); } }
Circle miniBall(int n) {
 Circle res(calc());
 for(int i = 0; i < n; i++) if(!in_circle(a[i], res)) {
  b.push_back(a[i]); res = miniBall(i); b.pop_back();
  if (i) { p3 tmp = a[i]; memmove(a + 1, a, sizeof(p3) * i);
    ↪a[0] = tmp; } }
 return res; }
```

2. Tree & Graph

2.1 Hopcroft-Karp $O(\sqrt{V}E)$

```
vector <int> E[N];
vector <int> ml, mr, a, p;
void match (int nl, int nr) { // 1-based
 | ml.assign(nl + 1, 0);
 | mr.assign(nr + 1, 0); // nr
 | while (true) {
 |  | bool ok = 0;
 |  | a.assign(nl + 1, 0);
 |  | p.assign(nl + 1, 0); // nl
 |  | static queue <int> q;
 |  | for (int i = 1; i <= nl; i++)
 |  |  | if (!ml[i]) a[i] = p[i] = i, q.push(i);
 |  | while (!q.empty()) {
 |  |  | int x = q.front(); q.pop();
 |  |  | if (ml[a[x]]) continue;
 |  |  | for (auto y : E[x]) {
 |  |  |  | if (!mr[y]) {
 |  |  |  |  | for (ok = 1; y; x = p[x])
 |  |  |  |  |  | mr[y] = x, swap(ml[x], y);
 |  |  |  |  | break;
 |  |  |  | } else if (!p[mr[y]])
 |  |  |  |  | q.push(y = mr[y]), p[y] = x, a[y] = a[x];
 |  |  | } } // while (!q.empty())
 |  | if (!ok) break; } }
array<vector<int>, 2> min_edge_cover(int nl, int nr) {
 match(nl, nr); vector <int> l, r;
 for (int i = 1; i <= nl; i++) if (!a[i]) l.push_back(i);
 for (int i = 1; i <= nr; i++) if (a[mr[i]]) r.push_back(i);
 return {l, r}; }
```

2.2 Hungarian $O(VE/w)$

```
using B = bitset<N>; B edge[N];
bool dfs(int x, B& unvis, vector<int>& match) {
 | for(B z = edge[x];;) {
 |  | z &= unvis;
 |  | int y = z._Find_first();
 |  | if(y == N) return 0;
 |  | unvis.reset(y);
 |  | if(!match[y] || dfs(match[y], unvis, match))
 |  |  | return match[y] = x, 1; } }
vector<int> match(int nl, int nr) {
 | B unvis; unvis.set();
 | vector<int> match(nr + 1), ret(nl + 1);
 | for(int i = 1;i <= nl;++i)
 |  | if(dfs(i, unvis, match)) unvis.set();
 | for(int i = 1;i <= nr;++i) ret[match[i]] = i;
 | return ret[0] = 0, ret; }
```

2.3 Shuffle 一般图最大匹配 $O(VE)$

```
int n, m, mat[N], vis[N]; vector<int> E[N];
bool dfs(int tim, int x) {
 | shuffle(E[x].begin(), E[x].end(), rng);
 | vis[x] = tim;
 | for (auto y : E[x]) {
 |  | int z = mat[y]; if (vis[z] == tim) continue;
 |  | mat[x] = y, mat[y] = x, mat[z] = 0;
 |  | if (!z || dfs(tim, z)) return true;
 |  | mat[x] = 0, mat[y] = z, mat[z] = y; }
 | return false; }
int main() { // 暗含二分图性质跑一次即可
 | for (int _ = 0; _ < 10; _++) {
 |  | fill(vis + 1, vis + n + 1, 0);
 |  | for (int i = 1; i <= n; ++i) if (!mat[i]) dfs(i, i);}}
```

2.4 极大团计数

```
// 0下标，需删除自环（即确保 E_ii = 0，补图要特别注意）
// 极大团计数，最坏情况O(3^(n/3))
ll ans; ull E[64]; #define bit(i) (1ULL << (i))
void dfs(ull P, ull X, ull R) { // 不要方案时可去掉R
 | if (!P && !X) { ++ans; sol.pb(R); return; }
 | ull Q = P & ~E[__builtin_ctzll(P | X)];
 | for (int i; i = __builtin_ctzll(Q), Q; Q &= ~bit(i)) {
 |  | dfs(P & E[i], X & E[i], R | bit(i));
 |  | P &= ~bit(i), X |= bit(i); }}
ans = 0; dfs(n == 64 ? ~0ULL : bit(n) - 1, 0, 0);
```

2.5 KM 最大权匹配 $O(V^3)$

```
LL e[N][M];
vector<int> KM(int n, int m) {
 | vector<LL> l(n + 1), r(m + 1);
 | vector<int> p(m + 1), ans(n + 1);
 | for (int i = 1; i <= n; ++i) {
 |  | vector<LL> d(m + 1, INF);
 |  | vector<int> pre(m + 1), ok(m + 1);
 |  | int x, v, u = 0;
 |  | for (p[0] = i; x = p[u]; u = v) {
 |  |  | ok[u] = 1;
 |  |  | LL min = INF;
 |  |  | for (int j = 1; j <= m; ++j) if (!ok[j]) {
 |  |  |  | auto w = e[x][j] - l[x] - r[j];
 |  |  |  | if (w < d[j]) d[j] = w, pre[j] = u;
 |  |  |  | if (d[j] < min) min = d[j], v = j;
 |  |  | }
 |  |  | for (int j = 0; j <= m; ++j) {
 |  |  |  | if (ok[j]) l[p[j]] += min, r[j] -= min;
 |  |  |  | else d[j] -= min;
 |  |  | }
 |  | }
 |  | for (int v; u; u = v) {
 |  |  | v = pre[u], p[u] = p[v];
 |  | }
 | }
 | for (int j = 1; j <= m; ++j) {
 |  | ans[p[j]] = j;
 | }
 | return ans;
}
```

2.6 欧拉回路

```
/* comment : directed */
int e, cur[N]/*, deg[N]*/;
vector<int>E[N];
int id[M]; bool vis[M];
stack<int>stk;
void dfs(int u) {
 | for (cur[u]; cur[u] < E[u].size(); cur[u]++) {
 |  | int i = cur[u];
 |  | if (vis[abs(E[u][i])]) continue;
 |  | int v = id[abs(E[u][i])] ^ u;
 |  | vis[abs(E[u][i])] = 1; dfs(v);
 |  | stk.push(E[u][i]); }
}// dfs for all when disconnect
void add(int u, int v) {
 | id[++e] = u ^ v; // s = u
 | E[u].push_back(e); E[v].push_back(-e);
/*  | E[u].push_back(e); deg[v]++; */
} bool valid() {
 | for (int i = 1; i <= n; i++)
 |  | if (E[i].size() & 1) return 0;
/* |  | if (E[i].size() != deg[i]) return 0;*/
 | return 1;}
```

2.7 2-SAT, 强连通分量 / Bitset Kosaraju

```
int stp, sccs, top; // N 开 **两倍**
int dfn[N], low[N], scc[N], stk[N], ans[N];
void add(int x, int a, int y, int b) { // 注意连边对称
 | E[x << 1 | a].push_back(y << 1 | b); }
void tarjan(int x) {
 | dfn[x] = low[x] = ++stp;
 | stk[top++] = x;
 | for (auto y : E[x]) {
 |  | if (!dfn[y])
 |  |  | tarjan(y), low[x] = min(low[x], low[y]);
```

```
  | | else if (!scc[y])
  | | | low[x] = min(low[x], dfn[y]); }
  | if (low[x] == dfn[x]) {
  | | sccs++;
  | | do scc[stk[--top]] = sccs;
  | | while (stk[top] != x); } }
bool solve() {
  | int cnt = n + n; stp = top = sccs = 0;
  | fill(dfn, dfn + cnt + 1, 0); fill(scc, scc + cnt + 1, 0);
  | for (int i = 0; i < cnt; ++i) if (!dfn[i]) tarjan(i);
  | for (int i = 0; i < n; ++i) {
  | | if (scc[i << 1] == scc[i << 1 | 1]) return false;
  | | ans[i] = (scc[i << 1 | 1] < scc[i << 1]); }
  | return true; }
```

```
bitset<N> e[N], re[N], vis; vector<int> sta;
void dfs0(int x, bitset<N> e[]) {
  | vis.reset(x);
  | while (true) {
  | | int go = (e[x] & vis)._Find_first();
  | | if(go == N) break;
  | | dfs0(go, e); }
  | sta.push_back(x); }
vector<vector<int>> solve() { // re 需要连好反向边
  | vis.set();
  | for(int i = 1;i <= n;++i) if(vis.test(i)) dfs0(i, e);
  | vis.set();
  | auto s = sta;
  | vector<vector<int>> ret;
  | for(int i = n - 1;i >= 0;--i) if(vis.test(s[i])) {
  | | sta.clear(), dfs0(s[i], re), ret.push_back(sta); }
  | return ret; }
```

2.7.1 Tarjan SCC（个人代码）

```
int cnt;
int low[N << 1], num[N << 1], dfn;
int sccno[N << 1], stk[N << 1], top;
std::vector<std::vector<int> > adj;

void dfs(int u) {
    stk[top++] = u;
    low[u] = num[u] = ++dfn;
    for (int v : adj[u]) {
        if (!num[v]) {
            dfs(v);
            low[u] = std::min(low[u], low[v]);
        } else if (!sccno[v]) {
            low[u] = std::min(low[u], num[v]);
        }
    }
    if (low[u] == num[u]) {
        ++cnt;
        while (1) {
            int v = stk[--top];
            sccno[v] = cnt;
            if (u == v) break;
        }
    }
}

void tarjan(int n) {
    cnt = dfn = top = 0;
    for (int i = 1; i <= n; ++i)
        if (!num[i])
            dfs(i);
}
```

2.8 Tarjan 点双，边双

```
/** 求割边 **/
// 注意！建立边双树或者圆方树后【边表大小】是否开够
// 求边双：无视掉 bri 后搜出每个连通块，记得多测
int DFN[N], low[N], dfscnt; // clear DFN low bri dfscnt
bool bri[M << 1]; // 注意此处是边数
void tarjan(int x, int last) { // last 是边
  | DFN[x] = low[x] = ++dfscnt;
  | for (int i = head[x], d; i; i = h[i].next) {
  | | d = h[i].node;
  | | if (!DFN[d]) {
  | | | tarjan(d, i);
  | | | low[x] = min(low[x], low[d]);
  | | | if (low[d] > DFN[x]) bri[i] = bri[i ^ 1] = 1;
  | | } else if (DFN[d] < DFN[x] && ((i ^ 1) != last))
  | | | low[x] = min(low[x], DFN[d]); } }
/** 建立圆方树+求割点 **/
int is_cut[N], DFN[N], low[N], dfscnt, pcnt;
int stk[N], dep; // clear DFN low is_cut dfscnt, let pcnt=n
void tarjan(int x, int fa) {
  | int child = 0;
  | DFN[x] = low[x] = ++dfscnt; stk[++dep] = x;
  | #define head org // org 是原图表头，tr 是圆方树表头
  | for (int i = head[x], d; i; i = h[i].next) {
  | | d = h[i].node;
  | | if (!DFN[d]) {
  | | | ++child; tarjan(d, x);
  | | | low[x] = min(low[x], low[d]);
  | | | if (low[d] >= DFN[x]) {
  | | | | is_cut[x] = true;
  | | | | ++pcnt; // square node index
  | | | | int j = 0, sz = 1;
  | | | | do {
  | | | | | j = stk[dep--];
  | | | | | addedge(pcnt, j, tr);
  | | | | | ++sz;
  | | | | } while (j != d);
  | | | | addedge(pcnt, x, tr); }
  | | } else if (DFN[d] < low[x]) low[x] = DFN[d]; }
  | #undef head
  | if (!fa && child == 1) is_cut[x] = false; }
```

2.9 Dominator Tree 支配树

```
vector<int> G[MAXN], R[MAXN], son[MAXN];
int ufs[MAXN]; // R 是反图，son 存的是 sdom 树上的儿子
int idom[MAXN], sdom[MAXN], anc[MAXN];
// anc: sdom的dfn最小的祖先
int pr[MAXN], dfn[MAXN], id[MAXN], stamp;
int findufs(int x) { if (ufs[x] == x) return x;
  | int t = ufs[x]; ufs[x] = findufs(ufs[x]);
  | if (dfn[sdom[anc[x]]] > dfn[sdom[anc[t]]])
  | | anc[x] = anc[t];
  | return ufs[x]; }
void dfs(int x) {
  | dfn[x] = ++stamp; id[stamp] = x; sdom[x] = x;
  | for (int y : G[x]) if (!dfn[y]) { pr[y] = x; dfs(y); } }
void get_dominator(int n) {
  | for (int i = 1; i <= n; i++) ufs[i] = anc[i] = i;
  | dfs(1);
  | for (int i = n; i > 1; i--) { int x = id[i];
  | | for (int y : R[x]) if (dfn[y]) { findufs(y);
  | | | if (dfn[sdom[x]] > dfn[sdom[anc[y]]])
  | | | | sdom[x] = sdom[anc[y]]; }
  | | son[sdom[x]].push_back(x); ufs[x] = pr[x];
  | | for (int u : son[pr[x]]) { findufs(u);
  | | | idom[u] = (sdom[u] == sdom[anc[u]] ?
  | | | | pr[x] : anc[u]); }
  | | son[pr[x]].clear(); }
  | for (int i = 2; i <= n; i++) { int x = id[i];
  | | if (idom[x] != sdom[x]) idom[x] = idom[idom[x]];
  | | son[idom[x]].push_back(x); } }
```

2.10 Dinic 最大流

复杂度证明思路 假设 dist 为残量网络上的距离. Dinic 一轮增广会找到一个极大的长度为 $\mathrm{dist}(s,t)$ 的增广路集合 blocking flow, 增广后 $\mathrm{dist}(s,t)$ 将会增大. 因此只有 $O(V)$ 轮; 如果一轮增广是 $O(VE)$ 的, 总复杂度是 $O(V^2E)$.

单位流量网络 在 0-1 流量图上 Dinic 有更好的性质.

- 复杂度为 $O(\min\{V^{2/3}, E^{1/2}\}E)$.
- $\mathrm{dist}(s,t) = d$, 残量网络上至多还存在 E/d 的流.
- 每个点只有一个入/出度时复杂度 $O(V^{1/2}E)$, 例如 Hopcroft-Karp.

```
struct edge { int v, nxt; value f; } e[M * 2];
int ecnt = 1, head[N], cur[N];
void add(int u, int v, value f) {
  | e[++ecnt] = {v, head[u],        f}; head[u] = ecnt;
  | e[++ecnt] = {u, head[v], (value)0}; head[v] = ecnt; }
int n, S, T, q[N], tag[N];
bool bfs() {
  | for (int i = S; i <= T; i++) tag[i] = 0; // S-T?
  | int he = 0, ta = 1; q[0] = T; tag[T] = 1;
  | while (he < ta) {
  | | int x = q[he++]; if (x == S) return true;
  | | for (int o = head[x]; o; o = e[o].nxt)
  | | | if (e[o ^ 1].f && !tag[e[o].v])
  | | | | tag[e[o].v] = tag[x] + 1, q[ta++] = e[o].v;
  | } return false; }
```

```
value dfs(int x, value flow) {
 | if (x == T) return flow;
 | value used = 0;
 | for (int &o = cur[x]; o; o = e[o].nxt) {
 |   | if (e[o].f && tag[x] - 1 == tag[e[o].v]) {
 |   |   | value ret = dfs(e[o].v, min(flow - used, e[o].f));
 |   |   | if (ret) {
 |   |   |   | e[o].f -= ret; e[o ^ 1].f += ret;
 |   |   |   | used += ret;
 |   |   |   | if (used == flow) return flow;
 | } } } return used; }
value dinic() {
 | value ans = 0;
 | while (bfs()) {
 |   | for (int i = S; i <= T; i++) cur[i] = head[i]; // S-T?
 |   | ans += dfs(S, INF);
 | } return ans; }
```

2.10.1 个人代码

```
struct edg {
    int ne, des, vol;
} edges[E];
int head[V], cur[V], lvl[V], S, T, idx;
const int INF = 0x3f3f3f3f;

void add(int a, int b, int c) {
    edges[idx] = {head[a], b, c}, head[a] = idx++;
    edges[idx] = {head[b], a, 0}, head[b] = idx++;
}

int dfs(int u, int flow) {
    if (u == T)
        return flow;
    int rmn = flow;
    for (int i = cur[u]; ~i; i = edges[i].ne) {
        cur[u] = i;
        int v = edges[i].des;
        if (edges[i].vol == 0 || lvl[v] != lvl[u] + 1)
          ↪ continue;
        int t = dfs(v, std::min(rmn, edges[i].vol));
        rmn -= t;
        edges[i].vol -= t;
        edges[i ^ 1].vol += t;
        if (rmn == 0)
            break;
    }
    return flow - rmn;
}

bool bfs() {
    std::fill(lvl, lvl + T + 1, -1);
    std::copy(head, head + T + 1, cur);
    lvl[S] = 0;
    std::queue<int> q;
    q.push(S);
    while (q.size()) {
        int u = q.front();
        q.pop();
        for (int i = head[u]; ~i; i = edges[i].ne) {
            if (!edges[i].vol) continue;
            int v = edges[i].des;
            if (lvl[v] == -1) {
                lvl[v] = lvl[u] + 1;
                q.push(v);
            }
        }
    }
    return lvl[T] != -1;
}

int dinic() {
    int ans = 0;
    while (bfs()) {
        ans += dfs(S, INF);
    }
    return ans;
}

// S = 0, T = ???
// idx = 0;
// std::fill(head, head + T + 1, -1);
```

2.11 原始对偶费用流

```
bool bfs() {
  for (int i = S; i <= T; i++) cur[i] = head[i]; // S-T?
  for (int i = S; i <= T; i++) dep[i] = 0; // S-T?
  dep[S] = 1; queue<int> q; q.push(S);
  while (!q.empty()) {
    int x = q.front(); q.pop();
    for (int i = head[x]; i; i = e[i].nxt) {
      auto [nxt, v, w, f] = e[i];
      if (f && h[v] == h[x] + w && !dep[v]) {
        dep[v] = dep[x] + 1, q.push(v);
  } } } return !!dep[T]; }
int dfs(int x, int flow) {
  if (x == T) return flow;
  int used = 0, ret;
  for (int &i = cur[x]; i; i = e[i].nxt) {
    auto [nxt, v, w, f] = e[o];
    if (dep[v] == dep[x] + 1 && h[v] == h[x] + w
    && f && (ret = dfs(d, min(flow - used, f))) ) {
      e[i].f -= ret; e[i ^ 1].f += f; used += ret;
      if (flow == used) break;
  } } return used; }
typedef pair <value, int> pii; // Unusual!
pii solve() { // return {cost, flow}
  value cost = 0; int flow = 0;
  for (int i = S; i <= T; i++) h[i] = 0; // S-T?
  for (bool first = true; ; ) {
    priority_queue<pii, vector<pii>, greater<pii>> q;
    for (int i = S; i <= T; i++) dis[i] = INF_value; // S-T?
    dis[S] = 0;
    if (first) {
      // TODO: SSSP, Bellman-Ford / DP on DAG
      first = false;
    } else { q.push({0, S});
      while (!q.empty()) {
        auto [d, x] = q.top(); q.pop();
        if (dis[x] != d) continue;
        for (int o = head[x]; o; o = e[o].nxt) {
          value w = d + e[o].w + h[x] - h[e[o].v];
          if (e[o].f > 0 && dis[e[o].v] > w) {
            dis[e[o].v] = w; q.push({w, e[o].v});
    } } } } if (dis[T] >= INF_value) break;
    // 所有点必须可达, 否则加 h[i] += min(dis[i], dis[T])
    for (int i = S; i <= T; i++) h[i] += dis[i]; // S-T?
    int f = 0; while (bfs()) f += dfs(S, INF_int);
    cost += f * h[T]; flow += f;
  } return {cost, flow}; }
```

2.12 虚树

```
bool has_root = false; for (int i : q) has_root |= (i == 1);
if (!has_root) q.push_back(1);
sort(q.begin(), q.end(), [](int u, int v) {
 | return dfn[u] < dfn[v]; });
int top = 0;
for (int x : q) {
 | used.push_back(x);
 | if (top == 0) stk[++top] = x;
 | else { int lca = LCA(stk[top], x); used.push_back(lca);
 |   | while (top > 1 && dep[lca] < dep[stk[top - 1]]) {
 |   |   | h[stk[top - 1]].push_back(stk[top]); --top; }
 |   | if (dep[lca] < dep[stk[top]])
 |   |   | h[lca].push_back(stk[top--]);
 |   | if (stk[top] != lca) stk[++top] = lca;
 |   | stk[++top] = x; } }
while (--top) // assert (top)
 | h[stk[top]].push_back(stk[top + 1]);
LL ans = solve(1, 0);
for (auto i : used) h[i].clear();
```

2.13 网络流总结

最小割集, 最小割必须边以及可行边

最小割集 从 S 出发, 在残余网络中BFS所有权值非 0 的边 (包括反向边), 得到点集 $\{S\}$, 另一集为 $\{V\} - \{S\}$.

最小割集必须点 残余网络中S直接连向的点必在S的割集中, 直接连向T的点必在T的割集中; 若这些点的并集为全集, 则最小割方案唯一.

最小割可行边 在残余网络中求强联通分量, 将强联通分量缩点后, 剩余的边即为最小割可行边, 同时这些边也必然满流.

最小割必须边 在残余网络中求强联通分量, 若S出发可到u, T出发可到v, 等价于 $\text{scc}_S = \text{scc}_u$ 且 $\text{scc}_T = \text{scc}_v$, 则该边为必须边.

常见问题

最大权闭合子图 点权, 限制条件形如: 选择A则必须选择B, 选择B则必须选择C, D. 建图方式: B向A连边, CD向B连边. 求解: S向正权点连边, 负权点向T连边, 其余边容量 ∞, 求最小割, 答案为S所在最小割集.

二次布尔型 (文理分科) n 个点分为两类, i 号点有 l_i 或 r_i 的代价, i,j 同属一侧分别获得 l_{ij} 或 r_{ij} 的代价, 问最小代价. $L \to i : (l_i + 1/2\sum_j l_{ij})$, $i \to R : (r_i + 1/2\sum_j r_{ij})$, $i \leftrightarrow j : 1/2(l_{ij} + r_{ij})$. 实现时边权乘 2 为整数, 求解后答案除 2 为整数. 图拆点可以看作二分图.
如果是二元限制是分类不同时有一个代价 d_{ij}, 建图可以简化为 $L \to i : l_i$, $i \to R : r_i$, $i \leftrightarrow j : d_{ij}$. 经典例子: xor 最小值, 按位拆开建图.

混合图欧拉回路 把无向边随便定向, 计算每个点的入度和出度, 如果有某个点出入度之差 $\deg_i = \text{in}_i - \text{out}_i$ 为奇数, 肯定不存在欧拉回路. 对于 $\deg_i > 0$ 的点, 连接边 $(i, T, \deg_i/2)$; 对于 $\deg_i < 0$ 的点, 连接边 $(S, i, -\deg_i/2)$. 最后检查是否满流即可.

二物流 水源 S_1, 水汇 T_1, 油源 S_2, 油汇 T_2, 每根管道流量共用. 求流量和最大. 建超级源 SS_1 汇 TT_1, 连边 $SS_1 \to S_1$,$SS_1 \to S_2$,$T_1 \to TT_1$,$T_2 \to TT_1$, 设最大流为 x_1. 建超级源 SS_2 汇 TT_2, 连边 $SS_2 \to S_1$, $SS_2 \to T_2$,$T_1 \to TT_2$, $S_2 \to TT_2$, 设最大流为 x_2. 则最大流中水流量 $\frac{x_1+x_2}{2}$, 油流量 $\frac{x_1-x_2}{2}$.

无源汇有上下界可行流 每条边 (u,v) 有一个上界容量 $C_{u,v}$ 和下界容量 $B_{u,v}$, 我们让下界变为 0, 上界变为 $C_{u,v} - B_{u,v}$, 但这样做流量不守恒. 建立超级源点 SS 和超级汇点 TT, 用 du_i 来记录每个节点的流量情况, $du_i = \sum B_{j,i} - \sum B_{i,j}$, 添加一些附加弧. 当 $du_i > 0$ 时, 连边 (SS, i, du_i); 当 $du_i < 0$ 时, 连边 $(i, TT, -du_i)$. 最后对 (SS, TT) 求一次最大流即可, 当所有附加边全部满流时 (即 maxflow == $du_i > 0$) 时有可行解.

有源汇有上下界最大可行流 建立超级源点 SS 和超级汇点 TT, 首先判断是否存在可行流, 用无源汇有上下界可行流的方法判断. 增设一条从 T 到 S 没有下界容量为无穷的边, 那么原图就变成了一个无源汇有上下界可行流问题. 同样地建图后, 对 (SS, TT) 进行一次最大流, 判断是否有可行解. 如果有可行解, 删除超级源点 SS 和超级汇点 TT, 并删去 T 到 S 的这条边, 再对 (S, T) 进行一次最大流, 此时得到的 maxflow 即为有源汇有上下界最大可行流.

有源汇有上下界最小可行流 建立超级源点 SS 和超级汇点 TT, 和无源汇有上下界可行流一样新增一些边, 然后从SS到TT跑最大流. 接着加上边 (T, S, ∞), 再从 SS 到 TT 跑一遍最大流. 如果所有新增边都是满的, 则存在可行流, 此时 T 到 S 这条边的流量即为最小可行流.

有上下界费用流 如果求无源汇有上下界最小费用可行流或有源汇有上下界最小费用最大可行流, 用1.6.3.1/1.6.3.2 的构图方法, 给边加上费用即可. 求有源汇有上下界最小费用最小可行流, 要先用1.6.3.3的方法建图, 先求出一个保证必要边满流情况下的最小费用. 如果费用全部非负, 那么这时的费用就是答案. 如果费用有负数, 那么流多了可能更好, 继续做从 S 到 T 的流量任意的最小费用流, 加上原来的费用就是答案.

费用流消负环 新建超级源SS汇TT, 对于所有流量非空的负权边e, 先流满 (ans+=e.f*e.c, e.rev.f+=e.f, e.f=0), 再连边SS→e.to, e.from→TT, 流量均为e.f(>0), 费用均为0. 再连边T→S流量 ∞ 费用0. 此时没有负环了. 做一遍SS到TT的最小费用最大流, 将费用累加ans, 拆掉T→S的那条边 (此边的流量为残量网络中S→T的流量). 此时负环已消, 再继续跑最小费用最大流.

整数线性规划转费用流
首先将约束关系转化为所有变量下界为 0, 上界没有要求, 并满足一些等式, 每个变量在均在等式左边且出现恰好两次, 系数为 +1 和 −1, 优化目标为 $\max \sum v_i x_i$ 的形式. 将等式看做点, 等式i右边的值 b_i 若为正, 则 S 向 i 连边 $(b_i, 0)$, 否则i向T连边 $(-b_i, 0)$. 将变量看做边, 记变量 x_i 的上界为 m_i(无上界则 $m_i = inf$), 将 x_i 系数为 +1 的那个等式 u 向系数为 −1 的等式 v 连边 (m_i, v_i).

2.14 Gomory-Hu 无向图最小割树 $O(V^3E)$

每次随便找两个点 s,t 求在**原图**的最小割, 在最小割树上连 (s, t, w_{cut}), 递归对由割集隔开的部分继续做. 在得到的树上, 两点最小割即为树上瓶颈路. 实现时, 由于是随意找点, 可以写为分治的形式.

2.15 Stoer-Wagner 无向图最小割 $O(VE + V^2 \log V)$

```
const int N = 601;
int f[N], siz[N], G[N][N];
int getf(int x) {return f[x] == x ? x : f[x] = getf(f[x]);}
int dis[N], vis[N], bin[N];
int n, m;
int contract(int &s, int &t) {  // Find s,t
  memset(vis, 0, sizeof(vis));
  memset(dis, 0, sizeof(dis));
  int i, j, k, mincut, maxc;
  for (i = 1; i <= n; i++) {
    k = -1; maxc = -1;
    for (j = 1; j <= n; j++)
      if (!bin[j] && !vis[j] && dis[j] > maxc) {
        k = j;
        maxc = dis[j]; }
    if (k == -1) return mincut;
    s = t; t = k; mincut = maxc; vis[k] = true;
    for (j = 1; j <= n; j++)
      if (!bin[j] && !vis[j]) dis[j] += G[k][j];
  } return mincut; }
const int inf = 0x3f3f3f3f;
int solve() {
  int mincut, i, j, s, t, ans;
  for (mincut = inf, i = 1; i < n; i++) {
    ans = contract(s, t);
    bin[t] = true;
    if (mincut > ans) mincut = ans;
    if (mincut == 0) return 0;
    for (j = 1; j <= n; j++)
      if (!bin[j]) G[s][j] = (G[j][s] += G[j][t]);
  } return mincut; }
int main() {
  cin >> n >> m;
  for (int i = 1; i <= n; ++i) f[i] = i, siz[i] = 1;
  for (int i = 1, u, v, w; i <= m; ++i) {
    cin >> u >> v >> w;
    int fu = getf(u), fv = getf(v);
    if (fu != fv) {
      if (siz[fu] > siz[fv]) swap(fu, fv);
      f[fu] = fv, siz[fv] += siz[fu]; }
    G[u][v] += w, G[v][u] += w; }
  cout << (siz[getf(1)] != n ? 0 : solve()); }
```

2.16 弦图

弦图的定义 连接环中不相邻的两个点的边称为弦. 一个无向图称为弦图, 当图中任意长度都大于 3 的环都至少有一个弦.
单纯点 一个点称为单纯点当 $\{v\} \cup A(v)$ 的导出子图为一个团. 任何一个弦图都至少有一个单纯点, 不是完全图的弦图至少有两个不相邻的单纯点.
完美消除序列 一个序列 $v_1, v_2, ..., v_n$ 满足 v_i 在 $v_i, \cdots, v_n$ 的诱导子图中为一个单纯点. 一个无向图是弦图当且仅当它有一个完美消除序列.
最大势算法 从 n 到 1 的顺序依次给点标号. 设 label_i 表示第 i 个点与多少个已标号的点相邻, 每次选择 label 最大的未标号的点进行标号. 用桶维护优先队列可以做到 $O(n+m)$.
弦图的判定 判定最大势算法输出是否合法即可. 如果依次判断是否构成团, 时间复杂度为 $O(nm)$. 考虑优化, 设 $v_{i+1}, \cdots, v_n$ 中所有与 v_i 相邻的点依次为 $N(v_i) = \{v_{j1}, \cdots, v_{jk}\}$. 只需判断 v_{j1} 是否与 $v_{j2}, \cdots, v_{jk}$ 相邻即可. 时间复杂度 $O(n+m)$.
弦图的染色 完美消除序列**从后往前**染色, 染上出度的 mex.
最大独立集 完美消除序列**从前往后**能选就选.
团数 最大团的点数. 一般图团数 ≤ 色数, 弦图团数 = 色数.
极大团 弦图的极大团一定为 $\{x\} \cup N(x)$.
最小团覆盖 用最少的团覆盖所有的点. 设最大独立集为 $\{p_1, \ldots, p_t\}$, 则 $\{p_1 \cup N(p_1), \ldots, p_t \cup N(p_t)\}$ 为最小团覆盖.
弦图 k 染色计数 $\prod_{v \in V} k - N(v) + 1$.
区间图 每个顶点代表一个区间, 有边当且仅当区间有交. 区间图是弦图, 一个完美消除序列是右端点排序.

```
vector <int> L[N];
int seq[N], lab[N], col[N], id[N], vis[N];
void mcs() {
  for (int i = 0; i < n; i++) L[i].clear();
  fill(lab + 1, lab + n + 1, 0);
  fill(id + 1, id + n + 1, 0);
  for (int i = 1; i <= n; i++) L[0].push_back(i);
  int top = 0;
  for (int k = n; k; k--) {
    int x = -1;
    for ( ; ; ) {
      if (L[top].empty()) top --;
      else {
        x = L[top].back(), L[top].pop_back();
        if (lab[x] == top) break;
      }
    }
    seq[k] = x; id[x] = k;
    for (auto v : E[x]) {
      if (!id[v]) {
        L[++lab[v]].push_back(v);
        top = max(top, lab[v]);
      } } } }
bool check() {
  fill(vis + 1, vis + n + 1, 0);
  for (int i = n; i; i--) {
    int x = seq[i];
    vector <int> to;
    for (auto v : E[x])
      if (id[v] > i) to.push_back(v);
    if (to.empty()) continue;
    int w = to.front();
    for (auto v : to) if (id[v] < id[w]) w = v;
    for (auto v : E[w]) vis[v] = i;
    for (auto v : to)
      if (v != w && vis[v] != i) return false;
  } return true; }
void color() {
  fill(vis + 1, vis + n + 1, 0);
  for (int i = n; i; i--) {
```

```
|  | int x = seq[i];
|  | for (auto v : E[x]) vis[col[v]] = x;
|  | for (int c = 1; !col[x]; c++)
|  |  | if (vis[c] != x) col[x] = c;
| } }
```

2.17 Minimum Mean Cycle 最小平均值环 $O(n^2)$

```
// 点标号为 1,2,···,n, 0为虚拟源点向其他点连权值为0的单向边.
// f[i][v] : 从 0 到 v 恰好经过 i 条路的最短路
ll f[N][N] = {Inf}; int u[M], v[M], w[M]; f[0][0] = 0;
for(int i = 1; i <= n + 1; i ++)
 | for(int j = 0; j < m; j ++)
 |  | f[i][v[j]] = min(f[i][v[j]], f[i - 1][u[j]] + w[j]);
double ans = Inf;
for(int i = 1; i <= n; i ++) {
 | double t = -Inf;
 | for(int j = 1; j <= n; j ++)
 |  | t = max(t, (f[n][i] - f[j][i]) / (double)(n - j));
 | ans = min(t, ans); }
```

2.18 一般图最大匹配 - Blossom

```
int n, m, l, r, u, v, res, q[N], p[N], h[N], fa[N], col[N];
vector<int> G[N];
inline int find(int x) {return fa[x] == x ? x : fa[x] =
  ↪ find(fa[x]);}
int lca(int u, int v) {
 | static int fl[N], tim; ++tim;
 | while (fl[u] != tim) {
 |  | if (u) fl[u] = tim, u = find(p[h[u]]);
 |  | swap(u, v);
 | } return u; }
void blossom(int u, int v, int t) {
 | while (find(u) != t) {
 |  | p[u] = v, v = h[u], fa[u] = fa[v] = t;
 |  | if (col[v] == 1) col[v] = 2, q[++r] = v;
 |  | u = p[v]; } }
bool match(int rt) {
 | for (int i = 1; i <= n; i++) fa[i] = i, col[i] = 0;
 | l = r = 1, q[1] = rt, col[rt] = 2;
 | while (l <= r) {
 |  | u = q[l++];
 |  | for (int v : G[u]) if (!col[v]) {
 |  |  | col[v] = 1, col[h[v]] = 2, p[v] = u, q[++r] = h[v];
 |  |  | if (!h[v]) {
 |  |  |  | while (u) u = h[p[v]], h[v] = p[v], h[p[v]] = v,
 |  |  |  |   ↪ v = u;
 |  |  |  | return true;
 |  |  | }
 |  | } else if (col[v] == 2) {
 |  |  | int t = lca(u, v); blossom(u, v, t), blossom(v, u,
 |  |  |   ↪ t);
 |  | }
 | } return false;
} // for(int i=1;i<=n;i++)if(!h[i]&&match(i))++res;
```

2.19 图论结论

2.19.1 最小乘积问题原理

每个元素有两个权值 $\{x_i\}$ 和 $\{y_i\}$, 要求在某个限制下 (例如生成树, 二分图匹配) 使得 $\Sigma x \Sigma y$ 最小. 对于任意一种符合限制的选取方法, 记 $X = \Sigma x_i, Y = \Sigma y_i$, 可看做平面内一点 (X, Y). 答案必在下凸壳上, 找出该下凸壳所有点, 即可枚举获得最优答案. 可以递归求出此下凸壳所有点, 分别找出距 x, y 轴最近的两点 A, B, 分别对应于 $\Sigma y_i, \Sigma x_i$ 最小. 找出距离线段最远的点 C, 则 C 也在下凸壳上, C 点满足 $AB \times AC$ 最小, 也即

$$(X_B - X_A)Y_C + (Y_A - Y_B)X_C - (X_B - X_A)Y_A - (Y_B - Y_A)X_A$$

最小, 后两项均为常数, 因此将所以权值改成 $(X_B - X_A)y_i + (Y_B - Y_A)x_i$, 求同样问题 (例如最小生成树, 最小权匹配) 即可. 求出 C 点以后, 递归 AC, BC.

2.19.2 最小环

无向图最小环: 每次floyd到 k 时, 判断 1 到 $k-1$ 的每一个 i, j:

$$\text{ans} = \min\{\text{ans}, d(i,j) + G(i,k) + G(k,j)\}.$$

有向图最小环: 做完floyd后, $d(i,i)$ 即为经过 i 的最小环.

2.19.3 度序列的可图性

判断一个度序列是否可转化为简单图, 除了一种贪心构造的方法外, 下列方法更快速. EG定理: 将度序列从大到小排序得到 $\{d_i\}$, 此序列可转化为简单图当且仅当 Σd_i 为偶数, 且对于任意的 $1 \le k \le n-1$ 满足 $\sum_{i=1}^{k} d_i \le k(k-1) + \sum_{i=k+1}^{n} \min(k, d_i)$.

2.19.4 切比雪夫距离与曼哈顿距离转化

曼哈顿转切比雪夫: $(x+y, x-y)$, 适用于一些每次只能向四联通的格子走一格的问题. 切比雪夫转曼哈顿: $(\frac{x+y}{2}, \frac{x-y}{2})$, 适用于统计距离.

2.19.5 树链的交

```
bool cmp(int a,int b){return dep[a]<dep[b];}
path merge(path u, path v){
 | int d[4], c[2];
 | if (!u.x||!v.x) return path(0, 0);
 | d[0]=lca(u.x,v.x); d[1]=lca(u.x,v.y);
 | d[2]=lca(u.y,v.x); d[3]=lca(u.y,v.y);
 | c[0]=lca(u.x,u.y); c[1]=lca(v.x,v.y);
 | sort(d,d+4,cmp); sort(c,c+2,cmp);
 | if (dep[c[0]] <= dep[d[0]] && dep[c[1]] <= dep[d[2]])
 |  | return path(d[2],d[3]);
 | else return path(0, 0); }
```

2.19.6 带修改MST

维护少量修改的 MST (银川 21: 求有 16 个 'e1 or e2' 的限制条件的MST)
找出必须边将修改边标 $-\infty$, 在MST上的其余边为必须边, 以此缩点.
找出无用边将修改边标 ∞, 不在MST上的其余边为无用边, 删除之.
假设修改边数为 k, 操作后图中最多剩下 $k+1$ 个点和 $2k$ 条边.

2.19.7 差分约束

$x_r - x_l \le c$:add(l, r, c) $x_r - x_l \ge c$:add(r, l, -c)

2.19.8 Segment Tree Beats

区间 min, 区间求和. 维护最大值 m, 严格次大值 s 以及最大值个数 t. 现在假设我们要让区间 $[L,R]$ 对 x 取 min, 先在线段树中定位若干个节点, 对于每个节点分三种情况讨论: 1, 当 $m \le x$ 时直接退出; 2, 当 $se < x < ma$ 时, 只会影响到所有最大值, 所以把 num 加上 $t * (x - ma)$, 把 ma 更新为 x, 打上标记退出; 3, 当 $se \ge x$ 时递归. 均摊 $O(\log^2 n)$.

2.19.9 二分图

最小点覆盖=最大匹配数. 独立集与覆盖集互补. 最小点覆盖构造方法: 对二分图流图求割集, 跨过的边指示最小点覆盖. **Hall定理** $G = (X, Y, E), |M| = |X| \Leftrightarrow \forall S \subseteq X, |S| \le |A(S)|$.

2.19.10 稳定婚姻问题

男士按自己喜欢程度从高到底依次向每位女士求婚, 女士遇到更喜欢的男士时就接受他, 并抛弃以前的配偶. 被抛弃的男士继续按照列表向剩下的女士依次求婚, 直到所有人都有配偶. 算法一定能得到一个匹配, 而且这个匹配一定是稳定的. 时间复杂度 $O(n^2)$.

2.19.11 竞赛图 Landau's Theorem

n 个点竞赛图点按出度按升序排序, 前 i 个点的出度之和不小于 $\frac{i(i-1)}{2}$, 度数总和等于 $\frac{n(n-1)}{2}$. 否则可以用优先队列构造出方案.

2.19.12 Ramsey Theorem R(3,3)=6, R(4,4)=18

6 个人中存在 3 人相互认识或者相互不认识.

2.19.13 树的计数 Prufer序列

prufer编码长度为 $n-2$, 且度数为 d_i 的点在prufer编码中出现 $d_i - 1$ 次.
由树得到序列: 总共需要 $n-2$ 步, 第 i 步在当前的树中寻找具有最小标号的叶子节点, 将与其相连的点的标号设为Prufer序列的第 i 个元素 p_i, 并将此叶子节点从树中删除, 直到最后得到一个长度为 $n-2$ 的Prufer 序列和一个只有两个节点的树.
由序列得到树: 先将所有点的度赋初值为 1, 然后加上它的编号在Prufer序列中出现的次数, 得到每个点的度; 执行 $n-2$ 步, 第 i 步选取具有最小标号的度为 1 的点 u 与 $v = p_i$ 相连, 得到树中的一条边, 并将 u 和 v 的度减一. 最后再把剩下的两个度为 1 的点连边, 加入到树中.
相关结论: n 个点完全图, 每个点度数依次为 d_1,d_2,...,d_n, 这样生成树的棵树为: $\frac{(n-2)!}{(d_1-1)!(d_2-1)!\ldots(d_n-1)!}$.
左边有 n_1 个点, 右边有 n_2 个点的完全二分图的生成树棵树为 $n_1^{n_2-1} \times n_2^{n_1-1}$.
m 个连通块, 每个连通块有 c_i 个点, 把他们全部连通的生成树方案数: $(\sum c_i)^{m-2} \prod c_i$

2.19.14 有根树计数 1,1,2,4,9,20,48,115,286,719,1842,4766

无标号 $a_{n+1} = 1/n \sum_{k=1}^{n} (\sum_{d|k} d \cdot a(d)) \cdot a(n-k+1)$

2.19.15 无根树计数

n 是奇数时, 有 $a_n - \sum_i^{n/2} a_i a_{n-i}$ 种不同的无根树.
n 时偶数时, 有 $a_n - \sum_i^{n/2} a_i a_{n-i} + \frac{1}{2} a_{n/2}(a_{n/2}+1)$ 种不同的无根树.

2.19.16 生成树计数 Kirchhoff's Matrix-Tree Thoerem

Kirchhoff Matrix $T = Deg - A$, Deg 是度数对角阵, A 是邻接矩阵. 无向图度数矩阵是每个点度数; 有向图度数矩阵是每个点入度.
邻接矩阵 $A[u][v]$ 表示 $u \to v$ 边个数, 重边按照边数计算, 自环不计入度数.
无向图生成树计数: $c = |K$ 的任意1个 $n-1$ 阶主子式 $|$
有向图外向树计数: $c = |$ 去掉根所在的那阶得到的主子式 $|$

2.19.17 有向图欧拉回路计数 BEST Thoerem

$$\text{ec}(G) = t_w(G) \prod_{v \in V} (\deg(v) - 1)!$$

其中 deg 为入度 (欧拉图中等于出度), $t_w(G)$ 为以 w 为根的外向树的个数. 相关计算参考生成树计数.
欧拉连通图中任意两点外向树个数相同: $t_v(G) = t_w(G)$.

2.19.18 Tutte Matrix

Tutte matrix A of a graph $G = (V, E)$:

$$A_{ij} = \begin{cases} x_{ij} & \text{if } (i,j) \in E \text{ and } i < j \\ -x_{ij} & \text{if } (i,j) \in E \text{ and } i > j \\ 0 & \text{otherwise} \end{cases}$$

where x_{ij} are indeterminates. The determinant of this skew-symmetric matrix is then a polynomial (in the variables x_{ij}, $i < j$): this coincides with the square of the pfaffian of the matrix A and is non-zero (as a polynomial) if and only if a perfect matching exists.

2.19.19 Edmonds Matrix

Edmonds matrix A of a balanced ($|U| = |V|$) bipartite graph $G = (U, V, E)$:

$$A_{ij} = \begin{cases} x_{ij} & (u_i, v_j) \in E \\ 0 & (u_i, v_j) \notin E \end{cases}$$

where the x_{ij} are indeterminates. G 有完美匹配当且仅当关于 x_{ij} 的多项式 $det(A_{ij})$ 不恒为 0. 完美匹配的个数等于多项式中单项式的个数.

2.19.20 有向图无环定向, 色多项式

图的色多项式 $P_G(q)$ 对图 G 的 q-染色计数.
Triangle K_3 : $x(x-1)(x-2)$
Complete graph K_n : $x(x-1)(x-2)\cdots(x-(n-1))$
Tree with n vertices : $x(x-1)^{n-1}$
Cycle C_n : $(x-1)^n + (-1)^n(x-1)$
acyclic orientations of an n-vertex graph G is $(-1)^n P_G(-1)$.

2.19.21 拟阵交问题

拟阵定义: , S, T 是独立集, 则 S 子集是, 若 $|S| > |T|$, 则 S 能扩充 T. 最大带权拟阵交问题: 全集 U 中每个元素都有权值 w_i. 设**同一个**全集 U 上有两个满足拟阵性质的集族 $\mathcal{F}_1, \mathcal{F}_2$. 对于 $k = 1..|U|$, 分别求出一个集合 S, 满足 $S \in \mathcal{F}_1 \cap \mathcal{F}_2$ 且 $|S|$ 恰好为 k 的前提下, S 中元素权值和最小.
设集合大小为 k 时已经求出了答案 S. 现在希望求出集合大小为 $k+1$ 的答案. U 中所有元素分为两个集合: 当前答案集合 S, 和剩余集合 $T = U \backslash S$. 考虑 T 中的某个元素 x_i. 记 $A = \{x_i | S \cup \{x_i\} \in \mathcal{F}_1\}$, $B = \{x_i | S \cup \{x_i\} \in \mathcal{F}_2\}$. 如果 T 中某个元素 $x_i \notin A$, 说明 x_i 加进 S 中形成了某个" 环", 从而不满足 $\mathcal{F}_1$ 的限制. 考虑这个" 环" 上每个元素 y_j, 满足 $S \backslash \{y_j\} \cup \{x_i\} \in \mathcal{F}_1$, 将 y_j 向每个 x_i 连边. 如果 T 中某个元素 $x_i \notin B$, 同理找出 S 中每一个元素 y_j 使得 $S \backslash \{y_j\} \cup \{x_i\} \in \mathcal{F}_2$, 将 x_i 向 y_j 连边. 现在求出从 A 到 B 的多源多汇最短路, 权值在点上, 若点属于 T 则权值为正, 否则属于 S, 权值为负. 最短路上每个 T 中的点放进 S, S 中的点放进 T, 则完成了一次增广. 由于每次增广路的起点和终点都在 T 中, 所以每次增广都会使得 $|S|$ 增加1.
最大拟阵交问题可以去掉权值直接求增广路.

2.19.22 双极定向

```cpp
//双极定向: 给定无向图和两个极点s,t, 要求将每条边定向后成为DAG, 使得s可达所有点, 所有点均可达t
//topo为定向后DAG的拓扑序, 边 (u,v) 定向为u->v当且仅当拓扑序中u在v的前面.
int n, dfn[N], low[N], stamp, p[N], preorder[N], topo[N];
bool fucked = 0, sign[N]; vector<int> G[N];
void dfs(int x, int fa, int s, int t){
  dfn[x] = low[x] = ++stamp;
  preorder[stamp] = x, p[x] = fa;
  if (x == s) dfs(t, x, s, t);
  for (int y : G[x]){
    if (x == s && y == t) continue;
    if (!dfn[y]){
      if (x == s) fucked = true;
      dfs(y, x, s, t);
      low[x] = min(low[x], low[y]); }
    else if (dfn[y] < dfn[x] && y != fa)
      low[x] = min(low[x], dfn[y]); } }
bool bipolar_orientation(int s, int t){
  G[s].push_back(t), G[t].push_back(s);
  stamp = fucked = 0, dfs(s, s, s, t);
  for (int i = 1; i <= n; i++)
    if (i != s && (!dfn[i] || low[i] >= dfn[i]))
      fucked = true;
  if (fucked) return false;
  sign[s] = 0;//memset sign[] is not necessary
  int pre[n + 5], suf[n + 5]; // list
  suf[0] = s; pre[s] = 0, suf[s] = t;
  pre[t] = s, suf[t] = n + 1; pre[n + 1] = t;
  for (int i = 3; i <= n; i++){
    int v = preorder[i];
    if (!sign[preorder[low[v]]]){ // insert before p[v]
      int P = pre[p[v]];
      pre[v] = P, suf[v] = p[v];
      suf[P] = pre[p[v]] = v; }
    else{ // insert after p[v]
      int S = suf[p[v]];
      pre[v] = p[v], suf[x] = S;
      suf[p[v]] = pre[S] = v; }
    sign[p[x]] = !sign[preorder[low[x]]]; }
  for (int x = s, cnt = 0; x != n + 1; x = suf[x])
    topo[++cnt] = x;
  return true; }
```

2.19.23 图中的环

没有奇环的图是**二分图**, 没有偶环的图是**仙人掌**. 判定没有奇环仅用深度奇偶性判即可; 判定没有偶环的图需要记录覆盖次数判定是否存在奇环有交.

3. Data Structure

3.1 非递归线段树

3.1.1 区间加, 区间求最大值

```cpp
void update(int l, int r, int d) {
  for (l += M-1, r += M+1; l^r^1; l >>= 1, r >>= 1) {
    if (l < M) {
      t[l] = max(t[l*2], t[l*2+1]) + mark[l];
      t[r] = max(t[r*2], t[r*2+1]) + mark[r]; }
    if (~l & 1) { t[l ^ 1] += d; mark[l ^ 1] += d; }
    if (r & 1) { t[r ^ 1] += d; mark[r ^ 1] += d; } }
  for (; l; l >>= 1, r >>= 1)
    if (l < M) t[l] = max(t[l*2], t[l*2+1]) + mark[l],
               t[r] = max(t[r*2], t[r*2+1]) + mark[r]; }
int query(int l, int r) {
  int maxl = -INF, maxr = -INF;
  for (l += M-1, r += M+1; l^r^1; l >>= 1, r >>= 1) {
    maxl += mark[l]; maxr += mark[r];
    if (~l & 1) maxl = max(maxl, t[l ^ 1]);
    if ( r & 1) maxr = max(maxr, t[r ^ 1]); }
  while (l) { maxl += mark[l]; maxr += mark[r];
    l >>= 1; r >>= 1; }
  return max(maxl, maxr); }
```

3.2 点分治

```cpp
vector<pair<int, int> > G[maxn];
int sz[maxn], son[maxn], q[maxn];
int pr[maxn], depth[maxn], rt[maxn][19], d[maxn][19];
int cnt_all[maxn],sum_all[maxn],cnt[maxn][],sum[maxn][];
bool vis[maxn], col[maxn];
int getcenter(int o, int s) {
  int head = 0, tail = 0; q[tail++] = o;
  while (head != tail) {
    int x = q[head++]; sz[x] = 1; son[x] = 0;
    for (auto [y, _] : G[x]) if (!vis[y] && y != pr[x]) {
      pr[y] = x; q[tail++] = y; } }
  for (int i = tail - 1; i; i--) {
    int x = q[i]; sz[pr[x]] += sz[x];
    if (sz[x] > sz[son[pr[x]]]) son[pr[x]] = x; }
  int x = q[0];
  while (son[x] && sz[son[x]] * 2 >= s) x = son[x];
  return x; }
void getdis(int o, int k) {
  int head = 0, tail = 0; q[tail++] = o;
  while (head != tail) {
    int x = q[head++]; sz[x] = 1; rt[x][k] = o;
    for (auto [y, w] : G[x]) if (!vis[y] && y != pr[x]) {
      pr[y]=x; d[y][k] = d[x][k] + w; q[tail++]=y; } }
  for (int i = tail - 1; i; i--)sz[pr[q[i]]] += sz[q[i]];}
void build(int o, int k, int s, int fa) {
  int x = getcenter(o, s);
  vis[x] = true; depth[x] = k; pr[x] = fa;
  for (auto [y, w] : G[x]) if (!vis[y]) {
    d[y][k] = w; pr[y] = x; getdis(y, k); }
  for (auto [y, w] : G[x]) if (!vis[y])
    build(y, k + 1, sz[y], x); }
void modify(int x) {
  int t = col[x] ? -1 : 1; cnt_all[x] += t;
  for (int u = pr[x], k = depth[x] - 1; u; u = pr[u],k--){
    sum_all[u] += t * d[x][k]; cnt_all[u] += t;
    sum[rt[x][k]][k] += t*d[x][k]; cnt[rt[x][k]][k] += t;
  } col[x] ^= true; }
int query(int x) { int ans = sum_all[x];
  for (int u = pr[x], k = depth[x] - 1; u; u = pr[u], k--)
    ans += sum_all[u] - sum[rt[x][k]][k]
      + d[x][k] * (cnt_all[u] - cnt[rt[x][k]][k]);
  return ans; }
```

3.3 LCT 动态树

```
// 记得初始化 mn; 维护虚子树: access link cut pushup
int fa[MX], ch[MX][2], w[MX], mn[MX], mark[MX];
int get(int x) {return x == ch[fa[x]][1];}
int nrt(int x) {return get(x) || x == ch[fa[x]][0];}
void pushup(int x) {
 | mn[x] = w[x];
 | if (lch(x)) mn[x] = min(mn[x], mn[lch(x)]);
 | if (rch(x)) mn[x] = min(mn[x], mn[rch(x)]); }
void rev(int x) {mark[x] ^= 1, swap(lch(x), rch(x));}
void pushdown(int x) {
 | if (mark[x]) {
 | | if (lch(x)) rev(lch(x));
 | | if (rch(x)) rev(rch(x));
 | | mark[x] = false; } }
void rot(int x) {
 | int f = fa[x], gf = fa[f];
 | int which = get(x), W = ch[x][!which];
 | if (nrt(f)) ch[gf][ch[gf][1] == f] = x;
 | ch[x][!which] = f, ch[f][which] = W;
 | if (W) fa[W] = f;
 | fa[f] = x, fa[x] = gf;
 | pushup(f); }
void splay(int x) {
 | static int stk[MX];
 | int f = x, dep = 0; stk[++dep] = f;
 | while (nrt(f)) stk[++dep] = f = fa[f];
 | while (dep) pushdown(stk[dep--]);
 | while (nrt(x)) {
 | | if (nrt(f = fa[x])) rot(get(x) == get(f) ? f : x);
 | | rot(x);
 | } pushup(x); }
void access(int x) {
 | for(int y = 0 ; x ; x = fa[y = x])
 | | splay(x), rch(x) = y, pushup(x); }
void makeroot(int x) {access(x), splay(x), rev(x);}
void split(int x, int y) {makeroot(x), access(y), splay(y);}
int findroot(int x) {
 | access(x), splay(x);
 | while (lch(x)) pushdown(x), x = lch(x);
 | return splay(x), x; | }
void link(int x, int y) {
 | makeroot(x);
 | if (findroot(y) != x) fa[x] = y; }
void cut(int x, int y) {
 | makeroot(x);
 | if (findroot(y) != x || fa[y] != x || lch(y)) return;
 | rch(x) = fa[y] = 0, pushup(x); }
```

3.4 可持久化 Treap

```
/* 不可持久化：把 copy(a, b) 换成 a = b, 并且去除新建结点 */
const int MX = (2e5 + 233) * 18 * 8;
int vcnt;
struct node {
  int sz, ch[2], pri;
  int rev; LL sum; int val;
} tr[MX];
int newnode(int v) {
  static mt19937 rng(114514);
  int x = ++vcnt;
  tr[x].sz = 1;
  lch = rch = 0;
  tr[x].pri = rng();
  tr[x].sum = tr[x].val = v;
  tr[x].rev = false;
  return x; }
void copy(int x, int y) { tr[x] = tr[y]; }
int merge(int x, int y) {
  if (!x || !y) return x + y;
  int z = ++vcnt;
  if (tr[x].pri < tr[y].pri) {
    pushdown(x); copy(z, x);
    tr[z].ch[1] = merge(tr[z].ch[1], y);
  } else {
    pushdown(y); copy(z, y);
    tr[z].ch[0] = merge(x, tr[z].ch[0]); }
  pushup(z); return z; }
void split(int x, int dsz, int &r1, int &r2) {
  if (!x) {
    r1 = r2 = 0;
  } else {
    pushdown(x);
    if (tr[lch].sz + 1 <= dsz) {
      r1 = ++vcnt; copy(r1, x);
      split(tr[r1].ch[1], dsz - 1 - tr[lch].sz
        , tr[r1].ch[1], r2);
      pushup(r1);
    } else {
      r2 = ++vcnt; copy(r2, x);
      split(tr[r2].ch[0], dsz, r1, tr[r2].ch[0]);
      pushup(r2); } } }
```

4. String

4.1 最小表示法

```
int min_pos(vector<int> a) { // 0-based
 | int n = a.size(), i = 0, j = 1, k = 0;
 | while (i < n && j < n && k < n) {
 | | auto u = a[(i + k) % n]; auto v = a[(j + k) % n];
 | | int t = u > v ? 1 : (u < v ? -1 : 0);
 | | if (t == 0) k++; else {
 | | | if (t > 0) i += k + 1; else j += k + 1;
 | | | if (i == j) j++;
 | | | k = 0; } } return min(i, j); }
```

4.2 Manacher

```
// n为串长，回文半径输出到p数组中，数组要开串长的两倍
void manacher(const char *t, int n) {
 | static char s[MAXN * 2];
 | for (int i = n; i; i--) s[i * 2] = t[i];
 | for (int i = 0; i <= n; i++) s[i * 2 + 1] = '#';
 | s[0] = '$'; s[(n + 1) * 2] = '\0'; n = n * 2 + 1;
 | int mx = 0, j = 0;
 | for (int i = 1; i <= n; i++) {
 | | p[i] = (mx > i ? min(p[j * 2 - i], mx - i) : 1);
 | | while (s[i - p[i]] == s[i + p[i]]) p[i]++;
 | | if (i + p[i] > mx) { mx = i + p[i]; j = i; } } }
```

4.3 KMP, exKMP

```
void kmp(char *s, int n) { // 1-based
 | fail[0] = fail[1] = 0;
 | for (int i = 1; i < n; i++) { int j = fail[i];
 | | while (j && s[i + 1] != s[j + 1]) j = fail[j];
 | | if (s[i + 1] == s[j + 1]) fail[i + 1] = j + 1;
 | | else fail[i + 1] = 0; } }
void exkmp(char *s, int *a, int n) { // 1-based
 | int l = 0, r = 0; a[1] = n;
 | for (int i = 2; i <= n; i++) {
 | | a[i] = i > r ? 0 : min(r - i + 1, a[i - l + 1]);
 | | while (i+a[i] <= n && s[1+a[i]] == s[i+a[i]]) a[i]++;
 | | if (i + a[i] - 1 > r) {l = i; r = i + a[i] - 1;}}}
```

4.4 AC 自动机

注意代码是以 0 为根的，如果要 1-base 的话要改一下没有儿子时的逻辑。

```
int ch[MAXN][26], fail[MAXN], q[MAXN], cnt = 0;
int insert(const char *c) { int x = 0; while (*c) {
 | if (!ch[x][*c - 'a']) ch[x][*c - 'a'] = ++cnt;
 | x = ch[x][*c++ - 'a']; } return x; }
void getfail() { int x, head = 0, tail = 0;
 | for (int c = 0; c < 26; c++) if (ch[0][c])
 | | | q[tail++] = ch[0][c];
 | while (head != tail) { x = q[head++];
 | | for (int c = 0; c < 26; c++) { if (ch[x][c]) {
 | | | | fail[ch[x][c]] = ch[fail[x]][c];
 | | | | q[tail++] = ch[x][c];
 | | | } else ch[x][c] = ch[fail[x]][c]; } } }
```

4.5 Lydon Word Decomposition

```
//满足s的最小后缀等于s本身的串s称为Lyndon串.
//等价于: s是它自己的所有循环移位中唯一最小的一个.
//任意字符串s可以分解为 s = s1s2sk, 其中 si 是Lyndon串,
  ↪ si ≥ si+1. 且这种分解方法是唯一的.
//后缀排序后, 排名的所有前缀最小值构成了 Ly 分解的左端点。
void mnsuf(char *s, int *mn, int n){ // 每个前缀的最小后缀
 | //1 - base , 求 Lyndon 分解去掉 mn 即可
    for(int i = 1; i <= n;)
        int j = i + 1, k = i; mn[i] = i;
        for(; j <= n && s[k] <= s[j]; j++){
            if(s[k] < s[j]) k = mn[j] = i;
                else mn[j] = mn[k] + j - k, k++;
        for(; i <= k; i += j - k) {} } } //
          ↪ lyn+=s[i..i+k-j-1]
```

```
void mxsuf(char *s, int *mx, int n){ // 每个前缀的最大后缀
    fill(mx + 1, mx + n + 1, 0); // 1 - base
    for(int i = 1; i <= n;){
        int j = i + 1, k = i; !mx[i] ? mx[i] = i : 0;
        for(; j <= n && s[k] >= s[j]; j++){
            !mx[j] ? mx[j] = i : 0;
            s[k] > s[j] ? k = i : k++; }
        for(; i <= k; i += j - k) {} } }
```

4.6 后缀自动机

```
int last, mxl[N * 2], par[N * 2], sam_cnt;
array <int, 26> go[N * 2];
void extend(int c) {
 | int p = last, np = ++sam_cnt; mxl[np] = mxl[p] + 1;
 | while (p && !go[p][c]) { go[p][c] = np; p = par[p]; }
 | if (!p) par[np] = 1; else { int q = go[p][c];
 |  | if (mxl[q] == mxl[p] + 1) par[np] = q;
 |  | else { int nq = ++sam_cnt; mxl[nq] = mxl[p] + 1;
 |  |  | go[nq] = go[q];
 |  |  | par[nq] = par[q]; par[np] = par[q] = nq;
 |  |  | while (p && go[p][c] == q) { go[p][c] = nq;
 |  |  |  | p = par[p]; } } } last = np; }
void init() { last = sam_cnt = 1; }
int c[N], q[N * 2];
void solve() { // 跑完得到的q是一个合法的拓扑序，c 记得清空
 | for (int i = 1; i <= sam_cnt; i++) c[mxl[i] + 1]++;
 | for (int i = 1; i <= n; i++) c[i] += c[i - 1]; // n: 串长
 | for (int i = 1; i <= sam_cnt; i++) q[++c[mxl[i]]] = i;}
```

4.7 SAMSA & 后缀树

```
bool vis[MAXN * 2]; char s[MAXN];
int id[MAXN * 2], ch[MAXN * 2][26], height[MAXN], stamp = 0;
void dfs(int x) {
 | if (id[x]) { height[stamp++] = val[last];
 |  | sa[stamp] = id[x]; last = x; }
    for (int c = 0; c < 26; c++)
 |  | if (ch[x][c]) dfs(ch[x][c]);
 | last = par[x]; }
int main() { last = ++cnt; scanf("%s", s + 1);
    int n = strlen(s + 1); for (int i = n; i; i--) {
 |  | expand(s[i] - 'a'); id[last] = i; }
 | vis[1] = true; for (int i = 1; i <= cnt; i++) if (id[i])
 |  |  | for (int x = i,pos = n; x && !vis[x]; x = par[x]){
 |  |  |  | vis[x] = true; pos -= val[x] - val[par[x]];
 |  |  |  | ch[par[x]][s[pos + 1] - 'a'] = x; }
 | dfs(1); for (int i = 1; i <= n; i++)
 |  | printf("%d%c", sa[i], i < n ? ' ' : '\n');
 | for (int i = 1; i < n; i++) printf("%d%c", height[i],
       i < n ? ' ' : '\n'); return 0; }
```

4.8 后缀数组

```
// height[i] = lcp(sa[i], sa[i - 1])
// 如果有多组数据，全部都清空到 max(n, m)
constexpr int MAXN = 1000005;
void get_sa(char *s, int n, int *sa,
 |  | int *rnk, int *height) { // 1-based
 | static int buc[MAXN], id[MAXN], p[MAXN], t[MAXN];
 | int m = 300;
 | for (int i = 1; i <= n; i++) buc[rnk[i] = s[i]]++;
 | for (int i = 1; i <= m; i++) buc[i] += buc[i - 1];
 | for (int i = n; i; i--) sa[buc[rnk[i]]--] = i;
 | memset(buc, 0, sizeof(int) * (m + 1));
 | for (int k = 1, cnt = 0; cnt != n; k *= 2, m = cnt) {
 |  | cnt = 0;
 |  | for (int i = n; i > n - k; i--) id[++cnt] = i;
 |  | for (int i = 1; i <= n; i++)
 |  |  | if (sa[i] > k) id[++cnt] = sa[i] - k;
 |  | for (int i = 1; i <= n; i++) buc[p[i]=rnk[id[i]]]++;
 |  | for (int i = 1; i <= m; i++) buc[i] += buc[i - 1];
 |  | for (int i = n; i; i--) sa[buc[p[i]]--] = id[i];
 |  | memset(buc, 0, sizeof(int) * (m + 1));
 |  | memcpy(t, rnk, sizeof(int) * (n + 1));
 |  | t[n + 1] = 0; // 记得清空 n + 1
 |  | cnt = 0; for (int i = 1; i <= n; i++) {
 |  |  | if (t[sa[i]] != t[sa[i - 1]] ||
 |  |  |  | t[sa[i] + k] != t[sa[i - 1] + k]) cnt++;
 |  |  | rnk[sa[i]] = cnt; } }
 | for (int i = 1; i <= n; i++) sa[rnk[i]] = i;
 | for (int i = 1, k = 0; i <= n; i++) { if (k) k--;
 |  | if (rnk[i] > 1) while (sa[rnk[i] - 1] + k <= n &&
 |  |  |  | s[i + k] == s[sa[rnk[i] - 1] + k]) k++;
 |  | height[rnk[i]] = k; } } // 两个都要判，否则会左/右越界
```

```
char s[MAXN]; int sa[MAXN], rnk[MAXN], height[MAXN];
int main() { scanf("%s", s + 1); int n = strlen(s + 1);
 | get_sa(s, n, sa, rnk, height); }
```

4.9 Suffix Balanced Tree 后缀平衡树

```
// 后缀平衡树每次在字符串开头添加或删除字符，考虑在当前字符串 S 前插入一个字符 c，那么相当于在后缀平衡树中插入一个新的后缀 cS，简单的话可以使用预处理哈希二分 LCP 判断两个后缀的大小作 cmp，直接写 set，时间复杂度 O(nlg^2n). 为了方便可以把字符反过来做
// 例题 : 加一个字符或删一个字符，同时询问不同子串个数
struct cmp{
 | bool operator()(int a,int b){
 |  | int p=lcp(a,b);//注意这里是后面加，lcp是反过来的
 |  | if(a==p)return 0;if(b==p)return 1;
 |  | return s[a-p]<s[b-p];}
};set<int,cmp>S;set<int,cmp>::iterator il,ir;
void del(){S.erase(L--);}//在后面删字符
void add(char ch){//在后面加字符
 | s[++L]=ch;mx=0;il=ir=S.lower_bound(L);
 | if(il!=S.begin())mx=max(mx,lcp(L,*--il));
 | if(ir!=S.end())mx=max(mx,lcp(L,*ir));
 | an[L]=an[L-1]+L-mx;S.insert(L); }
LL getan(){printf("%lld\n",an[L]);}//询问不同子串个数
```

4.10 广义在线 SAM

```
struct SAM{
int tot,fail[MM],len[MM],t[MM][26];
SAM(){tot=1;}
int insert(int c,int last){
 | if(t[last][c]){
 |  | int p=last,q=t[p][c];
 |  | if(len[p]+1==len[q])return q;
 |  | else { int nq=++tot;
 |  |  | fail[nq]=fail[q];fail[q]=nq;
 |  |  | len[nq]=len[p]+1;memcpy(t[nq],t[q],sizeof(t[q]));
 |  |  | for(;p && t[p][c]==q;p=fail[p])t[p][c]=nq;
 |  |  | //可以直接复制下面的代码。
 |  |  | return nq; } }
 | int p=last,np=++tot;
 | len[np]=len[p]+1;
 | for(;p && !t[p][c];p=fail[p])t[p][c]=np;
 | if(!p)fail[np]=1;
 | else {
 |  | int q=t[p][c];
 |  | if(len[q]==len[p]+1)fail[np]=q;
 |  | else { int nq=++tot;
 |  |  | fail[nq]=fail[q];fail[q]=nq;
 |  |  | len[nq]=len[p]+1;memcpy(t[nq],t[q],sizeof(t[q]));
 |  |  | for(;p && t[p][c]==q;p=fail[p])t[p][c]=nq;
 |  |  | fail[np]=nq; } }
 | return np; } }sam;
// scanf("%s",st+1);int slen=strlen(st+1);
// int last=1;
// for(int j=1;j<=slen;j++)last=sam.insert(st[j]-'a',last);
```

4.11 回文树

0 的子树是长为偶数的串, 1 的子树是长为奇数的. 0 代表空串, 1 没有意义.

```
int len[N], fail[N], go[N][26], last, pam_cnt; char s[N];
int dif[N], slink[N], g[N]; // 最小回文划分相关的 dp
void clgo(int x){memset(go[x], 0, sizeof(go[x]));}
void init(){ fail[0] = pam_cnt = 1; len[1] = -1;
 | last = 0; clgo(0); clgo(1);}
int getfail(int n, int p){
 | while (s[n - len[p] - 1] != s[n]) p = fail[p];
 | return p;}
void extend(int n) {// 1 - base
 | int p = getfail(n, last), c = s[n] - 'a';
 | if (!go[p][c]) { int q = ++pam_cnt, now = p;
 |  | clgo(q);
 |  | len[q] = len[p] + 2;
 |  | p = getfail(n, fail[p]);
 |  | fail[q] = go[p][c]; last = go[now][c] = q;
 |  | dif[q] = len[q] - len[fail[q]];
 |  | slink[q] = dif[q] == dif[fail[q]] ?
 |  | slink[fail[q]] : fail[q];} else last = go[p][c];}
void solve(){
 | for(int x = last; x > 1; x = slink[x]){
 |  | g[x] = dp[i - len[slink[x]] - dif[x]];
 |  | if(dif[x] == dif[fail[x]]) g[x] <- g[fail[x]];
```

```
 |  | dp[i] <- g[x]; } }
//双端插入: 前缀后缀分别写一个last和getfail, 回文所以只需要维护回文后缀, 当last=串长的时候两个last要等于整个串
//去均摊:quick[x][c] 表示x的最长真回文后缀能匹配上c, 初始化quick[0] 全为1
```

4.12 Runs

```
struct Runs{
    int l,r,p;
};vector<Runs> run;
bool operator==(Runs x,Runs y){return x.l==y.l && x.r==y.r;}
int gl(int x,int y); // 求 S[1,x],S[1,y] 的最长公共后缀
int gr(int x,int y); // 求 S[x,n],S[y,n] 的最长公共前缀
//上面两个可以用 二分 + Hash 或者后缀数组实现。
bool getcmp(int x,int y){//S[x,n]<S[y,n]
 | int len=gr(x,y);
 | return st[x+len]<st[y+len];}
int ly[N];
void lyndon(bool type){//后缀排序法求 Lyndon
 | stack<PII> stk;stk.push({n,n});ly[n]=n;
 | for(int i=n-1;i>=1;i--){
 |  | int now=i;
 |  | while(!stk.empty() && getcmp(i,stk.top().first)!=type)
 |  |  | now=stk.top().second,stk.pop();
 |  | ly[i]=now;
 |  | stk.push({i,now});
 | } }
void getrun(){
 | for(int l=1;l<=n;l++){
 |  | int r=ly[l],ll=l,rr=r;
 |  | if(l!=1)ll-=gl(l-1,r);
 |  | if(r!=n)rr+=gr(l,r+1);
 |  | if(rr-ll+1>=2*(r-l+1))run.push_back({ll,rr,r-l+1}); }
       }
void solve(){
 | st[n + 1] = '\0'; run.clear();
 | init();//Hash 或者 SA 的启动
 | for(int op=0;op<=1;op++){//0正常字典序,1反序
 |  | lyndon(op); getrun(); }
 | sort(run.begin(),run.end(),[](Runs x, Runs y){
        return x == y ? x.p < y.p : (x.l != y.l ? x.l < y.l : x.r < y.r);});
 | run.erase(unique(run.begin(),run.end()),run.end()); }
```

4.13 字符串 Hash

Random primes generated at Wed Jul 29 21:33:04 2026
3e2 179 197 199 223 239 269 271 283 337 349 359 383 397 409
1e6 941839 966527 993961 999149 1024421 1026251 1050737
1e12 993456341723 994567457593 1012341235883 1023452348473
1e13 9912341234179 10123412349007 10345634560631 10345634562217
2e13 19345634560223 20234523450937 20345634560327 20345634568433
1e15 991234123413817 993456345631987 1012341234128507
1e17 99567856785672557 101234123412342223 105678567856780511
5e17 491234123412342883 494567456745678059 495678567856782493
1e18 1045674567456748399 1056785678567857099 1056785678567857159
2e18 2012341234123414967 2034563456345634091 2056785678567854141

```
static constexpr u128 inv = []() {
 | u128 ret = P;
 | for (int i = 0; i < 6; i++) ret *= 2 - ret * P;
 | return ret; }();
constexpr u128 chk = u128(-1) / P;
bool check(i128 a, i128 b) {
 | if (a < b) swap(a, b);
 | return (a - b) * inv <= chk; }
```

4.14 String Conclusions

双回文串

如果 $s = x_1x_2 = y_1y_2 = z_1z_2, |x_1| < |y_1| < |z_1|, x_2, y_1, y_2, z_1$ 是回文串, 则 x_1 和 z_2 也是回文串.

Border 和周期

如果 r 是 S 的一个border, 则 $|S| - r$ 是 S 的一个周期.

如果 p 和 q 都是 S 的周期, 且满足 $p + q \le |S| + gcd(p, q)$, 则 $gcd(p, q)$ 也是一个周期.

字符串匹配与Border

若字符串 S, T 满足 $2|S| \ge |T|$, 则 S 在 T 中所有匹配位置成等差数列.

若 S 的匹配次数大于2, 则等差数列的周期恰好等于 S 的最小周期.

Border 的结构

字符串 S 的所有不小于 $|S|/2$ 的border长度组成一个等差数列.

字符串 S 的所有 border 按长度排序后可分成 $O(\log |S|)$ 段, 每段是一个等差数列.

回文串Border

回文串长度为 t 的后缀是一个回文后缀, 等价于 t 是该串的border. 因此回文后缀的长度也可以划分成 $O(\log |S|)$ 段.

子串最小后缀

设 $s[p..n]$ 是 $s[i..n], (l \le i \le r)$ 中最小者, 则minsuf(l, r) 等于 $s[p..r]$ 的最短非空 border. minsuf(l, r) = min{$s[p..r]$, minsuf(r − 2^k + 1, r)}, $(2^k < rl + 1 \le 2^{k+1})$.

子串最大后缀

从左往右扫, 用set维护后缀的字典序递减的单调队列, 并在对应时刻添加”小于事件” 点以便在之后修改队列; 查询直接在set里lower_bound.

ZJJ: SAM处理手法

1. 基本子串结构: CLB 搞的那玩意。
2. 正反串 SAM 的基本联系: 一个子串出现的位置将会在两个SAM中同时得到映照。
3. SAM 上转成数点问题。
4. 线段树合并维护 endpos 集合。
5. 树剖保证到根的链上只涉及 log 次修改和查询。(区间 border)
6. LCT 保证到根的链只修改均摊 log 个不同的颜色段。(区间本质不同子串数量)

ZJJ: 字符串常见错误

1. 字符串算法变式记得判匹配位置超出字符串的情况，例如多组数据下的双端插入回文串，后缀数组多组。2. 警惕 char 运算中 ‘a’ 和 ‘␣a’ 的区别。3. char kmp[]

5. Math 数学

5.1 Long Long $O(1)$ 乘, Barrett

```
LL modmul(LL a, LL b, LL M) { // skip2004, M < 63bit
 | LL ret = a * b - M * LL(1.L * a / M * b + 0.5);
 | return ret < 0 ? ret + M : ret; }
ULL modmul(ULL a, ULL b, LL M) { // orz@CF, M in 63 bit
 | ULL c = (long double)a * b / M;
 | LL ret = LL(a * b - c * M) % LL(M); // must be signed
 | return ret < 0 ? ret + M : ret; }
// use int128 instead if M > 63 bit
struct DIV {
 | ULL p, ip;
 | void init (ULL _p) { p = _p; ip = -1llu / p; }
 | int mod (ULL x) { // x < 2 ^ 64
 |  | ULL q = ULL(((u128)ip * x) >> 64);
 |  | ULL r = x - q * p;
 |  | return int(r >= p ? r - p : r);
} }; // speedup only when mod is not const
```

5.2 exgcd, 逆元

假设我们已经找到了一组解 (p_0, q_0) 满足 $ap_0 + bq_0 = \gcd(a, b)$, 那么其他的解都满足

$$p = p_0 + \frac{b}{\gcd(p,q)} \times t \qquad q = q_0 - \frac{a}{\gcd(p,q)} \times t$$

其中t为任意整数.

```
LL exgcd(LL a, LL b, LL &x, LL &y) {
 | if (b == 0) return x = 1, y = 0, a;
 | LL t = exgcd(b, a % b, y, x);
 | y -= a / b * x; return t;}
LL inv(LL x, LL m) {
 | LL a, b; exgcd(x, m, a, b); return (a % m + m) % m; }
```

递推逆元: $\text{inv}(i) \equiv (P - P/i) \cdot \text{inv}(P \bmod i)$

5.3 CRT 中国剩余定理

```
bool crt_merge(LL a1, LL m1, LL a2, LL m2, LL &A, LL &M) {
LL c = a2 - a1, d = __gcd(m1, m2); //合并两个模方程
if(c % d) return 0; // gcd(m1, m2) | (a2 - a1) 时才有解
c = (c % m2 + m2) % m2; c /= d; m1 /= d; m2 /= d;
c = c * inv(m1 % m2, m2) % m2; //0逆元可任意值
M = m1*m2*d; A = (c *m1 %M *d %M +a1) % M; return 1;}//有解
```

5.4 Miller Rabin, Pollard Rho

```
mt19937 rng(123);
#define rand() LL(rng() & LLONG_MAX)
const int BASE[] = {2, 7, 61};//int(7,3e9)
//{2,325,9375,28178,450775,9780504,1795265022}LL(37)
struct miller_rabin {
bool check (const LL &M, const LL &base) {
 | LL a = M - 1;
 | while (~a & 1) a >>= 1;
 | LL w = power (base, a, M); // power should use mul
 | for (; a != M - 1 && w != 1 && w != M - 1; a <<= 1)
 |  | w = mul (w, w, M);
 | return w == M - 1 || (a & 1) == 1; }
bool solve (const LL &a) {//O((3 or 7) · log n · mul)
 | if (a < 4) return a > 1;
 | if (~a & 1) return false;
 | for (int i = 0; i < sizeof(BASE)/4 && BASE[i] < a; ++i)
 |  | if (!check (a, BASE[i])) return false;
 | return true; } };
miller_rabin is_prime;
LL get_factor (LL a, LL seed) {//O(n^1/4 · log n · mul)
 | LL x = rand () % (a - 1) + 1, y = x;
 | for (int head = 1, tail = 2; ; ) {
 |  | x = mul (x, x, a); x = (x + seed) % a;
 |  | if (x == y) return a;
 |  | LL ans = gcd (abs (x - y), a);
 |  | if (ans > 1 && ans < a) return ans;
 |  | if (++head == tail) { y = x; tail <<= 1; } } }
void factor (LL a, vector<LL> &d) {
 | if (a <= 1) return;
 | if (is_prime.solve (a)) d.push_back (a);
 | else {
 |  | LL f = a;
 |  | for (; f >= a; f = get_factor (a, rand() % (a - 1) +
 |  |  ↪ 1));
 |  | factor (a / f, d);
 |  | factor (f, d); } }
```

5.4.1 Miller–Rabin（个人代码）

```
ll qmul(ll a, ll b, ll p) {
    ll q = (long double)a / p * b;
    ll res = (uint64_t)a * b - (uint64_t)q * p;
    return (res + p) % p;
}

ll qmi(ll a, ll b, ll p) {
    ll res = 1;
    while (b) {
        if (b & 1) res = qmul(res, a, p);
        b >>= 1;
        a = qmul(a, a, p);
    }
    return res;
}

ll cand[7] = {2, 325, 9375, 28178, 450775, 9780504,
  ↪ 1795265022};
bool is_prime(ll n) {
    if (n < 3 || n % 2 == 0) return n == 2;
    ll u = n - 1, more = 0;
    while (u % 2 == 0) {
        u /= 2;
        ++more;
    }
    for (auto bas : cand) {
        ll r = qmi(bas, u, n);
        if (r == 0 || r == 1 || r == n - 1) continue;
        for (int i = 0; i < more; ++i) {
            r = qmul(r, r, n);
            if (r == n - 1 && i != more - 1) {
                r = 1;
                break;
            }
            if (r == 1) return false;
        }
        if (r != 1) return false;
    }
    return true;
}
```

5.4.2 Pollard–Rho 质因数分解（个人代码）

```
std::mt19937 rnd(12306);
std::mt19937_64 rndll(13331);

ll pollard(ll n) { // get_factor
    assert(n > 1);
    if (is_prime(n)) return n;
    if (n % 2 == 0) return 2;
    while (1) {
        ll x = 0, y = x;
        ll c = rndll() % (n - 1) + 1;
        auto f = [&](ll x) -> ll {
            return ((lll)x * x + c) % n;
        };
        ll now = 1, pre;
        do {
            for (int i = 0; i < 128; ++i) {
                x = f(x), y = f(f(y));
                pre = now;
                if ((now = (lll)pre * std::abs(y - x) % n)
                  ↪ == 0)
                    break;
            }
            // 此时 y == x, 下一轮退出, 不会影响后续。
            if (now == 0) now = pre;
            ll g = std::gcd(now, n);
            if (g > 1) return g;
        } while (y != x);
    }
}

// 得到所有素因子（有重复）。
std::vector<ll> PrimeFactorize(ll x) {
    std::vector<ll> ret;
    std::queue<ll> q;
    for (int i = 2; i <= 100; ++i)
        if (x % i == 0) {
            while (x % i == 0)
                x /= i, ret.push_back(i);
        }
    if (x != 1)
```

```
        q.push(x);
    while (q.size()) {
        ll t = q.front();
        q.pop();
        if (is_prime(t)) {
            ret.push_back(t);
            continue;
        }
        ll fac = pollard(t);
        q.push(fac), q.push(t / fac);
    }
    return std::move(ret);
}
```

5.5 扩展卢卡斯

```
int l,a[33],p[33],P[33];
U fac(int k,LL n){// 求 n! mod pk^tk, 返回值 U{ 不包含 pk 的值 ,pk 出现的次数 }
 | if (!n)return U{1,0};LL x=n/p[k],y=n/P[k],ans=1;int i;
 | if(y){// 求出循环节的答案
 |  | for(i=2;i<P[k];i++)if(i%p[k])ans=ans*i%P[k];
 |  | ans=Pw(ans,y,P[k]);
 | }for(i=y*P[k];i<=n;i++) if(i%p[k])ans=ans*i%M;// 求零散部分
 | U z=fac(k,x);return U{ans*z.x%M,x+z.z};
}LL get(int k,LL n,LL m){// 求 C(n,m) mod pk^tk
 | U a=fac(k,n),b=fac(k,m),c=fac(k,n-m);// 分三部分求解
 | return Pw(p[k],a.z-b.z-c.z,P[k])*a.x%P[k]*inv(b.x,P[k])%P[k]*inv(c.x,P[k])%P[k];
}LL CRT(){// CRT 合并答案
 | LL d,w,y,x,ans=0;
 | fr(i,1,l)w=M/P[i],exgcd(w,P[i],x,y),ans=(ans+w*x%M*a[i])%M;
 | return (ans+M)%M;
}LL C(LL n,LL m){// 求 C(n,m)
 | fr(i,1,l)a[i]=get(i,n,m);
 | return CRT();
}LL exLucas(LL n,LL m,int M){
 | int jj=M,i //求 C(n,m)mod M,M=prod(pi^ki), O(pi^kilg^2n)
 | for(i=2;i*i<=jj;i++)if(jj%i==0)
 |  | for(p[++l]=i,P[l]=1;jj%i==0;P[l]*=p[l])jj/=i;
 | if(jj>1)l++,p[l]=P[l]=jj;
 | return C(n,m);}
```

5.6 阶乘取模

```
// n! mod p^q Time : O(pq^2 log^2 n / log p)
// Output : {a, b} means a*p^b
using Val=unsigned long long; //Val 需要 mod p^q 意义下 + *
typedef vector<Val> poly;
poly polymul(const poly &a,const poly &b){
 | int n = (int) a.size(); poly c (n, Val(0));
 | for (int i = 0; i < n; ++ i) {
 |  | for (int j = 0; i + j < n; ++ j) {
 |  |  | c[i + j] = c[i + j] + a[i] * b[j]; } }
 | return c; } Val choo[70][70];
poly polyshift(const poly &a, Val delta) {
 | int n = (int) a.size(); poly res (n, Val(0));
 | for (int i = 0; i < n; ++ i) { Val d = 1;
 |  | for (int j = 0; j <= i; ++ j) {
 |  |  | res[i - j] = res[i - j]+a[i]*choo[i][j]*d;
 |  |  | d = d * delta; } } return res; }
void prepare(int q) {
 | for (int i = 0; i < q; ++ i) { choo[i][0] = Val(1);
 |  | for (int j = 1; j <= i; ++ j)
 |  |  | choo[i][j]=choo[i-1][j-1]+choo[i-1][j]; } }
pair<Val, LL> fact(LL n, LL p, LL q) { Val ans = 1;
 | for (int r = 1; r < p; ++ r) {
 |  | poly x (q, Val(0)), res (q, Val(0));
 |  | res[0] = 1; LL _res = 0; x[0] = r; LL _x = 0;
 |  | if (q > 1) x[1] = p, _x = 1; LL m = (n - r + p) / p;
 |  | while (m) { if (m & 1) {
 |  |  |  | res=polymul(res,polyshift(x,_res)); _res+=_x; }
 |  |  | m >>= 1; x = polymul(x, polyshift(x, _x)); _x+=_x; }
 |  | ans = ans * res[0]; }
 | LL cnt = n / p; if (n >= p) { auto tmp=fact(n / p, p, q);
 |  | ans = ans * tmp.first; cnt += tmp.second; }
 | return {ans, cnt}; }
```

5.7 类欧几里得直线下格点统计

```
// ∑_{i=0}^{n-1} ⌊(a+bi)/m⌋, n,m,a,b > 0
```

```
LL solve(LL n, LL a, LL b, LL m){
 | if (b == 0) return n * (a / m);
 | if (a >= m) return n * (a / m) + solve(n, a % m, b, m);
 | if (b >= m) return (n-1)*n/2*(b/m) + solve(n,a,b%m,m);
 | return solve((a + b * n) / m, (a + b * n) % m, m, b); }
```

5.8 万能欧几里德

```
Val work(LL P, LL R, LL Q, LL n, Val VU, Val VR) {
//(Px+R)/Q, 1<=x<=i, 经过整点先U再R
 | if(!(((i128)n * P + R) / Q)) return ksm(VR, n);
 | if(P>=Q) return work(P%Q,R,Q,n, VU, ksm(VU, P/Q) * VR);
 | Val res; swap(VU,VR);
 | res = ksm(VU, (Q-R-1)/P)*VR;
 | LL m = ((i128)n * P + R) / Q;
 | res = res * work(Q, (Q-R-1)%P, P, m-1, VU, VR);
 | return res * ksm(VU, n - ((i128)m*Q - R - 1) / P); }
```

5.9 平方剩余

```
// x^2=a (mod p),0 <=a<p, 返回 true or false 代表是否存在解
// p必须是质数, 若是多个单次质数的乘积, 可以分别求解再用CRT合并
// 复杂度为 O(log n)
void multiply(ll &c, ll &d, ll a, ll b, ll w) {
 | int cc = (a * c + b * d % MOD * w) % MOD;
 | int dd = (a * d + b * c) % MOD; c = cc, d = dd; }
bool solve(int n, int &x) {
 | if (n==0) return x=0,true; if (MOD==2) return x=1,true;
 | if (power(n, MOD / 2, MOD) == MOD - 1) return false;
 | ll c = 1, d = 0, b = 1, a, w;
 | // finding a such that a^2 - n is not a square
 | do { a = rand() % MOD; w = (a * a - n + MOD) % MOD;
 |  | if (w == 0) return x = a, true;
 | } while (power(w, MOD / 2, MOD) != MOD - 1);
 | for (int times = (MOD + 1) / 2; times; times >>= 1) {
 |  | if (times & 1) multiply(c, d, a, b, w);
 |  | multiply(a, b, a, b, w); }
 | // x = (a + sqrt(w)) ^ ((p + 1) / 2)
 | return x = c, true; }
```

5.10 线性同余不等式

```
// Find the minimal non-negtive solutions for
   ↪ l ≤ d · x mod m ≤ r
// 0 ≤ d, l, r < m; l ≤ r, O(log n)
LL cal(LL m, LL d, LL l, LL r) {
 | if (l==0) return 0; if (d==0) return MXL; // 无解
 | if (d * 2 > m) return cal(m, m - d, m - r, m - l);
 | if ((l - 1) / d < r / d) return (l - 1) / d + 1;
 | LL k = cal(d, (-m % d + d) % d, l % d, r % d);
 | return k==MXL ? MXL : (k*m + l - 1)/d+1;}// 无解 2
// return all x satisfying l1<=x<=r1 and l2<=(x*mul+add) %LIM<=r2
// here LIM = 2^32 so we use UI instead of "%".
// O(log p + #solutions)
struct Jump { UI val, step;
 | Jump(UI val, UI step) : val(val), step(step) { }
 | Jump operator + (const Jump & b) const {
 |  | return Jump(val + b.val, step + b.step); }
 | Jump operator - (const Jump & b) const {
 |  | return Jump(val - b.val, step + b.step); }};
inline Jump operator * (UI x, const Jump & a) {
 | return Jump(x * a.val, x * a.step); }
vector<UI> solve(UI l1, UI r1, UI l2, UI r2, pair<UI,UI> muladd) {
 | UI mul = muladd.first, add = muladd.second, w = r2 - l2;
 | Jump up(mul, 1), dn(-mul, 1); UI s(l1 * mul + add);
 | Jump lo(r2 - s, 0), hi(s - l2, 0);
 | function<void(Jump&, Jump&)> sub=[&](Jump& a, Jump& b){
 |  | if (a.val > w) {
 |  |  | UI t(((LL)a.val-max(0LL, w+1LL-b.val)) / b.val);
 |  |  | a = a - t * b; } };
 | sub(lo, up), sub(hi, dn);
 | while (up.val > w || dn.val > w) {
 |  | sub(up, dn); sub(lo, up);
 |  | sub(dn, up); sub(hi, dn); }
 | assert(up.val + dn.val > w); vector<UI> res;
 | Jump bg(s + mul * min(lo.step, hi.step), min(lo.step, hi.step));
 | while (bg.step <= r1 - l1) {
 |  | if (l2 <= bg.val && bg.val <= r2)
 |  |  | res.push_back(bg.step + l1);
 |  | if (l2 <= bg.val-dn.val && bg.val-dn.val <= r2) {
```

```
| | | bg = bg - dn;
| | } else bg = bg + up; }
| return res; }
```

5.11 原根

定义 使得 $a^x \bmod m = 1$ 的最小的x, 记作 $\delta_m(a)$. 若 $a \equiv g^s \mod m$, 其中 g 为 m 的一个原根. 则虽然 s 随 g 的不同取值有所不同, 但是必然满足 $\delta_m(a) = \gcd(s, \varphi(m))$.

性质 $\delta_m(a^k) = \frac{\delta_m(a)}{\gcd(\delta_m(a),k)}$

k 次剩余 给定方程 $x^k \equiv a \mod m$, 求所有解. 若 k 与 $\varphi(m)$ 互质, 则可以直接求出 k 对 $\varphi(m)$ 的逆元. 否则, 将 k 拆成两部分, $k = uv$, 其中 $u \perp \varphi(m)$, $v | \varphi(m)$, 先求 $x^v \equiv a \mod m$, 则 $ans = x^{u^{-1}}$. 下面讨论 $k | \varphi(m)$ 的问题. 任取一原根 g, 对两侧取离散对数, 设 $x = g^s$, $a = g^t$, 其中 t 可以用BSGS求出, 则问题转化为求出所有的 s 满足 $ks \equiv t \mod \varphi(m)$, exgcd 即可求解, 显然有解的条件是 $k | \delta_m(a)$.

5.12 FFT

```
using cp = complex<double>; const double PI = acos(-1.0);
vector<cp> omega[25]; // 单位根
// n 是 DFT 的最大长度，例如如果最多有两个长为 m 的多项式相乘，
// 或者求逆的长度为 m，那么 n 需要 >= 2m
void fft_init(int n) { // n = 2^k
 | for (int k = 2, d = 0; k <= n; k *= 2, d++) {
 | | omega[d].resize(k + 1);
 | | for (int i = 0; i <= k; i++) // polar 是用模和辐角求复
 |   ↪ 数
 | | | omega[d][i] = polar(1.0, 2 * PI * i / k); } }
void fft(cp* a, int n, int t) {
 | for (int i = 1, j = 0; i < n - 1; i++) {
 | | int k = n; do j ^= (k >>= 1); while (j < k);
 | | if (i < j) swap(a[i], a[j]); }
 | for (int k = 1, d = 0; k < n; k *= 2, d++)
 | | for (int i = 0; i < n; i += k * 2)
 | | | for (int j = 0; j < k; j++) {
 | | | | cp w = omega[d][t > 0 ? j : k * 2 - j];
 | | | | cp u = a[i + j], v = w * a[i + j + k];
 | | | | a[i + j] = u + v; a[i + j + k] = u - v; }
 | if (t < 0) for (int i = 0; i < n; i++) a[i] /= n; }
```

5.12.1 个人代码

```
#define _USE_MATH_DEFINES
#include <bits/stdc++.h>
#include <cmath>

struct complex {
    double x, y;
    complex(double xx = 0, double yy = 0) {
        x = xx, y = yy;
    }
};
complex operator + (complex &a, complex &b) {
    return complex(a.x + b.x, a.y + b.y);
}
complex operator - (complex &a, complex &b) {
    return complex(a.x - b.x, a.y - b.y);
}
complex operator * (complex &a, complex &b) {
    return complex(a.x * b.x - a.y * b.y, a.x * b.y + a.y *
      ↪ b.x);
}
complex a[1 << 21], b[1 << 21];
int limit = 1, n, m;
int to[1 << 21];

void FFT(complex *a, int type) {
    for (int i = 0; i < limit; ++i) {
        if (to[i] > i) {
            std::swap(a[i], a[to[i]]);
        }
    }
    for (int len = 2; len <= limit; len <<= 1) {
        complex wn = complex(cos(2.0 * M_PI / len),
                             type * sin(2.0 * M_PI / len));
        for (int st = 0; st < limit; st += len) {
            complex w = complex(1, 0);
            for (int j = 0; j < len / 2; ++j) {
                complex X = a[st + j], Y = w * a[st + j +
                  ↪ len / 2];
                a[st + j] = X + Y;
                a[st + j + len / 2] = X - Y;
                w = w * wn;
            }
        }
    }
}

int main() {
    while (limit <= n + m) limit <<= 1;
    for (int i = n + 1; i < limit; ++i) a[i] = 0;
    for (int i = m + 1; i < limit; ++i) b[i] = 0;
    int t = std::__lg(limit);
    for (int i = 1; i < limit; ++i) {
        to[i] = (to[i >> 1] >> 1) | ((i & 1) << (t - 1));
    }
    FFT(a, 1);
    FFT(b, 1);
    for (int i = 0; i < limit; ++i) {
        a[i] = a[i] * b[i];
    }
    FFT(a, -1);
    return 0;
}
```

5.13 NTT

```
vector<int> omega[25]; // 单位根
// n 是 DFT 的最大长度，例如如果最多有两个长为 m 的多项式相乘，
// 或者求逆的长度为 m，那么 n 需要 >= 2m
void ntt_init(int n) { // n = 2^k
 | for (int k = 2, d = 0; k <= n; k *= 2, d++) {
 | | omega[d].resize(k + 1);
 | | int wn = qpow(3, (p - 1) / k), tmp = 1;
 | | for (int i = 0; i <= k; i++) { omega[d][i] = tmp;
 | | | tmp = (LL)tmp * wn % p; } } }
// 传入的数必须是 [0, p) 范围内，不能有负的
// 否则把 d == 16 改成 d % 8 == 0 之类，多取几次模
void ntt(int *c, int n, int tp) {
 | static ULL a[N];
 | for (int i = 0; i < n; i++) a[i] = c[i];
 | for (int i = 1, j = 0; i < n - 1; i++) {
 | | int k = n; do j ^= (k >>= 1); while (j < k);
 | | if (i < j) swap(a[i], a[j]); }
 | for (int k = 1, d = 0; k < n; k *= 2, d++) {
 | | if (d == 16) for (int i = 0; i < n; i++) a[i] %= p;
 | | for (int i = 0; i < n; i += k * 2)
 | | | for (int j = 0; j < k; j++) {
 | | | | int w = omega[d][tp > 0 ? j : k * 2 - j];
 | | | | ULL u = a[i + j], v = w * a[i + j + k] % p;
 | | | | a[i + j] = u + v;
 | | | | a[i + j + k] = u - v + p; } }
 | if (tp>0) {for (int i = 0; i < n; i++) c[i] = a[i] % p;}
 | else { int inv = qpow(n, p - 2);
 | | for (int i = 0; i < n; i++) c[i] = a[i] * inv % p;}}
```

5.13.1 个人代码

```
const int mod = 998244353, g = 3, gr = 332748118;
int qmi(int a, int b, int p) {
    int res = 1;
    while (b) {
        if (b & 1) res = (long long)res * a % p;
        b >>= 1;
        a = (long long)a * a % p;
    }
    return res;
}
int a[1 << 21], b[1 << 21], to[1 << 21];
int n, m;

void NTT(int *a, int limit, int type) {
    for (int i = 0; i < limit; ++i) {
        if (i < to[i]) {
            std::swap(a[i], a[to[i]]);
        }
    }
    for (int len = 2; len <= limit; len <<= 1) {
        int gn = qmi(type == 1 ? g : gr, (mod - 1) / len,
          ↪ mod);
        for (int st = 0; st < limit; st += len) {
            for (int j = 0, now = 1; j < len / 2;
                 ++j, now = (ll)now * gn % mod) {
                int A = a[st + j];
                int B = (ll)now * a[st + j + len / 2] % mod;
                a[st + j] = (A + B) % mod;
                a[st + j + len / 2] = (A - B + mod) % mod;
```

```
            }
        }
    }
}

void work(int *a, int na, int *b, int nb) {
    // 从低次到高次，na、nb 是最高次数。
    int limit = 1;
    while (limit <= na + nb) limit <<= 1;
    for (int i = na + 1; i < limit; ++i) a[i] = 0;
    for (int i = nb + 1; i < limit; ++i) b[i] = 0;
    int bit = std::__lg(limit);
    for (int i = 1; i < limit; ++i)
        to[i] = (to[i >> 1] >> 1) | ((i & 1) << (bit - 1));
    NTT(a, limit, 1), NTT(b, limit, 1);
    for (int i = 0; i < limit; ++i)
        a[i] = 1ll * a[i] * b[i] % mod;
    NTT(a, limit, -1);
    int rev = qmi(limit, mod - 2, mod);
    for (int i = 0; i <= na + nb; ++i)
        a[i] = 1ll * a[i] * rev % mod;
}
```

5.14 MTT 任意模数卷积

```
void dft(cp* a, cp* b, int n) { static cp c[MAXN];
 | for (int i = 0; i < n; i++)
 |  | c[i] = cp(a[i].real(), b[i].real());
 | fft(c, n, 1);
 | for (int i = 0; i < n; i++) { int j = (n - i) & (n - 1);
 |  | a[i] = (c[i] + conj(c[j])) * 0.5;
 |  | b[i] = (c[i] - conj(c[j])) * -0.5i; } }
void idft(cp* a, cp* b, int n) { static cp c[MAXN];
 | for (int i = 0; i < n; i++) c[i] = a[i] + 1i * b[i];
 | fft(c, n, -1);
 | for (int i = 0; i < n; i++) {
 |  | a[i] = c[i].real(); b[i] = c[i].imag(); } }
vector<int> multiply(const vector<int>& u,
 |  | const vector<int>& v, int mod) { // 任意模数卷积
 | static cp a[2][MAXN], b[2][MAXN], c[3][MAXN];
 | int base = ceil(sqrt(mod));
 | int n = (int)u.size(), m = (int)v.size();
 | int fft_n = 1; while (fft_n < n + m - 1) fft_n *= 2;
 | for (int i = 0; i < 2; i++) {
 |  | fill(a[i], a[i] + fft_n, 0);
 |  | fill(b[i], b[i] + fft_n, 0); }
 | for (int i = 0; i < 3; i++)
 |  | fill(c[i], c[i] + fft_n, 0);
 | for (int i = 0; i < n; i++) { // 一定要取模！
 |  | a[0][i] = (u[i] % mod) % base;
 |  | a[1][i] = (u[i] % mod) / base; }
 | for (int i = 0; i < m; i++) { // 一定要取模！
 |  | b[0][i] = (v[i] % mod) % base;
 |  | b[1][i] = (v[i] % mod) / base; }
 | dft(a[0], a[1], fft_n); dft(b[0], b[1], fft_n);
 | for (int i = 0; i < fft_n; i++) {
 |  | c[0][i] = a[0][i] * b[0][i];
 |  | c[1][i] = a[0][i] * b[1][i] + a[1][i] * b[0][i];
 |  | c[2][i] = a[1][i] * b[1][i]; }
 | fft(c[1], fft_n, -1); idft(c[0], c[2], fft_n);
 | int base2 = base * base % mod;
 | vector<int> ans(n + m - 1);
 | for (int i = 0; i < n + m - 1; i++)
 |  | ans[i] = ((LL)(c[0][i].real() + 0.5) +
 |  |  | (LL)(c[1][i].real() + 0.5) % mod * base +
 |  |  | (LL)(c[2][i].real() + 0.5) % mod * base2) % mod;
 | return ans; }
```

5.15 多项式运算

5.15.1 多项式求逆 开根 ln exp

```
using poly = vector<int>; // 用到 poly 的部分补成 2 ^ k
poly poly_calc(const poly& u, const poly& v, // 长度要相同
 | function<int(int, int)> op) { // 返回长度是两倍
 | static int a[MAXN], b[MAXN], c[MAXN];
 | int n = (int)u.size();
 | memcpy(a, u.data(), sizeof(int) * n);
 | fill(a + n, a + n * 2, 0);
 | memcpy(b, v.data(), sizeof(int) * n);
 | fill(b + n, b + n * 2, 0);
 | ntt(a, n * 2, 1); ntt(b, n * 2, 1);
 | for (int i = 0; i < n * 2; i++) c[i] = op(a[i], b[i]);
 | ntt(c, n * 2, -1); return poly(c, c + n * 2); }
poly poly_mul(const poly& u, const poly& v) { // 乘法
 | return poly_calc(u, v, [](int a, int b)
 |  | { return (LL)a * b % p; }); } // 返回长度是两倍
poly poly_inv(const poly& a) { // 求逆，返回长度不变
 | poly c{qpow(a[0], p - 2)}; // 常数项一般都是 1
 | for (int k = 2; k <= (int)a.size(); k *= 2) {
 |  | c.resize(k); poly b(a.begin(), a.begin() + k);
 |  | c = poly_calc(b, c, [](int bi, int ci) {
 |  |  | return ((2 - (LL)bi * ci) % p + p) * ci % p; });
 |  | memset(c.data() + k, 0, sizeof(int) * k); }
 | c.resize(a.size()); return c; }
poly poly_sqrt(const poly& a) { // 开根，返回长度不变
 | poly c{1}; // 常数项不是 1 的话要写二次剩余
 | for (int k = 2; k <= (int)a.size(); k *= 2) {
 |  | c.resize(k); poly b(a.begin(), a.begin() + k);
 |  | b = poly_mul(b, poly_inv(c));
 |  | for (int i = 0; i < k; i++) // inv_2 是 2 的逆元
 |  |  | c[i] = (LL)(c[i] + b[i]) * inv_2 % p; }
 | c.resize(a.size()); return c; }
poly poly_derivative(const poly& a) { poly c(a.size());
 | for (int i = 1; i < (int)a.size(); i++) // 求导
 |  | c[i - 1] = (LL)a[i] * i % p; return c; }
poly poly_integrate(const poly& a) { poly c(a.size());
 | for (int i = 1; i < (int)a.size(); i++) // 不定积分
 |  | c[i] = (LL)a[i - 1] * inv[i] % p; return c; }
poly poly_ln(const poly& a) { // ln，常数项非0，返回长度不变
 | auto c = poly_mul(poly_derivative(a), poly_inv(a));
 | c.resize(a.size()); return poly_integrate(c); }
// exp，常数项必须是 0，返回长度不变
// 常数很大并且总代码很长，一般可以改用分治 FFT
// 依据：设 G(x) = exp F(x)，则 g_i = (1/i) * sum_{k=1}^{i-1} g_{i-k} k f_k
poly poly_exp(const poly& a) { poly c{1};
 | for (int k = 2; k <= (int)a.size(); k *= 2) {
 |  | c.resize(k); auto b = poly_ln(c);
 |  | for (int i = 0; i < k; i++)
 |  |  | b[i] = (a[i] - b[i] + p) % p;
 |  | (++b[0]) %= p; c = poly_mul(b, c);
 |  | memset(c.data() + k, 0, sizeof(int) * k); }
 | c.resize(a.size()); return c; }
```

5.15.2 多项式除法 取模

需要抄求逆。

```
poly poly_auto_mul(poly a, poly b) { // 自动判断长度的乘法
 | int res_len = (int)a.size() + (int)b.size() - 1;
 | int ntt_n = 1; while (ntt_n < res_len) ntt_n *= 2;
 | a.resize(ntt_n); b.resize(ntt_n);
 | ntt(a.data(), ntt_n, 1); ntt(b.data(), ntt_n, 1);
 | for (int i = 0; i < ntt_n; i++)
 |  | a[i] = (LL)a[i] * b[i] % p;
 | ntt(a.data(), ntt_n, -1); a.resize(res_len); return a; }
// 多项式除法，a 和 b 长度可以任意
// 商的长度是 n - m + 1，余数的长度是 m - 1
poly poly_div(const poly& a, const poly& b) {
 | int n = (int)a.size(), m = (int)b.size();
 | if (n < m) return {};
 | int ntt_n = 1; while (ntt_n < n - m + 1) ntt_n *= 2;
 | poly f(ntt_n), g(ntt_n);
 | for (int i = 0; i < n - m + 1; i++) f[i] = a[n - i - 1];
 | for (int i = 0; i < m && i < n - m + 1; i++)
 |  | g[i] = b[m - i - 1];
 | auto g_inv = poly_inv(g);
 | fill(g_inv.begin() + n - m + 1, g_inv.end(), 0);
 | auto c = poly_mul(f, g_inv); c.resize(n - m + 1);
 | reverse(c.begin(), c.end()); return c; }
// 多项式取模，a 和 b 长度可以任意，返回 (余数, 商)
pair<poly, poly> poly_mod(const poly& a, const poly& b) {
 | int n = (int)a.size(), m = (int)b.size();
 | if (n < m) return {a, {}};
 | auto d = poly_div(a, b); auto c = poly_auto_mul(b, d);
 | poly r(m - 1);
 | for (int i = 0; i < m - 1; i++)
 |  | r[i] = (a[i] - c[i] + p) % p;
 | return {r, d}; }
```

5.15.3 多点求值

需要抄取模。

```
struct poly_eval { poly f; vector<int> x; // 函数和询问点
 | vector<poly> gs; vector<int> ans; // gs 是预处理数组
 | poly_eval(poly f, vector<int> x) : f(f), x(x) {}
 | void pretreat(int l, int r, int o) { poly& g = gs[o];
 |  | if (l == r) { g = poly{p - x[l], 1}; return; }
 |  | int mid = (l + r) / 2; pretreat(l, mid, o * 2);
```

```
  |   | pretreat(mid + 1, r, o * 2 + 1);
  |   | if (o > 1)
  |   |   | g = poly_auto_mul(gs[o * 2], gs[o * 2 + 1]); }
  | void solve(int l, int r, int o, const poly& f) {
  |   | if (l == r) { ans[l] = f[0]; return; }
  |   | int mid = (l + r) / 2;
  |   | solve(l, mid, o * 2, poly_mod(f, gs[o * 2]).first);
  |   | solve(mid + 1, r, o * 2 + 1,
  |   |   | poly_mod(f, gs[o * 2 + 1]).first); }
  | vector<int> operator() () { // 包装好的接口
  |   | int n = (int)f.size(), m = (int)x.size();
  |   | if (m <= n) x.resize(m = n + 1);
  |   | else if (n < m - 1) f.resize(n = m - 1);
  |   | int bit_ceil = 1; while (bit_ceil < m) bit_ceil *= 2;
  |   | ntt_init(bit_ceil * 2); // 注意这里 ntt_init 过了
  |   | gs.resize(2 * bit_ceil + 1); pretreat(0, m - 1, 1);
  |   | ans.resize(m); solve(0, m - 1, 1, f); return ans;} };
```

5.15.4 插值

牛顿插值 实现时可以用 k 次差分替代右边的式子，也可以卷积。

$$f(n)=\sum_{i=0}^{k}\binom{n}{i}r_i \iff r_i=\sum_{j=0}^{i}(-1)^{i-j}\binom{i}{j}f(j)$$

拉格朗日插值

$$f(x)=\sum_i f(x_i)\prod_{j\neq i}\frac{x-x_j}{x_i-x_j}$$

5.16 线性递推

$O(k^2\log n)$

```
// Complexity: init O(n^2log) query O(n^2logk)
// Requirement: const LOG const MOD
// Example: In: {1, 3} {2, 1} an = 2an-1 + an-2
//          Out: calc(3) = 7
typedef vector<int> poly;
struct LinearRec {
  | int n; poly first, trans; vector<poly> bin;
poly add(poly &a, poly &b) {
  | poly res(n * 2 + 1, 0);
  | // 不要每次新开 vector，可以使用矩阵乘法优化
  | for (int i = 0; i <= n; ++i) {
  |   | for (int j = 0; j <= n; ++j) {
  |   |   | (res[i+j]+=(LL)a[i] * b[j] % MOD) %= MOD;
  | for (int i = 2 * n; i > n; --i) {
  |   | for (int j = 0; j < n; ++j) {
  |   |   | (res[i-1-j]+=(LL)res[i]*trans[j]%MOD) %=MOD;}
  |   | res[i] = 0; }
  | res.erase(res.begin() + n + 1, res.end());
  | return res; }
LinearRec(poly &first, poly &trans): first(first),
  ↪ trans(trans) {
  | n = first.size(); poly a(n + 1, 0); a[1] = 1;
  | bin.push_back(a); for (int i = 1; i < LOG; ++i)
  |   | bin.push_back(add(bin[i - 1], bin[i - 1])); }
int calc(int k) { poly a(n + 1, 0); a[0] = 1;
  | for (int i = 0; i < LOG; ++i)
  |   | if (k >> i & 1) a = add(a, bin[i]);
  | int ret = 0; for (int i = 0; i < n; ++i)
  |   | if ((ret += (LL)a[i + 1] * first[i] % MOD) >= MOD)
  |   |   | ret -= MOD;
  | return ret; }};
```

$O(k\log k\log n)$ - **Bostan-Mori**

```
int bostan_mori(int k, poly a, poly b) {
  | int n = (int)a.size(); while (k) { poly c = b;
  |   | for (int i = 1; i < n; i += 2) c[i] = (p - c[i]) % p;
  |   | a = poly_mul(a, c); b = poly_mul(b, c);
  |   | for (int i = 0; i < n; i++) {
  |   |   | a[i] = a[i * 2 + k % 2]; b[i] = b[i * 2]; }
  |   | a.resize(n); b.resize(n); k /= 2; }
  | return (LL)a[0] * qpow(b[0], p - 2) % p; }
// a_n = sum_{i=1}^{m} f_i a_{n-i}   (f_0 = 0), f.size() = a.size() + 1
int linear_recurrance(int n, poly f, poly a) {
  | int m = (int)a.size(), ntt_n = 1;
  | while (ntt_n <= m) ntt_n *= 2; ntt_init(ntt_n * 2);
  | f.resize(ntt_n); a.resize(ntt_n); f[0] = 1;
  | for (int i = 1; i <= m; i++) f[i] = (p - f[i]) % p;
  | a = poly_mul(a, f); a.resize(ntt_n);
  | fill(a.data() + m, a.data() + ntt_n, 0);
  | return bostan_mori(n, a, f); }
```

$O(k\log k\log n)$ - **多项式取模**

需要抄前面的多项式取模。预处理 $O(k\log k\log n)$，固定 n 和系数只改变初始值的话，询问一次 $O(k)$。注意只询问一次的话不如 Bostan-Mori 快。

```
poly poly_power_mod(LL k, const poly& m) { // x^k mod m
  | poly ans{1}, a{0, 1}; while (k) { if (k & 1)
  |   |   | ans = poly_mod(poly_auto_mul(ans, a), m).first;
  |   | a = poly_mod(poly_auto_mul(a, a), m).first; k /= 2; }
  | return ans; }
// a_n = sum_{i=1}^{m} c_i a_{n-i}   (c_0 = 0)
struct linear_recurrence { poly f; // f是预处理结果
  | linear_recurrence(const poly& c, LL n) {
  |   | assert(c[0] == 0); // c[0] 是没有用的
  |   | int m = (int)c.size() - 1;
  |   | int ntt_n = 1; while (ntt_n < m * 2) ntt_n *= 2;
  |   | ntt_init(ntt_n); // 图省事就直接 ntt_init(1 << 18)
  |   | poly t(m + 1); t[m] = 1;
  |   | for (int i = 0; i < m; i++)t[i] = (p - c[m - i]) % p;
  |   | f = poly_power_mod(n, t); }
  | int operator()(const vector<int>& a) { // 0~m-1项初始值
  |   | assert(a.size() == f.size()); int ans = 0;
  |   | for (int i = 0; i < (int)a.size(); i++)
  |   |   | ans = (ans + (LL)f[i] * a[i]) % p;
  |   | return ans; } };
```

5.17 Berlekamp-Massey 最小多项式

如果要求出一个次数为 k 的递推式，则输入的数列需要至少有 $2k$ 项。返回的内容满足 $\sum_{j=0}^{m-1}a_{i-j}c_j=0$，并且 $c_0=1$。

如果不加最后的处理的话，代码返回的结果会变成 $a_i=\sum_{j=0}^{m-1}c_{j-1}a_{i-j}$，有时候这样会方便接着跑递推，需要的话就删掉最后的处理。

```
vector<int> berlekamp_massey(const vector<int> &a) {
  | vector<int> v, last; // v is the answer, 0-based
  | int k = -1, delta = 0;
  | for (int i = 0; i < (int)a.size(); i++) { int tmp = 0;
  |   | for (int j = 0; j < (int)v.size(); j++)
  |   |   | tmp = (tmp + (LL)a[i - j - 1] * v[j]) % p;
  |   | if (a[i] == tmp) continue;
  |   | if (k < 0) { k = i; delta = (a[i] - tmp + p) % p;
  |   |   | v = vector<int>(i + 1); continue; }
  |   | vector<int> u = v;
  |   | int val = (LL)(a[i] - tmp + p) *
  |   |   | qpow(delta, p - 2) % p;
  |   | if (v.size() < last.size() + i - k)
  |   |   | v.resize(last.size() + i - k);
  |   | (v[i - k - 1] += val) %= p;
  |   | for (int j = 0; j < (int)last.size(); j++) {
  |   |   | v[i - k + j] = (v[i - k + j] -
  |   |   |   | (LL)val * last[j]) % p;
  |   |   | if (v[i - k + j] < 0) v[i - k + j] += p; }
  |   | if ((int)u.size() - i < (int)last.size() - k) {
  |   |   | last = u; k = i; delta = a[i] - tmp;
  |   |   | if (delta < 0) delta += p; } }
  | for (auto &x : v) x = (p - x) % p;
  | v.insert(v.begin(), 1); //一般是需要最小递推式的，处理一下
  | return v; } // ∀i, sum_{j=0}^{m} a_{i-j} v_j = 0
```

如果要求向量序列的递推式，就把每位乘一个随机权值 (或者说是乘一个随机行向量 v^T) 变成求数列递推式即可。如果是矩阵序列的话就随机一个行向量 u^T 和列向量 v，然后把矩阵变成 u^TAv 的数列。

优化矩阵快速幂DP 假设 f_i 有 n 维，先暴力求出 $f_{0\sim 2n-1}$，然后跑 Berlekamp-Massey，最后调用快速线性递推即可。

求矩阵最小多项式 矩阵 A 的最小多项式是次数最小的并且 $f(A)=0$ 的多项式 f。实际上最小多项式就是 $\{A^i\}$ 的最小递推式，所以直接调用 Berlekamp-Massey 就好了，显然它的次数不超过 n。

瓶颈在于求出 A^i，实际上我们只要处理 A^iv 就行了，每次对向量做递推。

求稀疏矩阵的行列式 如果能求出特征多项式，则常数项乘上 $(-1)^n$ 就是行列式，但是最小多项式不一定就是特征多项式。

把 A 乘上一个随机对角阵 B，则 AB 的最小多项式有很大概率就是特征多项式，最后再除掉 $\det B$ 就行了。

求稀疏矩阵的秩 设 A 是一个 $n\times m$ 的矩阵，首先随机一个 $n\times n$ 的对角阵 P 和一个 $m\times m$ 的对角阵 Q，然后计算 $QAPA^TQ$ 的最小多项式即可。

实际上不用计算这个矩阵，因为求最小多项式时要用它乘一个向量，我们依次把这几个矩阵乘到向量里就行了。答案就是最小多项式除掉所有 x 因子后剩下的次数。

解稀疏方程组 $Ax=b$，其中 A 是一个 $n\times n$ 的**满秩**稀疏矩阵，b 和 x 是 $1\times n$ 的**列**向量，A,b 已知，需要解出 x。

做法：显然 $x=A^{-1}b$. 如果我们能求出 $\{A^ib\}(i\geq 0)$ 的最小递推式 $\{r_{0\ldots m-1}\}(m\leq n)$，那么就有结论

$$A^{-1}b = -\frac{1}{r_{m-1}} \sum_{i=0}^{m-2} A^i b r_{m-2-i}$$

因为 A 是稀疏矩阵，直接按定义递推出 $b \dots A^{2n-1}b$ 即可。

```
vector<int> solve_sparse_equations(const vector<tuple<int,
  ↪int, int> > &A, const vector<int> &b) {
 | int n = (int)b.size(); // 0-based
 | vector<vector<int> > f({b});
 | for (int i = 1; i < 2 * n; i++) {
 |  | vector<int> v(n); auto &u = f.back();
 |  | for (auto [x, y, z] : A) // [x, y, value]
 |  |  | v[x] = (v[x] + (long long)u[y] * z) % p;
 |  | f.push_back(v); }
 | vector<int> w(n); mt19937 gen;
 | for (auto &x : w)
 |  | x = uniform_int_distribution<int>(1, p - 1)(gen);
 | vector<int> a(2 * n);
 | for (int i = 0; i < 2 * n; i++)
 |  | for (int j = 0; j < n; j++)
 |  |  | a[i] = (a[i] + (long long)f[i][j] * w[j]) % p;
 | auto c = berlekamp_massey(a); int m = (int)c.size();
 | vector<int> ans(n);
 | for (int i = 0; i < m - 1; i++)
 |  | for (int j = 0; j < n; j++)
 |  |  | ans[j] = (ans[j] +
 |  |  |  | (long long)c[m - 2 - i] * f[i][j]) % p;
 | int inv = qpow(p - c[m - 1], p - 2);
 | for (int i = 0; i < n; i++)
 |  | ans[i] = (long long)ans[i] * inv % p;
 | return ans; }
```

5.18 FWT

```
/*
And: (1,1 0,1) (1,−1 0,1) Or: (1,0 1,1) (1,0 −1,1) Xor: (1,1 1,−1) (0.5,0.5 0.5,−0.5)
IFWT的矩阵时FWT的逆，对于任意运算 ⊕，满足FWT的矩阵需要：
C[i][j] × C[i][k] = C[i][j ⊕ k]
对于不存在FWT矩阵的运算：通过映射01变成另外一个可行的运算。*/
const LL XOR[2][2] = {{1, 1}, {1, M-1}};
const LL i2 = (M+1)/2, iXOR[2][2] = {{i2, i2}, {i2, M-i2}};
void FWT(LL f[], const LL C[2][2], int n) {
  for (int t = 1; t < n; t <<= 1) {
    for (int l = 0; l < n; l += t + t) {
      for (int i = 0; i < t; i++) {
        LL x = f[l + i], y = f[l + t + i];
        f[l + i]     = (C[0][0] * x + C[0][1] * y) % M;
        f[l + t + i] = (C[1][0] * x + C[1][1] * y) % M;
      } } } }
```

5.19 K 进制 FWT

```
// n : power of k, omega[i] : (primitive kth root) ^ i
void fwt(int* a, int k, int type) {
  static int tmp[K];
  for (int i = 1; i < n; i *= k)
    for (int j = 0, len = i * k; j < n; j += len)
      for (int low = 0; low < i; low++) {
        for (int t = 0; t < k; t++)
          tmp[t] = a[j + t * i + low];
        for (int t = 0; t < k; t++){
          int x = j + t * i + low;
          a[x] = 0;
          for (int y = 0; y < k; y++)
            a[x] = int(a[x] + 1ll * tmp[y] * omega[(k +
              ↪type) * t * y % k] % MOD);
        }
      }
  if (type == -1)
    for (int i = 0, invn = inv(n); i < n; i++)
      a[i] = int(1ll * a[i] * invn % MOD); }
```

5.20 Simplex 单纯形

```
const LD eps = 1e-9, INF = 1e9; const int N = 105;
namespace Simplex {
int n, m, id[N], tp[N]; LD a[N][N];
void pivot(int r, int c) {
 | swap(id[r + n], id[c]);
 | LD t = -a[r][c]; a[r][c] = -1;
 | for (int i = 0; i <= n; i++) a[r][i] /= t;
 | for (int i = 0; i <= m; i++) if (a[i][c] && r != i) {
 |  | t = a[i][c]; a[i][c] = 0;
 |  | for (int j = 0; j <= n; j++) a[i][j] += t*a[r][j];}}
bool solve() {
 | for (int i = 1; i <= n; i++) id[i] = i;
 | for ( ; ; ) {
 |  | int i = 0, j = 0; LD w = -eps;
 |  | for (int k = 1; k <= m; k++)
 |  |  | if (a[k][0] < w || (a[k][0] < -eps && rand() & 1))
 |  |  |  | w = a[i = k][0];
 |  | if (!i) break;
 |  | for (int k = 1; k <= n; k++)
 |  |  | if (a[i][k] > eps) {j = k; break;}
 |  | if (!j) { printf("Infeasible"); return 0;}
 |  | pivot(i, j);}
 | for ( ; ; ) {
 |  | int i = 0, j = 0; LD w = eps, t;
 |  | for (int k = 1; k <= n; k++)
 |  |  | if (a[0][k] > w) w = a[0][j = k];
 |  | if (!j) break;
 |  | w = INF;
 |  | for (int k = 1; k <= m; k++)
 |  |  | if (a[k][j] < -eps && (t = -a[k][0]/a[k][j]) < w)
 |  |  |  | w = t, i = k;
 |  | if (!i) { printf("Unbounded"); return 0;}
 |  | pivot(i, j);}
 | return 1;}
LD ans() {return a[0][0];}
void output() {
 | for (int i = n + 1; i <= n + m; i++) tp[id[i]] = i - n;
 | for (int i = 1; i <=n; i++) printf("%.9lf ", tp[i] ?
   ↪a[tp[i]][0] : 0);}
}using namespace Simplex;
int main() { int K; read(n); read(m); read(K);
for (int i = 1; i <= n; i++) {LD x; scanf("%lf", &x);
  ↪a[0][i] = x;}
for (int i = 1; i <= m; i++) {LD x;
 | for (int j = 1; j <= n; j++) scanf("%lf", &x), a[i][j] =
   ↪-x;
 | scanf("%lf", &x); a[i][0] = x;}
if (solve()) { printf("%.9lf\n", (LD)ans()); if (K)
  ↪output();}}
// 标准型: maximize c^T x, subject to Ax ≤ b and x ≥ 0
// 对偶型: minimize b^T y, subject to A^T x ≥ c and y ≥ 0
```

5.21 高斯消元最小范数解

```
typedef vector <LD> vec; /* sum a[i][0..d] = 0 */
pair<vec,vector<vec>> gauss(vector<vec> &a, int n, int d) {
 | vector <int> pivot(d, -1);
 | for (int i = 0, o = 0; i < d; i++) {
 |  | int j = o; while (j < n && abs(a[j][i]) < eps) j++;
 |  | if (j == n) continue;
 |  | swap(a[j], a[o]); LD w = a[o][i];
 |  | for (int k = 0; k <= d; k++) a[o][k] /= w;
 |  | for (int x = 0; x < n; x++)
 |  |  | if (x != o && abs(a[x][i]) > eps) {
 |  |  |  | w = a[x][i];
 |  |  |  | for (int k = 0; k <= d; k++)
 |  |  |  |  | a[x][k] -= a[o][k] * w;   }
 |  | pivot[i] = o++;
 | } vec x0(d); vector <vec> t; int free = 0;
 | for (int i = 0; i < d; i++)
 |  | if (pivot[i] != -1) x0[i] = -a[pivot[i]][d];
 |  | else free ++;
 | for (int i = 0; i < d; i++) if (pivot[i] == -1) {
 |  | vec x(d); x[i] = -1;
 |  | for (int j = 0; j < d; j++)
 |  |  | if (pivot[j] != -1) x[j] = a[pivot[j]][i];
 |  | t.push_back(x);
 | } if (t.size()) {
 |  | vector <vec> f;
 |  | for (int u = 0; u < free; u++) {
 |  |  | vec x(free + 1);
 |  |  | for (int i = 0; i < free; i++)
 |  |  |  | for (int j = 0; j < d; j++)
 |  |  |  |  | x[i] += t[u][j] * t[i][j];
 |  |  | for (int j = 0; j < d; j++)
 |  |  |  | x[free] += t[u][j] * x0[j];
 |  |  | f.push_back(x);
 |  | }
 |  | auto [k, tt] = gauss(f, free, free);
 |  | assert (tt.size() == 0);
 |  | for (int x = 0; x < free; x++)
 |  |  | for (int i = 0; i < d; i++)
```

```
| | | | x0[i] += k[x] * t[x][i];
| } return {x0, t}; }
```

5.22 Pell 方程

```
// x^2 - n * y^2 = 1 最小正整数根，n 为完全平方数时无解
// x_{k+1} = x_0 x_k + n y_0 y_k
// y_{k+1} = x_0 y_k + y_0 x_k
pair<LL, LL> pell(LL n) {
| static LL p[N], q[N], g[N], h[N], a[N];
| p[1] = q[0] = h[1] = 1; p[0] = q[1] = g[1] = 0;
| a[2] = (LL)(floor(sqrtl(n) + 1e-7L));
| for(int i = 2; ; i ++) {
| | g[i] = -g[i - 1] + a[i] * h[i - 1];
| | h[i] = (n - g[i] * g[i]) / h[i - 1];
| | a[i + 1] = (g[i] + a[2]) / h[i];
| | p[i] = a[i] * p[i - 1] + p[i - 2];
| | q[i] = a[i] * q[i - 1] + q[i - 2];
| | if(p[i] * p[i] - n * q[i] * q[i] == 1)
| | | return {p[i], q[i]}; }}
```

5.23 解一元三次方程

```
double a(p[3]), b(p[2]), c(p[1]), d(p[0]);
double k(b / a), m(c / a), n(d / a);
double p(-k * k / 3. + m);
double q(2. * k * k * k / 27 - k * m / 3. + n);
Complex omega[3] = {Complex(1, 0), Complex(-0.5, 0.5 *
  ↪ sqrt(3)), Complex(-0.5, -0.5 * sqrt(3))};
Complex r1, r2; double delta(q * q / 4 + p * p * p / 27);
if (delta > 0) {
| r1 = cubrt(-q / 2. + sqrt(delta));
| r2 = cubrt(-q / 2. - sqrt(delta));
} else {
| r1 = pow(-q / 2. + pow(Complex(delta), 0.5), 1. / 3);
| r2 = pow(-q / 2. - pow(Complex(delta), 0.5), 1. / 3); }
for(int _(0); _ < 3; _++) {
| Complex x = -k/3. + r1*omega[_] + r2*omega[_* 2 % 3]; }
```

5.24 自适应 Simpson

```
// Adaptive Simpson's method : LD simpson::solve (LD (*f)
  ↪ (LD), LD l, LD r, LD eps) : integrates f over (l, r)
  ↪ with error eps.
struct simpson {
LD area (LD (*f) (LD), LD l, LD r) {
| LD m = l + (r - l) / 2;
| return (f (l) + 4 * f (m) + f (r)) * (r - l) / 6;
}
LD solve (LD (*f) (LD), LD l, LD r, LD eps, LD a) {
| LD m = l + (r - l) / 2;
| LD left = area (f, l, m), right = area (f, m, r);
| if (abs (left + right - a) <= 15 * eps) // TLE: || eps <
    ↪ EPS ** 2
| | return left + right + (left + right - a) / 15.0;
| return solve (f, l, m, eps / 2, left) + solve (f, m, r,
    ↪ eps / 2, right);
}
LD solve (LD (*f) (LD), LD l, LD r, LD eps) {
| return solve (f, l, r, eps, area (f, l, r));
}};
```

6. Appendix

6.1 Formulas 公式表

6.1.1 Mobius Inversion

$$F(n)=\sum_{d|n}f(d)\Rightarrow f(n)=\sum_{d|n}\mu(d)F\left(\frac{n}{d}\right)$$

$$F(n)=\sum_{n|d}f(d)\Rightarrow f(n)=\sum_{n|d}\mu\left(\frac{d}{n}\right)F(d)$$

$$[x=1]=\sum_{d|x}\mu(d),\ \ x=\sum_{d|x}\mu(d)$$

6.1.2 杜教筛

$$S_\varphi(n)=\frac{n(n+1)}{2}-\sum_{d=2}^{n}S_\varphi\left(\left\lfloor\frac{n}{d}\right\rfloor\right)$$

$$S_\mu(n)=1-\sum_{d=2}^{n}S_\mu\left(\left\lfloor\frac{n}{d}\right\rfloor\right)$$

6.1.3 降幂公式

$$a^k\equiv a^{k\bmod\varphi(p)+\varphi(p)},\ k\geq\varphi(p)$$

6.1.4 其他常用公式

$$\sum_{i=1}^{n}[(i,n)=1]\,i=n\frac{\varphi(n)+e(n)}{2}$$

$$\sum_{i=1}^{n}\sum_{j=1}^{i}[(i,j)=d]=S_\varphi\left(\left\lfloor\frac{n}{d}\right\rfloor\right)$$

$$\sum_{i=1}^{n}\sum_{j=1}^{m}[(i,j)=d]=\sum_{d|k}\mu\left(\frac{k}{d}\right)\left\lfloor\frac{n}{k}\right\rfloor\left\lfloor\frac{m}{k}\right\rfloor$$

另，如果要求的是 $[n\%k=t]$，其实就是 $[k\mid(n-t)]$. 同理推式子即可.

$$\sum_{i=1}^{n}f(i)\sum_{j=1}^{\lfloor\frac{n}{i}\rfloor}g(j)=\sum_{i=1}^{n}g(i)\sum_{j=1}^{\lfloor\frac{n}{i}\rfloor}f(j)$$

6.1.5 单位根反演

$$\sum_{i=0}^{\lfloor\frac{n}{k}\rfloor}C_n^{ik}$$

引理

$$\frac{1}{k}\sum_{i=0}^{k-1}\omega_k^{in}=[k\mid n]$$

反演

$$Ans=\sum_{i=0}^{n}C_n^i[k\mid i]$$

$$=\sum_{i=0}^{n}C_n^i\left(\frac{1}{k}\sum_{j=0}^{k-1}\omega_k^{ij}\right)$$

$$=\frac{1}{k}\sum_{i=0}^{n}C_n^i\sum_{j=0}^{k-1}\omega_k^{ij}$$

$$=\frac{1}{k}\sum_{j=0}^{k-1}\left(\sum_{i=0}^{n}C_n^i(\omega_k^j)^i\right)$$

$$=\frac{1}{k}\sum_{j=0}^{k-1}(1+\omega_k^j)^n$$

6.1.6 Arithmetic Function

$$(p-1)!\equiv-1(mod\ p)$$

$$a>1,m,n>0,\text{ then }\gcd(a^m-1,a^n-1)=a^{\gcd(n,m)}-1$$

$$\mu^2(n)=\sum_{d^2|n}\mu(d)$$

$$a>b,\gcd(a,b)=1,\text{ then }\gcd(a^m-b^m,a^n-b^n)=a^{\gcd(m,n)}-b^{\gcd(m,n)}$$

$$\prod_{k=1,gcd(k,m)=1}^{m}k\equiv\begin{cases}-1 & \bmod\ m,\ m=4,p^q,2p^q\\ 1 & \bmod\ m,\text{ otherwise}\end{cases}$$

$$\sigma_k(n)=\sum_{d|n}d^k=\prod_{i=1}^{\omega(n)}\frac{p_i^{(a_i+1)k}-1}{p_i^k-1}$$

$$J_k(n)=n^k\prod_{p|n}(1-\frac{1}{p^k})$$

$J_k(n)$ is the number of k-tuples of positive integers all less than or equal to n that form a coprime $(k+1)$-tuple together with n.

$$\sum_{\delta|n}J_k(\delta)=n^k$$

$$\sum_{i=1}^{n}\sum_{j=1}^{n}[gcd(i,j)=1]ij=\sum_{i=1}^{n}i^2\varphi(i)$$

$$\sum_{\delta|n}\delta^sJ_r(\delta)J_s(\frac{n}{\delta})=J_{r+s}(n)$$

$$\sum_{\delta|n}\varphi(\delta)d\left(\frac{n}{\delta}\right)=\sigma(n),\ \sum_{\delta|n}|\mu(\delta)|=2^{\omega(n)}$$

$$\sum_{\delta|n}2^{\omega(\delta)}=d(n^2),\ \sum_{\delta|n}d(\delta^2)=d^2(n)$$

$$\sum_{\delta|n}d\left(\frac{n}{\delta}\right)2^{\omega(\delta)}=d^2(n),\ \sum_{\delta|n}\frac{\mu(\delta)}{\delta}=\frac{\varphi(n)}{n}$$

$$\sum_{\delta|n}\frac{\mu(\delta)}{\varphi(\delta)}=d(n),\ \sum_{\delta|n}\frac{\mu^2(\delta)}{\varphi(\delta)}=\frac{n}{\varphi(n)}$$

$$n|\varphi(a^n-1)$$

$$\sum_{\substack{1\leq k\leq n\\ \gcd(k,n)=1}}f(\gcd(k-1,n))=\varphi(n)\sum_{d|n}\frac{(\mu*f)(d)}{\varphi(d)}$$

$$\varphi(\mathrm{lcm}(m,n))\varphi(\gcd(m,n))=\varphi(m)\varphi(n)$$

$$\sum_{\delta|n}d^3(\delta)=(\sum_{\delta|n}d(\delta))^2$$

$$d(uv)=\sum_{\delta|\gcd(u,v)}\mu(\delta)d(\frac{u}{\delta})d(\frac{v}{\delta})$$

$$\sigma_k(u)\sigma_k(v)=\sum_{\delta|\gcd(u,v)}\delta^k\sigma_k(\frac{uv}{\delta^2})$$

$$\mu(n)=\sum_{k=1}^{n}[\gcd(k,n)=1]\cos 2\pi\frac{k}{n}$$

$$\varphi(n)=\sum_{k=1}^{n}[\gcd(k,n)=1]=\sum_{k=1}^{n}\gcd(k,n)\cos 2\pi\frac{k}{n}$$

$$\begin{cases}S(n)=\sum_{k=1}^{n}(f*g)(k)\\ \sum_{k=1}^{n}S(\lfloor\frac{n}{k}\rfloor)=\sum_{i=1}^{n}f(i)\sum_{j=1}^{\lfloor\frac{n}{i}\rfloor}(g*1)(j)\end{cases}$$

$$\begin{cases}S(n)=\sum_{k=1}^{n}(f\cdot g)(k), g\text{ completely multiplicative}\\ \sum_{k=1}^{n}S\left(\left\lfloor\frac{n}{k}\right\rfloor\right)g(k)=\sum_{k=1}^{n}(f*1)(k)g(k)\end{cases}$$

6.1.7 Binomial Coefficients

C	0	1	2	3	4	5	6	7	8	9	10
0	1										
1	1	1									
2	1	2	1								
3	1	3	3	1							
4	1	4	6	4	1						
5	1	5	10	10	5	1					
6	1	6	15	20	15	6	1				
7	1	7	21	35	35	21	7	1			
8	1	8	28	56	70	56	28	8	1		
9	1	9	36	84	126	126	84	36	9	1	
10	1	10	45	120	210	252	210	120	45	10	1

$$\binom{n}{k}\equiv[n\&k=k]\pmod 2$$

$$\sum_{k=0}^{n}\binom{k}{m}=\binom{n+1}{m+1}$$

$$\sqrt{1+z}=1+\sum_{k=1}^{\infty}\frac{(-1)^{k-1}}{k\times 2^{2k-1}}\binom{2k-2}{k-1}z^k$$

$$\sum_{k=0}^{r}\binom{r-k}{m}\binom{s+k}{n}=\binom{r+s+1}{m+n+1}$$

$$C_{n,m}=\binom{n+m}{m}-\binom{n+m}{m-1},n\geq m$$

$$\binom{n}{k}=(-1)^k\binom{k-n-1}{k},\ \sum_{k\leq n}\binom{r+k}{k}=\binom{r+n+1}{n}$$

$$\binom{n_1+\cdots+n_p}{m}=\sum_{k_1+\cdots+k_p=m}\binom{n_1}{k_1}\cdots\binom{n_p}{k_p}$$

6.1.8 Fibonacci Numbers, Lucas Numbers

$$F(z)=\frac{z}{1-z-z^2}$$

$$\hat{\phi}=\frac{1-\sqrt{5}}{2}$$

$$\sum_{k=1}^{n}f_k=f_{n+2}-1,\ \sum_{k=1}^{n}f_k^2=f_nf_{n+1}$$

$$\sum_{k=0}^{n}f_kf_{n-k}=\frac{1}{5}(n-1)f_n+\frac{2}{5}nf_{n-1}$$

$$\frac{f_{2n}}{f_n}=f_{n-1}+f_{n+1}$$

$$f_1+2f_2+3f_3+\cdots+nf_n=nf_{n+2}-f_{n+3}+2$$

$$\gcd(f_m,f_n)=f_{\gcd(m,n)}$$

$$f_n^2+(-1)^n=f_{n+1}f_{n-1}$$

$$f_{n+k}=f_nf_{k+1}+f_{n-1}f_k$$

$$f_{2n+1}=f_n^2+f_{n+1}^2$$

$$(-1)^kf_{n-k}=f_nf_{k-1}-f_{n-1}f_k$$

```
def fib(n): # F(n), F(n + 1)
    if not n: return (0, 1)
    a, b = fib(n >> 1)
    c = a * (2 * b - a)
    d = a * a + b * b
    if n & 1:
        return (d, c + d)
    else:
        return (c, d)
```

$$\text{Modulo } f_n, f_{mn+r}\equiv\begin{cases}f_r, & m \bmod 4=0;\\ (-1)^{r+1}f_{n-r}, & m \bmod 4=1;\\ (-1)^nf_r, & m \bmod 4=2;\\ (-1)^{r+1+n}f_{n-r}, & m \bmod 4=3.\end{cases}$$

$$L_0=2,L_1=1,L_n=L_{n-1}+L_{n-2}=(\frac{1+\sqrt{5}}{2})^n+(\frac{1-\sqrt{5}}{2})^n$$

$$L(x)=\frac{2-x}{1-x-x^2}$$

除了 $n=0,4,8,16,L_n$ 是素数, 则 n 是素数.

$$\phi=\frac{1+\sqrt{5}}{2},\ \hat{\phi}=\frac{1-\sqrt{5}}{2}$$

$$F_n=\frac{\phi^n-\hat{\phi}^n}{\sqrt{5}},\ L_n=\phi^n+\hat{\phi}^n$$

$$\frac{L_n+F_n\sqrt{5}}{2}=\left(\frac{1+\sqrt{5}}{2}\right)^n$$

6.1.9 Sum of Powers

$$\sum_{i=1}^{n}i^2=\frac{n(n+1)(2n+1)}{6},\ \sum_{i=1}^{n}i^3=\left(\frac{n(n+1)}{2}\right)^2$$

$$\sum_{i=1}^{n}i^4=\frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

$$\sum_{i=1}^{n}i^5=\frac{n^2(n+1)^2(2n^2+2n-1)}{12}$$

6.1.10 Catalan Numbers 1, 1, 2, 5, 14, 42, 132, 429, 1430...

$$c_0=1,c_n=\sum_{i=0}^{n-1}c_ic_{n-1-i}=c_{n-1}\frac{4n-2}{n+1}=\frac{\binom{2n}{n}}{n+1}=\binom{2n}{n}-\binom{2n}{n-1}$$

$$c(x)=\frac{1-\sqrt{1-4x}}{2x}$$

Usage: n 对括号序列; n 个点满二叉树; $n\times n$ 的方格左下到右上不过对角线方案数; 凸 $n+2$ 边形三角形分割数; n 个数的出栈方案数; $2n$ 个顶点连接, 线段两两不交的方案数.

类卡特兰数 从 $(1,1)$ 出发走到 (n,m), 只能向右或者向上走, 不能越过 $y=x$ 这条线 (即保证 $x\geq y$), 合法方案数是 $C_{n+m-2}^{n}-C_{n+m-2}^{n-1}$.

6.1.11 Motzkin Numbers 1, 1, 2, 4, 9, 21, 51, 127, 323, 835...

圆上 n 点间画不相交弦的方案数. 选 n 个数 $k_1,k_2,...,k_n\in\{-1,0,1\}$, 保证 $\sum_i^a k_i(1\leq a\leq n)$ 非负且所有数总和为 0 的方案数.

$$M_{n+1}=M_n+\sum_{i}^{n-1}M_iM_{n-1-i}=\frac{(2n+3)M_n+3nM_{n-1}}{n+3}$$

$$M_n=\sum_{i=0}^{\lfloor\frac{n}{2}\rfloor}\binom{n}{2k}\text{Catlan}(k)$$

$$M(X)=\frac{1-x-\sqrt{1-2x-3x^2}}{2x^2}$$

6.1.12 Derangement 错排数 0, 1, 2, 9, 44, 265, 1854, 14833...

$$D_1=0,D_2=1,D_n=n!(\frac{1}{0!}-\frac{1}{1!}+\frac{1}{2!}-\frac{1}{3!}+...+\frac{(-1)^n}{n!})$$

$$D_n=(n-1)(D_{n-1}+D_{n-2})$$

6.1.13 Bell Numbers 1, 1, 2, 5, 15, 52, 203, 877, 4140 ...

n 个元素集合划分的方案数.

$$B_n=\sum_{k=1}^{n}\left\{n\atop k\right\},\ B_{n+1}=\sum_{k=0}^{n}\binom{n}{k}B_k$$

$$B_{p^m+n}\equiv mB_n+B_{n+1}\pmod p$$

$$B(x)=\sum_{n=0}^{\infty}\frac{B_n}{n!}x^n=e^{e^x-1}$$

6.1.14 Stirling Numbers

第一类 n 个元素集合分作 k 个**非空轮换**方案数.

$$\left[n+1\atop k\right]=n\left[n\atop k\right]+\left[n\atop k-1\right]$$

$$s(n,k)=(-1)^{n-k}\left[n\atop k\right]$$

n\k	0	1	2	3	4	5	6
0	1						
1	0	1					
2	0	1	1				
3	0	2	3	1			
4	0	6	11	6	1		
5	0	24	50	35	10	1	
6	0	120	274	225	85	15	1
7	0	720	1764	1624	735	175	21

$$\left[n+1\atop 2\right]=n!H_n\text{ (see 6.1.16)}$$

$$x^{\underline{n}}=\sum_k\left[n\atop k\right](-1)^{n-k}x^k$$

$$x^{\overline{n}}=\sum_k\left[n\atop k\right]x^k$$

For fixed k, EGF:

$$\sum_{n=0}^{\infty}\left[n\atop k\right]\frac{x^n}{n!}=\frac{x^k}{k!}\left(\frac{\ln(1-x)}{x}\right)^k$$

第二类 把 n 个元素集合分作 k 个**非空子集**方案数.

$$\left\{n+1\atop k\right\}=k\left\{n\atop k\right\}+\left\{n\atop k-1\right\}$$

$$m!\left\{n\atop m\right\}=\sum_k\binom{m}{k}k^n(-1)^{m-k}$$

n\k	0	1	2	3	4	5	6
0	1						
1	0	1					
2	0	1	1				
3	0	1	3	1			
4	0	1	7	6	1		
5	0	1	15	25	10	1	
6	0	1	31	90	65	15	1
7	0	1	63	301	350	140	21

$$x^n=\sum_k\left\{n\atop k\right\}x^{\underline{k}}$$

$$=\sum_k\left\{n\atop k\right\}(-1)^{n-k}x^{\overline{k}}$$

For fixed k, EGF and OGF:

$$\sum_{n=0}^{\infty}\left\{n\atop k\right\}\frac{x^n}{n!}=\frac{x^k}{k!}\left(\frac{e^x-1}{x}\right)^k$$

$$\sum_{n=0}^{\infty}\left\{n\atop k\right\}x^n=x^k\prod_{i=1}^{k}(1-ix)^{-1}$$

6.1.15 Eulerian Numbers

n\k	0	1	2	3	4	5	6
1	1						
2	1	1					
3	1	4	1				
4	1	11	11	1			
5	1	26	66	26	1		
6	1	57	302	302	57	1	
7	1	120	1191	2416	1191	120	1

$$\left\langle n\atop k\right\rangle=(k+1)\left\langle n-1\atop k\right\rangle+(n-k)\left\langle n-1\atop k-1\right\rangle$$

$$x^n=\sum_k\left\langle n\atop k\right\rangle\binom{x+k}{n}$$

$$\left\langle n\atop m\right\rangle=\sum_{k=0}^{m}\binom{n+1}{k}(m+1-k)^n(-1)^k$$

6.1.16 Harmonic Numbers, 1, 3/2, 11/6, 25/12, 137/60...

$$H_n=\sum_{k=1}^{n}\frac{1}{k},\sum_{k=1}^{n}H_k=(n+1)H_n-n$$

$$\sum_{k=1}^{n}kH_k=\frac{n(n+1)}{2}H_n-\frac{n(n-1)}{4}$$

$$\sum_{k=1}^{n}\binom{k}{m}H_k=\binom{n+1}{m+1}(H_{n+1}-\frac{1}{m+1})$$

6.1.17 卡迈克尔函数

卡迈克尔函数表示模 m 剩余系下最大的阶, 即 $\lambda(m)=\max_{a\perp m}\delta_m(a)$.

容易看出, 若 $\lambda(m)=\varphi(m)$, 则 m 存在原根.

该函数可由下述方法计算: 分解质因数 $m=p_1^{\alpha_1}p_2^{\alpha_2}...p_t^{\alpha_t}$. 则 $\lambda(m)=\text{lcm}(\lambda(p_1^{\alpha_1}),p_2^{\alpha_2},...,p_t^{\alpha_t})$. 其中对奇质数 p, $\lambda(p^\alpha)=(p-1)p^{\alpha-1}$. 对2的幂, $\lambda(2^k)=2^{k-2},s.t.k\geq 3$. $\lambda(4)=\lambda(2)=2$.

6.1.18 求拆分数

```
def penta(k):
  return k*(3*k-1)//2
def compute_partition(goal):
  p = [1]
  for n in range(1,goal+1):
    p.append(0)
    for k in range(1,n+1):
      c = (-1)**(k+1)
      for t in [penta(k), penta(-k)]:
        if (n-t) >= 0:
          p[n] = p[n] + c*p[n-t]
  return p
```

$\Phi(x) = \prod_{n=1}^{\infty}(1-x^n)$
$= \sum_{n=-\infty}^{\infty}(-1)^k x^{k(3k-1)/2}$

记 $p(n)$ 表示 n 的拆分数,$f(n,k)$ 表示将 n 拆分且每种数字使用次数必须小于等于 k 的拆分数. 则

$$P(x)\Phi(x) = 1, F(x^k)\Phi(x) = 1$$

暴力拆开卷积, 可以得到将1,-1,2,-2... 带入五边形数 $(-1)^k x^{k(3k-1)/2}$ 中, 由于小于n的五边形数只有 $\sqrt{n}$ 个, 可以 $O(n\sqrt{n})$ 计算答案:

$p(n) = p(n-1) + p(n-2) - p(n-5) - p(n-7) + \cdots,$
$f(n,k) = p(n) - p(n-k) - p(n-2k) + p(n-5k) + p(n-7k) - \cdots$

6.1.19 Bernoulli Numbers 1, 1/2, 1/6, 0, -1/30, 0, 1/42 ...

$$B(x) = \sum_{i\geq 0}\frac{B_i x^i}{i!} = \frac{x}{e^x-1}$$

$$B_n = [n=0] - \sum_{i=0}^{n-1}\binom{n}{i}\frac{B_i}{n-k+1}, \sum_{i=0}^{n}\binom{n+1}{i}B_i = 0$$

$$S_n(m) = \sum_{i=0}^{m-1} i^n = \sum_{i=0}^{n}\binom{n}{i}B_{n-i}\frac{m^{i+1}}{i+1}$$

$B_0 = 1,\ B_1 = -\frac{1}{2},\ B_4 = -\frac{1}{30},\ B_6 = \frac{1}{42},\ B_8 = -\frac{1}{30},\ \ldots.$
(除了 $B_1 = -\frac{1}{2}$ 以外, 伯努利数的奇数项都是 0.)
自然数幂次和关于次数的EGF:

$$F(x) = \sum_{k=0}^{\infty}\frac{\sum_{i=0}^{n} i^k}{k!}x^k$$
$$= \sum_{i=0}^{n} e^{ix} = \frac{e^{(n+1)x}-1}{e^x-1}$$

6.1.20 kMAX-MIN反演

$$k\mathrm{MAX}(S) = \sum_{T\subset S, T\neq\emptyset}(-1)^{|T|-k}C_{|T|-1}^{k-1}\mathrm{MIN}(T)$$

代入 $k=1$ 即为MAX-MIN反演:

$$\mathrm{MAX}(S) = \sum_{T\subset S, T\neq\emptyset}(-1)^{|T|-1}\mathrm{MIN}(T)$$

6.1.21 伍德伯里矩阵不等式

$$(A+UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1}+VA^{-1}U)^{-1}VA^{-1}$$

该等式可以动态维护矩阵的逆, 令 $C=[1], U, V$ 分别为 $1\times n$ 和 $n\times 1$ 的向量, 这样可以构造出 UCV 为只有某行或者某列不为0的矩阵, 一次修改复杂度为 $O(n^2)$.

6.1.22 Sum of Squares

$r_k(n)$ 表示用 k 个平方数组成 n 的方案数. 假设:

$$n = 2^{a_0}p_1^{2a_1}\cdots p_r^{2a_r}q_1^{b_1}\cdots q_s^{b_s}$$

其中 $p_i \equiv 3 \mod 4, q_i \equiv 1 \mod 4$, 那么

$$r_2(n) = \begin{cases} 0 & \text{if any } a_i \text{ is a half-integer} \\ 4\prod_1^r(b_i+1) & \text{if all } a_i \text{ are integers}\end{cases}$$

$r_3(n) > 0$ 当且仅当 n 不满足 $4^a(8b+7)$ 的形式 $(a,b$ 为整数).

6.1.23 枚举勾股数 Pythagorean Triple

枚举 $x^2+y^2=z^2$ 的三元组: 可令 $x=m^2-n^2, y=2mn, z=m^2+n^2$, 枚举 m 和 n 即可 $O(n)$ 枚举勾股数. 判断素勾股数方法: m,n 至少一个为偶数并且 m,n 互质, 那么 x,y,z 就是素勾股数.

6.1.24 四面体体积 Tetrahedron Volume

If U, V, W, u, v, w are lengths of edges of the tetrahedron (first three form a triangle; u opposite to U and so on)

$$V = \frac{\sqrt{4u^2v^2w^2 - \sum_{cyc}u^2(v^2+w^2-U^2)^2 + \prod_{cyc}(v^2+w^2-U^2)}}{12}$$

6.1.25 杨氏矩阵与钩子公式

满足: 格子 (i,j) 没有元素, 则它右边和上边相邻格子也没有元素; 格子 (i,j) 有元素 $a[i][j]$, 则它右边和上边相邻格子要么没有元素, 要么有元素且比 $a[i][j]$ 大.

计数: $F_1=1, F_2=2, F_n = F_{n-1} + (n-1)F_{n-2}, F(x) = e^{x+\frac{x^2}{2}}$
钩子公式: 对于给定形状 λ, 不同杨氏矩阵的个数为:

$$d_\lambda = \frac{n!}{\prod h_\lambda(i,j)}$$

$h_\lambda(i,j)$ 表示该格子右边和上边的格子数量加1.

6.1.26 常见博弈游戏

Nim-K游戏 n 堆石子轮流拿, 每次最多可以拿k堆石子, 谁走最后一步输。结论: 把每一堆石子的sg值 (即石子数量) 二进制分解, 先手必败当且仅当每一位二进制位上1的个数是 (k+1) 的倍数.

Anti-Nim游戏 n 堆石子轮流拿, 谁走最后一步输。结论: 先手胜当且仅当1. 所有堆石子数都为1且游戏的SG值为0 (即有偶数个孤单堆-每堆只有1个石子数) 2. 存在某堆石子数大于1且游戏的SG值不为0.

斐波那契博弈 有一堆物品, 两人轮流取物品, 先手最少取一个, 至多无上限, 但不能把物品取完, 之后每次取的物品数不能超过上一次取的物品数的二倍且至少为一件, 取走最后一件物品的人获胜. 结论: 先手胜当且仅当物品数 n 不是斐波那契数.

威佐夫博弈 有两堆石子, 博弈双方每次可以取一堆石子中的任意个, 不能不取, 或者取两堆石子中的相同个. 先取完者赢. 结论: 求出两堆石子 A 和 B 的差值 C, 如果 $\left\lfloor C * \frac{\sqrt{5}+1}{2}\right\rfloor = min(A,B)$ 那么后手赢, 否则先手赢.

约瑟夫环 n 个人 $0,\ldots,n-1$, 令 $f_{i,m}$ 为 i 个人报 m 的胜利者, 则 $f_{1,m} = 0, f_{i,m} = (f_{i-1,m}+m) \bmod i$.

阶梯Nim 在一个阶梯上, 每次选一个台阶上任意个式子移到下一个台阶上, 不可移动者输. 结论: SG值等于奇数层台阶上石子数的异或和. 对于树形结构也适用, 奇数层节点上所有石子数异或起来即可.

图上博弈 给定无向图, 先手从某点开始走, 只能走相邻且未走过的点, 无法移动者输. 对该图求最大匹配, 若某个点不一定在最大匹配中则先手必败, 否则先手必胜.

最大最小定理求纳什均衡点 在二人零和博弈中, 可以用以下方式求出一个纳什均衡点: 在博弈双方中**任选**一方, 求混合策略 $\mathbf{p}$ 使得对方选择任意一个**纯策略**时, 己方的最小收益最大 (等价于对方的最大收益最小). 据此可以求出双方在此局面下的最优期望得分, 分别等于己方最大的最小收益和对方最小的最大收益. 一般而言, 可以得到形如

$$\max_{\mathbf{p}}\min_{i}\sum_{p_j\in\mathbf{p}, p_j\geq 0, \sum p_j=1} p_j w_{i,j}$$

的形式. 当 $\sum p_j w_{i,j}$ 可以表示成只与 i 有关的函数 $f(i)$ 时, 可以令初始时 $p_i=0$, 不断调整 $\sum p_j w_{i,j}$ 最小的那个i的概率 p_i, 直至无法调整或者 $\sum p_j = 1$ 为止.

6.1.27 概率相关

$D(X) = E(X-E(X))^2 = E(X^2) - (E(X))^2, D(X+Y) = D(X) + D(Y),$
$D(aX) = a^2D(X), E[x] = \sum_{i=1}^{\infty}P(X\geq i)$, m 个数的方差: $s^2 = \frac{\sum_{i=1}^{m}x_i^2}{m} - \bar{x}^2$

6.1.28 邻接矩阵行列式的意义

在无向图中取若干个环, 一种取法权值就是边权的乘积, 对行列式的贡献是 $(-1)^{\text{even}}$, 其中 even 是偶环的个数.

6.1.29 Others (某些近似数值公式在这里)

$$e^x = \sum_{n=0}^{\infty}\frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$$

$$\sin(x) = \sum_{n=0}^{\infty}\frac{(-1)^n x^{2n+1}}{(2n+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots$$

$$\cos(x) = \sum_{n=0}^{\infty}\frac{(-1)^n x^{2n}}{(2n)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \cdots$$

$$\ln(1+x) = \sum_{n=1}^{\infty}\frac{(-1)^{n+1}x^n}{n} = x - \frac{x^2}{2} + \frac{x^3}{3} - \cdots \quad \text{for} \quad -1 < x \leq 1$$

$$\arctan(x) = \sum_{n=0}^{\infty}\frac{(-1)^n x^{2n+1}}{2n+1} = x - \frac{x^3}{3} + \frac{x^5}{5} - \cdots \quad \text{for} \quad -1 \leq x \leq 1$$

$$(1+x)^k = \sum_{n=0}^{\infty}\binom{k}{n}x^n = 1 + kx + \frac{k(k-1)}{2!}x^2 + \frac{k(k-1)(k-2)}{3!}x^3 + \cdots$$

$$\arcsin(x) = x + \sum_{n=1}^{\infty}\frac{(2n-1)!!}{(2n)!!}\frac{x^{2n+1}}{2n+1}$$

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \cdots, \frac{1}{\pi} = \frac{2\sqrt{2}}{9801}\sum_{k=0}^{\infty}\frac{(4k)!(1103+26390k)}{(k!)^4 396^{4k}}$$

$$\sum_{i=1}^{n}\frac{1}{n} \approx \ln\left(n+\frac{1}{2}\right) + \frac{1}{24(n+0.5)^2} + \Gamma, (\Gamma \approx 0.57721566490153286060 65)$$

$$n! = \sqrt{2\pi n}(\frac{n}{e})^n(1 + \frac{1}{12n} + \frac{1}{288n^2} + O(\frac{1}{n^3}))$$

$$\max\{x_a-x_b, y_a-y_b, z_a-z_b\} - \min\{x_a-x_b, y_a-y_b, z_a-z_b\}$$
$$= \frac{1}{2}\sum_{cyc}|(x_a-y_a)-(x_b-y_b)|$$

$$\sum_{k=0}^{n}kc^k = \frac{nc^{n+2} - (n+1)c^{n+1} + c}{(c-1)^2}$$

$$(a+b)(b+c)(c+a) = \frac{(a+b+c)^3 - a^3 - b^3 - c^3}{3}$$

$$a^3+b^3 = (a+b)(a^2-ab+b^2), a^3-b^3 = (a-b)(a^2+ab+b^2)$$

$n \bmod 2 = 1$:

$$a^n + b^n = (a+b)(a^{n-1} - a^{n-2}b + a^{n-3}b^2 - \ldots - ab^{n-2} + b^{n-1})$$

划分问题: n 个 $k-1$ 维向量最多把 k 维空间分为 $\sum_{i=0}^{k}C_n^i$ 份.

6.2 Calculus, Integration Table 导数积分表

$(\frac{u}{v})' = \frac{u'v-uv'}{v^2}$
$(a^x)' = (\ln a)a^x$
$(\tan x)' = \sec^2 x$
$(\cot x)' = \csc^2 x$
$(\sec x)' = \tan x \sec x$
$(\csc x)' = -\cot x \csc x$
$(\arcsin x)' = \frac{1}{\sqrt{1-x^2}}$
$(\arccos x)' = -\frac{1}{\sqrt{1-x^2}}$
$(\arctan x)' = \frac{1}{1+x^2}$
$(\operatorname{arccot} x)' = -\frac{1}{1+x^2}$
$(\operatorname{arccsc} x)' = -\frac{1}{x\sqrt{1-x^2}}$
$(\operatorname{arcsec} x)' = \frac{1}{x\sqrt{1-x^2}}$
$(\tanh x)' = \operatorname{sech}^2 x$
$(\coth x)' = -\operatorname{csch}^2 x$
$(\operatorname{sech} x)' = -\operatorname{sech} x \tanh x$
$(\operatorname{csch} x)' = -\operatorname{csch} x \coth x$
$(\operatorname{arcsinh} x)' = \frac{1}{\sqrt{1+x^2}}$
$(\operatorname{arccosh} x)' = \frac{1}{\sqrt{x^2-1}}$
$(\operatorname{arctanh} x)' = \frac{1}{1-x^2}$
$(\operatorname{arccoth} x)' = \frac{1}{x^2-1}$
$(\operatorname{arccsch} x)' = -\frac{1}{|x|\sqrt{1+x^2}}$
$(\operatorname{arcsech} x)' = -\frac{1}{x\sqrt{1-x^2}}$

· $ax^2+bx+c(a>0)$

$$\int \frac{\mathrm{d}x}{ax^2+bx+c}=\begin{cases}\frac{2}{\sqrt{4ac-b^2}}\arctan\frac{2ax+b}{\sqrt{4ac-b^2}}+C & (b^2<4ac)\\ \frac{1}{\sqrt{b^2-4ac}}\ln\left|\frac{2ax+b-\sqrt{b^2-4ac}}{2ax+b+\sqrt{b^2-4ac}}\right|+C & (b^2>4ac)\end{cases}$$

$$\int \frac{x}{ax^2+bx+c}\mathrm{d}x=\frac{1}{2a}\ln|ax^2+bx+c|-\frac{b}{2a}\int\frac{\mathrm{d}x}{ax^2+bx+c}$$

· $\sqrt{\pm ax^2+bx+c}(a>0)$

$$\int\frac{\mathrm{d}x}{\sqrt{ax^2+bx+c}}=\frac{1}{\sqrt{a}}\ln|2ax+b+2\sqrt{a}\sqrt{ax^2+bx+c}|+C$$

$$\int\sqrt{ax^2+bx+c}\mathrm{d}x=\frac{2ax+b}{4a}\sqrt{ax^2+bx+c}+\frac{4ac-b^2}{8\sqrt{a^3}}\ln|2ax+b+2\sqrt{a}\sqrt{ax^2+bx+c}|+C$$

$$\int\frac{x}{\sqrt{ax^2+bx+c}}\mathrm{d}x=\frac{1}{a}\sqrt{ax^2+bx+c}-\frac{b}{2\sqrt{a^3}}\ln|2ax+b+2\sqrt{a}\sqrt{ax^2+bx+c}|+C$$

$$\int\frac{\mathrm{d}x}{\sqrt{c+bx-ax^2}}=-\frac{1}{\sqrt{a}}\arcsin\frac{2ax-b}{\sqrt{b^2+4ac}}+C$$

$$\int\sqrt{c+bx-ax^2}\mathrm{d}x=\frac{2ax-b}{4a}\sqrt{c+bx-ax^2}+\frac{b^2+4ac}{8\sqrt{a^3}}\arcsin\frac{2ax-b}{\sqrt{b^2+4ac}}+C$$

$$\int\frac{x}{\sqrt{c+bx-ax^2}}\mathrm{d}x=-\frac{1}{a}\sqrt{c+bx-ax^2}+\frac{b}{2\sqrt{a^3}}\arcsin\frac{2ax-b}{\sqrt{b^2+4ac}}+C$$

· $\sqrt{\pm\frac{x-a}{x-b}}$ 或 $\sqrt{(x-a)(x-b)}$, $(a<b)$

$$\int\frac{\mathrm{d}x}{\sqrt{(x-a)(b-x)}}=2\arcsin\sqrt{\frac{x-a}{b-x}}+C$$

$$\int\sqrt{(x-a)(b-x)}\mathrm{d}x=\frac{2x-a-b}{4}\sqrt{(x-a)(b-x)}+\frac{(b-a)^2}{4}\arcsin\sqrt{\frac{x-a}{b-x}}+C$$

三角函数的积分 (省略 $+C$)

$$\int\tan x\mathrm{d}x=-\ln|\cos x|$$
$$\int\cot x\mathrm{d}x=\ln|\sin x|$$
$$\int\sec x\mathrm{d}x=\ln\left|\tan\left(\frac{\pi}{4}+\frac{x}{2}\right)\right|=\ln|\sec x+\tan x|$$
$$\int\csc x\mathrm{d}x=\ln\left|\tan\frac{x}{2}\right|=\ln|\csc x-\cot x|$$

$$\int\sec^2 x\mathrm{d}x=\tan x$$
$$\int\csc^2 x\mathrm{d}x=-\cot x$$
$$\int\sec x\tan x\mathrm{d}x=\sec x$$
$$\int\csc x\cot x\mathrm{d}x=-\csc x$$
$$\int\sin^2 x\mathrm{d}x=\frac{x}{2}-\frac{1}{4}\sin 2x$$
$$\int\cos^2 x\mathrm{d}x=\frac{x}{2}+\frac{1}{4}\sin 2x$$

$$\int\sin^n x\mathrm{d}x=-\frac{1}{n}\sin^{n-1}x\cos x+\frac{n-1}{n}\int\sin^{n-2}x\mathrm{d}x$$
$$\int\cos^n x\mathrm{d}x=\frac{1}{n}\cos^{n-1}x\sin x+\frac{n-1}{n}\int\cos^{n-2}x\mathrm{d}x$$
$$\int\frac{\mathrm{d}x}{\sin^n x}=-\frac{1}{n-1}\frac{\cos x}{\sin^{n-1}x}+\frac{n-2}{n-1}\int\frac{\mathrm{d}x}{\sin^{n-2}x}$$
$$\int\frac{\mathrm{d}x}{\cos^n x}=\frac{1}{n-1}\frac{\sin x}{\cos^{n-1}x}+\frac{n-2}{n-1}\int\frac{\mathrm{d}x}{\cos^{n-2}x}$$

$$\begin{aligned}&\int\cos^m x\sin^n x\mathrm{d}x\\ =&\frac{1}{m+n}\cos^{m-1}x\sin^{n+1}x+\frac{m-1}{m+n}\int\cos^{m-2}x\sin^n x\mathrm{d}x\\ =&-\frac{1}{m+n}\cos^{m+1}x\sin^{n-1}x+\frac{n-1}{m+1}\int\cos^m x\sin^{n-2}x\mathrm{d}x\end{aligned}$$

$$\int\frac{\mathrm{d}x}{a+b\sin x}=\begin{cases}\frac{2}{\sqrt{a^2-b^2}}\arctan\frac{a\tan\frac{x}{2}+b}{\sqrt{a^2-b^2}} & (a^2>b^2)\\ \frac{1}{\sqrt{b^2-a^2}}\ln\left|\frac{a\tan\frac{x}{2}+b-\sqrt{b^2-a^2}}{a\tan\frac{x}{2}+b+\sqrt{b^2-a^2}}\right| & (a^2<b^2)\end{cases}$$

$$\int\frac{\mathrm{d}x}{a+b\cos x}=\begin{cases}\frac{2}{a+b}\sqrt{\frac{a+b}{a-b}}\arctan\left(\sqrt{\frac{a-b}{a+b}}\tan\frac{x}{2}\right) & (a^2>b^2)\\ \frac{1}{a+b}\sqrt{\frac{a+b}{a-b}}\ln\left|\frac{\tan\frac{x}{2}+\sqrt{\frac{a+b}{b-a}}}{\tan\frac{x}{2}-\sqrt{\frac{a+b}{b-a}}}\right| & (a^2<b^2)\end{cases}$$

$$\int\frac{\mathrm{d}x}{a^2\cos^2x+b^2\sin^2x}=\frac{1}{ab}\arctan\left(\frac{b}{a}\tan x\right)$$
$$\int\frac{\mathrm{d}x}{a^2\cos^2x-b^2\sin^2x}=\frac{1}{2ab}\ln\left|\frac{b\tan x+a}{b\tan x-a}\right|$$
$$\int x\sin ax\mathrm{d}x=\frac{1}{a^2}\sin ax-\frac{1}{a}x\cos ax$$
$$\int x^2\sin ax\mathrm{d}x=-\frac{1}{a}x^2\cos ax+\frac{2}{a^2}x\sin ax+\frac{2}{a^3}\cos ax$$
$$\int x\cos ax\mathrm{d}x=\frac{1}{a^2}\cos ax+\frac{1}{a}x\sin ax$$
$$\int x^2\cos ax\mathrm{d}x=\frac{1}{a}x^2\sin ax+\frac{2}{a^2}x\cos ax-\frac{2}{a^3}\sin ax$$

反三角函数的积分 (其中 $a>0$)

$$\int\arcsin\frac{x}{a}\mathrm{d}x=x\arcsin\frac{x}{a}+\sqrt{a^2-x^2}+C$$
$$\int x\arcsin\frac{x}{a}\mathrm{d}x=\left(\frac{x^2}{2}-\frac{a^2}{4}\right)\arcsin\frac{x}{a}+\frac{x}{4}\sqrt{x^2-x^2}+C$$
$$\int x^2\arcsin\frac{x}{a}\mathrm{d}x=\frac{x^3}{3}\arcsin\frac{x}{a}+\frac{1}{9}(x^2+2a^2)\sqrt{a^2-x^2}+C$$
$$\int\arccos\frac{x}{a}\mathrm{d}x=x\,arccos\frac{x}{a}-\sqrt{a^2-x^2}+C$$
$$\int x\arccos\frac{x}{a}\mathrm{d}x=\left(\frac{x^2}{2}-\frac{a^2}{4}\right)\arccos\frac{x}{a}-\frac{x}{4}\sqrt{a^2-x^2}+C$$
$$\int x^2\arccos\frac{x}{a}\mathrm{d}x=\frac{x^3}{3}\arccos\frac{x}{a}-\frac{1}{9}(x^2+2a^2)\sqrt{a^2-x^2}+C$$
$$\int\arctan\frac{x}{a}\mathrm{d}x=x\arctan\frac{x}{a}-\frac{a}{2}\ln(a^2+x^2)+C$$
$$\int x\arctan\frac{x}{a}\mathrm{d}x=\frac{1}{2}(a^2+x^2)\arctan\frac{x}{a}-\frac{a}{2}x+C$$
$$\int x^2\arctan\frac{x}{a}\mathrm{d}x=\frac{x^3}{3}\arctan\frac{x}{a}-\frac{a}{6}x^2+\frac{a^3}{6}\ln(a^2+x^2)+C$$

指数函数的积分

$$\int a^x\mathrm{d}x=\frac{1}{\ln a}a^x+C$$
$$\int\mathrm{e}^{ax}\mathrm{d}x=\frac{1}{a}a^{ax}+C$$
$$\int x\mathrm{e}^{ax}\mathrm{d}x=\frac{1}{a^2}(ax-1)a^{ax}+C$$
$$\int x^n\mathrm{e}^{ax}\mathrm{d}x=\frac{1}{a}x^n\mathrm{e}^{ax}-\frac{n}{a}\int x^{n-1}\mathrm{e}^{ax}\mathrm{d}x$$
$$\int xa^x\mathrm{d}x=\frac{x}{\ln a}a^x-\frac{1}{(\ln a)^2}a^x+C$$
$$\int x^n a^x\mathrm{d}x=\frac{1}{\ln a}x^n a^x-\frac{n}{\ln a}\int x^{n-1}a^x\mathrm{d}x$$
$$\int\mathrm{e}^{ax}\sin bx\mathrm{d}x=\frac{1}{a^2+b^2}\mathrm{e}^{ax}(a\sin bx-b\cos bx)+C$$
$$\int\mathrm{e}^{ax}\cos bx\mathrm{d}x=\frac{1}{a^2+b^2}\mathrm{e}^{ax}(b\sin bx+a\cos bx)+C$$
$$\int\mathrm{e}^{ax}\sin^n bx\mathrm{d}x=\frac{1}{a^2+b^2n^2}\mathrm{e}^{ax}\sin^{n-1}bx(a\sin bx-nb\cos bx)+\frac{n(n-1)b^2}{a^2+b^2n^2}\int\mathrm{e}^{ax}\sin^{n-2}bx\mathrm{d}x$$
$$\int\mathrm{e}^{ax}\cos^n bx\mathrm{d}x=\frac{1}{a^2+b^2n^2}\mathrm{e}^{ax}\cos^{n-1}bx(a\cos bx+nb\sin bx)+\frac{n(n-1)b^2}{a^2+b^2n^2}\int\mathrm{e}^{ax}\cos^{n-2}bx\mathrm{d}x$$

对数函数的积分

$$\int\ln x\mathrm{d}x=x\ln x-x+C$$
$$\int\frac{\mathrm{d}x}{x\ln x}=\ln|\ln x|+C$$
$$\int x^n\ln x\mathrm{d}x=\frac{1}{n+1}x^{n+1}(\ln x-\frac{1}{n+1})+C$$
$$\int(\ln x)^n\mathrm{d}x=x(\ln x)^n-n\int(\ln x)^{n-1}\mathrm{d}x$$
$$\int x^m(\ln x)^n\mathrm{d}x=\frac{1}{m+1}x^{m+1}(\ln x)^n-\frac{n}{m+1}\int x^m(\ln x)^{n-1}\mathrm{d}x$$

6.3 Python Hint

```python
def RandomAndList():
  import random
  random.normalvariate(0.5, 0.1)
  l = [str(i) for i in range(9)]
  sorted(l), min(l), max(l), len(l)
  random.shuffle(l)
  l.sort(key=lambda x:x ^ 1,reverse=True)
  from functools import cmp_to_key
  l.sort(key=cmp_to_key(lambda x, y:(y^1)-(x^1)))
  from itertools import *
  for i in product('ABCD', repeat=2):
    pass # AA AB AC AD BA BB BC BD CA CB CC CD DA DB DC DD
  for i in permutations('ABCD', repeat=2):
    pass # AB AC AD BA BC BD CA CB CD DA DB DC
  for i in combinations('ABCD', repeat=2):
    pass # AB AC AD BC BD CD
  for i in combinations_with_replacement('ABCD', repeat=2):
    pass # AA AB AC AD BB BC BD CC CD DD
def FractionOperation():
  from fractions import Fraction
  a.numerator, a.denominator, str(a)
  a = Fraction(0.233).limit_denominator(1000)
def DecimalOperation():
  from decimal import Decimal, getcontext, FloatOperation
  getcontext().prec = 100
  getcontext().rounding = getattr(decimal, 'ROUND_HALF_EVEN')
  # default; other: FLOOR, CELILING, DOWN, ...
  getcontext().traps[FloatOperation] = True
  Decimal((0, (1, 4, 1, 4), -3)) # 1.414
  a = Decimal(1<<31) / Decimal(100000)
  print(f"{a:.9f}") # 21474.83648
  print(a.sqrt(), a.ln(), a.log10(), a.exp(), a ** 2)
def Complex():
  a = 1-2j
  print(a.real, a.imag, a.conjugate())
def FastIO():
  import sys, atexit, io
  _INPUT_LINES = sys.stdin.read().splitlines()
  input = iter(_INPUT_LINES).__next__
  _OUTPUT_BUFFER = io.StringIO()
  sys.stdout = _OUTPUT_BUFFER
  @atexit.register
  def write():
    sys.__stdout__.write(_OUTPUT_BUFFER.getvalue())
```

7. Miscellany

7.1 Zeller 日期公式

```cpp
// weekday=(id+1)%7;{Sun=0,Mon=1,...}   getId(1, 1, 1) = 0
int getId(int y, int m, int d) {
  if (m < 3) { y --; m += 12; }
  return 365 * y + y / 4 - y / 100 + y / 400 + (153 * (m - 3) + 2) / 5 + d - 307; }
//  y<0: 统一加400的倍数年
auto date(int id) {
  int x=id+1789995, n, i, j, y, m, d;
  n = 4 * x / 146097; x -= (146097 * n + 3) / 4;
  i = (4000 * (x + 1)) / 1461001; x -= 1461 * i / 4 - 31;
  j = 80 * x / 2447; d = x - 2447 * j / 80; x = j / 11;
  m = j + 2 - 12 * x; y = 100 * (n - 49) + i + x;
  return make_tuple(y, m, d); }
```

7.2 有理数二分: Stern-Brocot 树, Farey 序列

```cpp
def build(a, b, c, d, level = 1):
    x = a + c; y = b + d
    build(a, b, x, y, level + 1)
    build(x, y, c, d, level + 1)
build(0, 1, 1, 0) # 最简分数, Stern-Brocot
build(0, 1, 1, 1) # 最简真分数, Farey
```

7.3 黄金三分

```
constexpr LD R = (sqrt(5) - 1) / 2;
auto split = [](LD l, LD r) { return l + (r - l) * R; };
LD solve(LD a, LD c, auto f) {
 | LD b = split(a, c), bv = f(b);
 | for (int _ = T; _; _--) {
 |  | LD x = split(a, b), xv = f(x);
 |  | if (xv < bv) c = b, b = x, bv = xv; // 最小化，注意符号
 |  | else a = c, c = x;
 | } return bv; }
```

7.4 DP 优化

7.4.1 四边形不等式

```
// a ≤ b ≤ c ≤ d : w(b,c) ≤ w(a,d),
  ↪ w(a,c) + w(b,d) ≤ w(a,d) + w(b,c)
for (int len = 2; len <= n; ++len) {
 | for (int l = 1, r = len; r <= n; ++l, ++r) {
 |  | f[l][r] = INF;
 |  | for (int k = m[l][r - 1]; k <= m[l + 1][r]; ++k) {
 |  |  | if (f[l][r] > f[l][k] + f[k + 1][r] + w(l, r)) {
 |  |  |  | f[l][r] = f[l][k] + f[k + 1][r] + w(l, r);
 |  |  |  | m[l][r] = k;
 | } } }
```

7.4.2 树形背包优化

限制: 必须取与根节点相连的一个连通块.
转化: 一个点的子树对应于DFS序中的一个区间. 则每个点的决策为, 取该点, 或者舍弃该点对应的区间. 从后往前dp, 设 $f(i,v)$ 表示从后往前考虑到i号点, 总体积为V的最优价值, 设i号点对应的区间为 $[i, i+\text{size}_i-1]$, 转移为 $f(i,v)=\max\{f(i+1,V-v_i)+w_i, f(i+\text{size}_i, v)\}$.
如果要求任意连通块, 则点分治后转为指定根的连通块问题即可.

7.4.3 $O(n \cdot \max a_i)$ Subset Sum

```
int SubsetSum(vector <int> &a, int t) {
  int B = *max_element(a.begin(), a.end());
  int n = (int) a.size(), s = 0, i = 0;
  while (i < n && s + a[i] <= t) s += a[i ++];
  if (i == n) return s;
  vector <int> f(2 * B + 1, -1), pre (B + 1, -1);
  f[s - (t - B)] = i;
  for ( ; i < n; i++) {
    s += a[i];
    for (int d = 0; d <= B; d++) pre[d] = max(0, f[d + B]);
    for (int d = B; d >= 0; d--)
      f[d + a[i]] = max(f[d + a[i]], f[d]);
    for (int d = 2 * B; d > B; --d)
      for (int j = pre[d - B]; j < f[d]; j++)
        f[d - a[j]] = max(f[d - a[j]], j); }
  for (i = 0; i <= B; i++) if (f[B - i] >= 0) return t - i;}
```

7.5 Hash Table

```cpp
template <class T,int P = 314159/*,451411,1141109,2119969*/>
struct hashmap {
ULL id[P]; T val[P];
int R[P]; // del: few clears
hashmap() {memset(id, -1, sizeof id);}
T get(const ULL &x) const {
 | for (int i = int(x % P), j = 1; ~id[i]; i = (i + j) % P,
     ↪ j = (j + 2) % P /*unroll if needed*/) {
 |  | if (id[i] == x) return val[i]; }
 | return 0; }
T& operator [] (const ULL &x) {
 | for (int i = int(x % P), j = 1;       ; i = (i + j) % P,
     ↪ j = (j + 2) % P) {
 |  | if (id[i] == x) return val[i];
 |  | else if (id[i] == -1llu) {
 |  |  | id[i] = x;
 |  |  | R[++R[0]] = i;  // del: few clears
 |  |  | return val[i]; } } }
void clear() { // del: few clears
for (int &x = R[0]; x; id[R[x]] = -1, val[R[x]] = 0, --x);
void fullclear() {memset(id, -1, sizeof id); R[0] = 0; } };
```

7.6 基数排序

```cpp
const int SZ = 1 << 8; // almost always fit in L1 cache
void SORT(int a[], int c[], int n, int w) {
 | for(int i=0; i<SZ; i++) b[i] = 0;
 | for(int i=1; i<=n; i++) b[(a[i]>>w) & (SZ-1)]++;
 | for(int i=1; i<SZ; i++) b[i] += b[i - 1];
 | for(int i=n; i; i--) c[b[(a[i]>>w) & (SZ-1)]--] = a[i];}
void Sort(int *a, int n){
 | SORT(a, c, n, 0); SORT(c, a, n, 8);
 | SORT(a, c, n, 16); SORT(c, a, n, 24); }
```

7.7 Hacks: O3, 读入优化, Bitset, builtin

```cpp
//fast = O3 + ffast-math + fallow-store-data-races
#pragma GCC optimize("Ofast")
#pragma GCC target("sse2,abm,fma,mmx,avx2,tune=native")
const int SZ = 1 << 16; int getc() {
 | static char buf[SZ], *ptr = buf, *top = buf;
 | if (ptr == top) {
 |  | ptr = buf, top = buf + fread(buf, 1, SZ, stdin);
 |  | if (top == buf) return -1; }
 | return *ptr++; }
idx=b._Find_first();idx!=b.size();idx=b._Find_next(idx);
struct HashFunc{size_t operator()(const KEY &key)const{}};
__builtin_uaddll_overflow(a, b, &c) // binary big int
void GospersHack(int k, int n) {
 | for (int s = (1 << k) - 1, c, r; s < (1 << n);
 | c = s & -s, r = s + c, s = (((r ^ s) >> 2) / c) | r); }
```

7.8 试机赛与纪律文件

- 检查身份证件: 护照、学生证、胸牌以及现场所需通行证。
- 确认什么东西能带进场。特别注意: 智能手表、金属（钥匙）等等。
- 测试鼠标、键盘、显示器和座椅。如果有问题, 立刻联系工作人员。
- 测试比赛提交方式。如果有 `submit` 命令, 确认如何使用。
- 设置自动保存，自动备份。
- 测试编译器版本。C++20 `cin >> (s + 1)`; C++17 `auto [x, y]: a`; C++14 `[] (auto x, auto y)`; C++11 `auto`; `bits/stdc++.h`; `pb_ds`。

```cpp
#include <ext/rope>
using namespace __gnu_cxx;
rope <int> R; R.insert(y, x); R[x]; R.erase(x, 1);
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
tree <int, null_type, less<int>, rb_tree_tag,
tree_order_statistics_node_update> s;
s.insert(1); s.find_by_order(0); s.order_of_key(5);
```

- 测试 `__int128`, `__float128`, `long double`
- 测试代码长度限制，尝试触发 NO-OUTPUT, OUTPUT-LIMIT, RUN-ERROR。
- 测试 pragma 是否 CE。
- 测试 clar: 如果问不同类型的愚蠢的问题, 得到的回复是否不一样?
- 测试 `-fsanitize address, undefined, define _GLIBCXX_DEBUG`
- 测试 `time` 命令是否能显示内存占用。`/usr/bin/time -v ./a.out`
- 测试 clock() 是否能够正常工作; 测试本地性能与提交性能。

```cpp
const int N = 1 << 20;
for (int T = 4; T; T--) {
    int a[N], b[N], c[N];
    for (int i = 0; i < N; i++) a[i] = i, b[i] = N - i;
    ntt_init(N); ntt(a, N, 1); ntt(b, N, 1);
    for (int i = 0; i < N; i++) c[i] = a[i] * b[i];
    ntt(c, N, -1); assert (c[0] == 0xad223d); }
// 机房: 190ms; (1s T = 21)
// QOJ: 340+-40ms; (1s T = 11.9)
// CF: 1050+-100ms; (1s T = 3.9)
```

```cpp
import sys
sys.set_int_max_str_digits(0) # >= Python 3.11
T = 3
for _ in range(T):
    assert str(3 ** 100000 * 10 ** 100000)[:9] == "133497141"
# 机房: 590ms; (1s T = 5)
# QOJ: 201ms; (1s T = 16)
# CF Python3: 1100ms; (1s T = 2.9) Pypy3: 310ms; (1s T = 12)
```

提交前检查:

- 保存, 编译, 测过样例，测过边界 n=0/1
- 数组开到 n/2n/m/n+32, LL/int128
- 多测清空，调用初始化, 清空时数组大小, 没读入完就 break
- 取模取到正值，输入输出格式 (`%Lf`, `%llu`)
- 时间空间限制, 关闭流同步，时间卡开 Ofast

提交后检查:

- sum 多组输入: 应只用了输入内容 (memset TL, 错下标 WA), 是否正确撤销
- 输入是否保证顺序，保证是整数，保证三点共线
- int/LL 溢出, INF/-1 大小，浮点数 eps 和 = 0, `__builtin_popcountll`
- 类似 `pair <LL, int> x = pair <int, int>()`; 不会报警告
- 离散化与二分: `lower_bound upper_bound +-1`, begin(), end()
- 自定义排序：排序方向，比较函数为小于，考虑坏点: 如叉积 (0, 0)
- 样例是否为对称/回文的？考虑构造不对称的情况，正序逆序
- 复制过的代码对应位置正确修改
- 检查变量重名，变量复用
- 图: 边标号初始是否为 1, 单双向边, 反向边边权
- 几何: 共线, sqrt(-0.0), nan / inf, 重点, 除零 det/dot, 旋转方向, 求得的是否是所求

7.9 Constant Table 常数表

Random primes generated at Wed Jul 29 21:33:08 2026
5e2 397 421 449 461 491 499 503 521 523 547 563 569 577 619
1e3 853 857 929 941 953 983 991 1019 1039 1051 1069 1087
3e4 27847 27851 28429 30313 30509 31547 31601 31849 32003
1e5 93199 93763 97609 97919 98639 98807 98939 100547 103141
3e5 281663 290737 294181 296563 306301 309541 311203 313699
1e6 948877 954829 958553 1034233 1048129 1055321 1058263
2e6 1864691 1870933 1968721 1989233 2000353 2044963 2048521
5e6 4726499 4738457 4763477 4794961 4908979 5261071 5324381
1e7 9586763 9644819 9716513 9824393 9924493 10360657 10551977
2e7 18957847 19130539 19834979 20558311 20725853 21061883
1e9 951811417 953531647 954219817 986298539 998876579 1033787771
2e9 1869760247 1932754013 1932856193 2105062559 2111869147
NTT 976224257 r=3 (20) 985661441 r=3 (22) 998244353 r=3 (23)
1004535809 r=3 (21) 1007681537 r=3 (20) 1012924417 r=5 (21)
1045430273 r=3 (20) 1051721729 r=6 (20) 1053818881 r=7 (20)

n	$\log_{10} n$	$n!$	$C(n, n/2)$	$\mathrm{LCM}(1\ldots n)$	P_n
2	0.30102999	2	2	2	2
3	0.47712125	6	3	6	3
4	0.60205999	24	6	12	5
5	0.69897000	120	10	60	7
6	0.77815125	720	20	60	11
7	0.84509804	5040	35	420	15
8	0.90308998	40320	70	840	22
9	0.95424251	362880	126	2520	30
10	1	3628800	252	2520	42
11	1.04139269	39916800	462	27720	56
12	1.07918125	479001600	924	27720	77
15	1.17609126	1.31e12	6435	360360	176
20	1.30103000	2.43e18	184756	232792560	627
25	1.39794001	1.55e25	5200300	26771144400	1958
30	1.47712125	2.65e32	155117520	1.444e14	5604
P_n	37338_{40}	204226_{50}	966467_{60}	190569292_{100}	$1e9_{114}$

$n \le$	10	100	1e3	1e4	1e5	1e6
$\max \omega(n)$	2	3	4	5	6	7
$\max d(n)$	4	12	32	64	128	240
$\pi(n)$	4	25	168	1229	9592	78498
$n \le$	1e7	1e8	1e9	1e10	1e11	1e12
$\max \omega(n)$	8	8	9	10	10	11
$\max d(n)$	448	768	1344	2304	4032	6720
$\pi(n)$	664579	5761455	5.08e7	4.55e8	4.12e9	3.7e10
$n \le$	1e13	1e14	1e15	1e16	1e17	1e18
$\max \omega(n)$	12	12	13	13	14	15
$\max d(n)$	10752	17280	26880	41472	64512	103680
$\pi(n)$	Prime number theorem: $\pi(x) \sim x/\log(x)$					

Vimrc, Bashrc

```
source $VIMRUNTIME/mswin.vim
behave mswin
set mouse=a ci ai si nu ts=4 sw=4 is hls backup undofile
color slate
map <F7> : ! make %<<CR>
map <F8> : ! time ./%< <CR>
```

```
export CXXFLAGS='-g -Wall -Wextra -Wconversion -Wshadow
  ↪ -std=gnu++20'
ulimit -s 1048576
```

VSCode: 打开自动保存

STL 积分/求和

1. $\int_0^1 t^{x-1}(1-t)^{y-1}dt = beta(x,y) = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)}$
2. $\int_0^{+\infty} t^{num-1}e^{-t}dt = tgamma(num) = e^{lgamma(num)} = \Gamma(num)$
3. $\int_0^{phi} \frac{d\theta}{\sqrt{1-k^2\sin^2\theta}} = ellint_1(k,phi)$
4. $\int_0^{phi} \sqrt{1-k^2\sin^2\theta}d\theta = ellint_2(k,phi)$
5. $\int_{num}^{+\infty} \frac{e^{-t}}{t}dt = -expint(-num)$
6. $\sum_{n=1}^{+\infty} n^{-num} = riemann_zeta(num)$
7. $\frac{2}{\sqrt{\pi}}\int_0^{arg} e^{-t^2}dt = erf(arg)$

8. Personal Code Snippets

8.1 矩阵乘法与快速幂

```
struct matrix {
    int m[N][N];
};

matrix operator * (const matrix &a, const matrix &b) {
    matrix res;
    memset(res.m, 0, sizeof res.m);
    for (int i = 0; i < N; ++i)
        for (int j = 0; j < N; ++j)
            for (int k = 0; k < N; ++k)
                res.m[i][k] =
                    ((ll)a.m[i][j] * b.m[j][k] +
                      ↪ res.m[i][k]) % mod;

    return res;
}

matrix qmi(matrix &a, ll b) {
    matrix res;
    memset(res.m, 0, sizeof res.m);
    for (int i = 0; i < N; ++i)
        res.m[i][i] = 1;

    while (b) {
        if (b & 1) res = res * a;
        b >>= 1;
        a = a * a;
    }

    return res;
}
```

9. main(4) 补充材料

7.7 Hacks: 指令集、读入优化、Bitset、builtin、随机种子

```
// fast = O3 + ffast-math + fallow-store-data-races
#pragma GCC optimize("Ofast")
// Add after #include: bit operations, floating point, SIMD,
  ↪ registers
#pragma GCC target("abm,bmi,bmi2,fma,sse2,avx2,tune=native")

const int SZ = 1 << 16;
int getc() {
 | static char buf[SZ], *ptr = buf, *top = buf;
 | if (ptr == top) {
 |  | ptr = buf, top = buf + fread(buf, 1, SZ, stdin);
 |  | if (top == buf) return -1;
 | }
 | return *ptr++;
}

idx = b._Find_first();
idx != b.size();
idx = b._Find_next(idx);

struct HashFunc {
 | size_t operator()(const KEY &key) const {}
};

__builtin_uaddll_overflow(a, b, &c); // binary big int

void GospersHack(int k, int n) {
 | for (int s = (1 << k) - 1, c, r; s < (1 << n);
 |       c = s & -s, r = s + c,
 |       s = (((r ^ s) >> 2) / c) | r);
}

chrono::steady_clock::now().time_since_epoch().count();
```

7.8 试机赛与纪律文件

- **入场检查**：检查身份证件、胸牌。
- 检查手机、智能手表、金属物品（钥匙）等。
- 完整测试键盘、鼠标、显示器和座椅；热身赛发现问题时联系工作人员。
- **环境检查**：测试所有可能的提交方式，找到题面和排行榜的位置。
- 测试 `-fsanitize=address,undefined` 和 `-D_GLIBCXX_DEBUG`。
- 测试 `time` 命令能否显示内存占用：`/usr/bin/time -v ./a.out`。
- **OJ 检查**：测试编译器版本及特性：C++20 `cin >> (s + 1)`；C++17 结构化绑定 `auto [x,y]`；a；C++14 泛型 lambda；C++11 `auto`；`bits/stdc++.h`；`pb_ds`。

```
#include <ext/rope>
using namespace __gnu_cxx;
rope<int> R;
R.insert(y, x);
R[x];
R.erase(x, 1);

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
tree<int, null_type, less<int>, rb_tree_tag,
     tree_order_statistics_node_update> s;
s.insert(1);
s.find_by_order(0);
s.order_of_key(5);
```

- 测试 `__int128`、`__float128` 和 `long double`。
- 测试 `pragma target` 是否 CE；确认代码长度限制（如 256K），并尝试 NO-OUTPUT、OLE、MLE。
- 测试连续提交：`for i in 1..11; do pc2submit a.cpp -y; done`；确认是否有提交次数和频率限制。默认每分钟 10 次；若前一次提交尚未结束，下一次提交在队列中的优先级可能降低。
- 测试 `clock()` 是否正常工作，并比较本地性能与评测机性能。

```
const int N = 1 << 20;
for (int T = 4; T; T--) {
 | int a[N], b[N], c[N];
 | for (int i = 0; i < N; i++)
 |  | a[i] = i, b[i] = N - i;
 | ntt_init(N);
 | ntt(a, N, 1);
 | ntt(b, N, 1);
 | for (int i = 0; i < N; i++)
 |  | c[i] = a[i] * b[i];
 | ntt(c, N, -1);
 | assert(c[0] == 0xad223d);
}

// Machine room: 190 ms; 1 s permits T = 21
// QOJ: 340 +- 40 ms; 1 s permits T = 11.9
// Codeforces: 1050 +- 100 ms; 1 s permits T = 3.9
```

```
import sys

sys.set_int_max_str_digits(0)  # Python >= 3.11
T = 3
for _ in range(T):
    # Check for incorrect digit handling after a local assertion
      ↪ failure.
    assert str(3 ** 100000 * 10 ** 100000)[:9] == "133497141"

# Machine room PyPy: 130 ms; 1 s permits T = 39
# QOJ: 201 ms; 1 s permits T = 16
# Codeforces Python3: 1100 ms; 1 s permits T = 2.9
# Codeforces PyPy3: 310 ms; 1 s permits T = 12
```

提交前检查

1. 保存、编译、测过样例，测过 $n = 0/1$。
2. 检查数组大小 N/M、时间与空间限制、是否输入实数、MOD 的值，并确认是否需要开 `long long`。
3. 调用 `init`；清空数组时确认范围是 $(0, 1, n)$、m 还是 $n + 1$。
4. 输出取模到正值，并检查输出格式。
5. 多组数据没有读入完时就 `break`。
6. 关闭流同步时，不要混用 C 与 C++ 的输入输出。
7. 加上需要重点怀疑位置的 `assert`。
8. 开 `Ofast`；使用位运算时开 `target("abm")`。

提交后检查

1. 变量复用后是否忘记改回去。
2. `sum` 与多组输入：是否只使用了有效输入；清空范围是否正确（`memset` 可能 TL，错下标可能 WA）；应在用前还是用后清空。
3. `int/long long` 溢出，INF/−1 的大小，浮点 `eps` 与等于零，以及 `__builtin_popcountll`。
4. 类似 `pair<LL,int> x = pair<int,int>()` 的类型截断不会给出 warning。
5. 离散化与二分：`lower_bound/upper_bound` 的 ±1，以及 `begin/end`。
6. 自定义排序：排序方向、比较函数应表示“小于”，并考虑坏点，如叉积 $(0, 0)$。
7. 样例是否对称或回文；主动构造不对称、正序和逆序样例。
8. 复制代码后，是否正确修改了所有对应位置。
9. 检查 shadow（变量重名）。
10. 分类讨论中 `if` 的嵌套和 `break` 的位置。
11. 图：边编号是否从 1 开始、单双向边、反向边的边权。
12. 几何：共线、`sqrt(-0.0)`、NaN/Inf、重点、`det/dot` 除零、旋转方向。
13. 浮点数：`eps` 的量纲；至少取到“最短线段 × 精度要求”。
14. 二分、三分：迭代次数。

7.9 Constant Table 常数表

```
Random primes generated at Sat Mar 7 19:57:03 2026
5e2  373 379 389 409 449 463 467 479 487 491 499 521 557 607 617
1e3  877 881 911 919 947 983 1019 1039 1049 1091 1129 1153 1163
3e4  27967 28309 29311 29741 31079 31121 31181 31573 31699 32189
1e5  96269 98227 99469 99707 99839 100981 101363 105883 107069
1e6  948331 955991 960523 1013477 1024883 1028597 1064519 1068329
5e6  4901129 5035897 5094667 5115239 5234927 5240929 5299771
1e7  9554183 9737989 10016801 10046359 10056337 10470469 10564877
2e7  18761903 19644761 19893697 19994477 21055319 21088003
1e9  939524543 965847559 998808287 999326093 1054502167 1066250359
```

```
2e9  1895521417 1987317301 2011965253 2068107487 2131178887

NTT  976224257 r=3 (20)  985661441 r=3 (22)  998244353 r=3 (23)
     1004535809 r=3 (21) 1007681537 r=3 (20) 1012924417 r=5 (21)
     1045430273 r=3 (20) 1051721729 r=6 (20) 1053818881 r=7 (20)
```

n	$\log_{10} n$	$n!$	$C(n, n/2)$	$\mathrm{LCM}(1 \dots n)$	P_n
2	0.30102999	2	2	2	2
3	0.47712125	6	3	6	3
4	0.60205999	24	6	12	5
5	0.69897000	120	10	60	7
6	0.77815125	720	20	60	11
7	0.84509804	5040	35	420	15
8	0.90308998	40320	70	840	22
9	0.95424251	362880	126	2520	30
10	1	3628800	252	2520	42
11	1.04139269	39916800	462	27720	56
12	1.07918125	479001600	924	27720	77
15	1.17609126	1.31e12	6435	360360	176
20	1.30103000	2.43e18	184756	232792560	627
25	1.39794001	1.55e25	5200300	26771144400	1958
30	1.47712125	2.65e32	155117520	1.444e14	5604
P_n	37338_{40}	204226_{50}	966467_{60}	190569292_{100}	10^9_{114}

$n \le$	10	100	10^3	10^4	10^5	10^6
$\max \omega(n)$	2	3	4	5	6	7
$\max d(n)$	4	12	32	64	128	240
$\arg\max d$	6	60	840	7560	83160	720720
$\pi(n)$	4	25	168	1229	9592	78498

$n \le$	10^7	10^8	10^9	10^{10}	10^{11}	10^{12}
$\max \omega(n)$	8	8	9	10	10	11
$\max d(n)$	448	768	1344	2304	4032	6720
$\arg\max d$	8648640	73513440	735134400	6983776800	97772875200	963761198400
$\pi(n)$	664579	5761455	5.08e7	4.55e8	4.12e9	3.7e10

$n \le$	10^{13}	10^{14}	10^{15}	10^{16}	10^{17}	10^{18}
$\max \omega(n)$	12	12	13	13	14	15
$\max d(n)$	10752	17280	26880	41472	64512	103680
$\arg\max d$ at 10^{18}: $897612484786617600 = 2^8 3^4 5^2 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 37$						
$\pi(n)$	Prime number theorem: $\pi(x) \sim x/\log x$					

Bashrc

```
export CXXFLAGS='-g -Wall -Wextra -Wconversion -Wshadow
  ↪ -std=gnu++20'
ulimit -s 1048576
```